

Name: Roosc Zaño

Section: M006 - ITWM104

Professor: Rodrigo Belleza Jr.

Final Output

NSync

A Gamified Personal Knowledge and Review App

Brief Description of the Mobile Application

NSync is a gamified personal knowledge and memory application built with Kotlin and Jetpack Compose. It helps users capture important ideas, organize them into a knowledge base, create review cards, complete quiz-style review sessions, and track learning progress through XP, levels, streaks, accuracy, and mastery.

The first version of NSync was a mostly static prototype focused on matching the Figma interface. The current version connects the Android app to a Django REST Framework backend with PostgreSQL, JWT authentication, user-specific notes, flashcards, review sessions, profile data, progress tracking, and local settings.

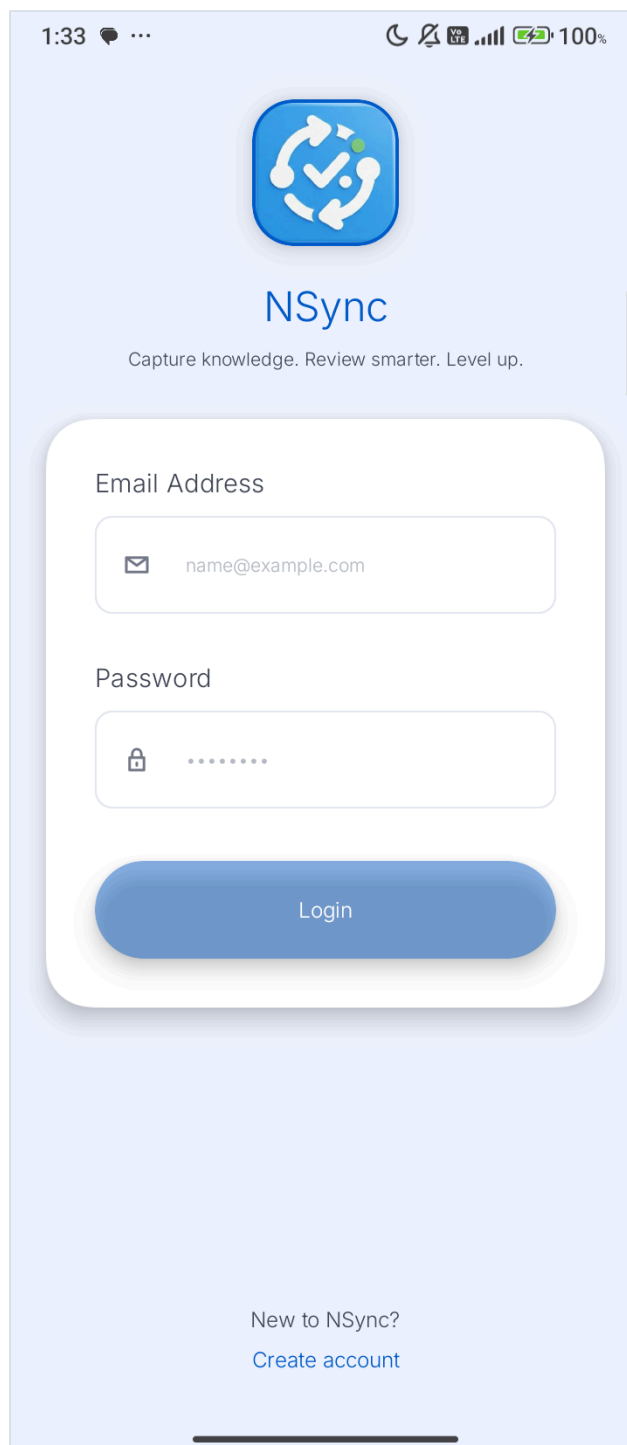
Application Requirements Checklist

Requirement	Implementation
Kotlin Android application	The Android app is built with Kotlin.
Jetpack Compose UI	Screens are implemented with Compose and Material 3 components.
At least 3 screens with navigation	The app includes auth, dashboard, knowledge, review, mastery, profile, and settings screens connected through Navigation Compose.
Multiple composable functions	Screens are split into screen-level composables and reusable component composables.
Separate Kotlin files by functionality	Screens, components, DTOs, repositories, state classes, ViewModels, navigation, notifications, and theme files are separated.
Scaffold layout	Scaffold is used in shared screen layouts and major screens with bottom navigation.
Save data locally or on cloud	Main app data is saved through Django/PostgreSQL; JWT session and settings are saved locally.

Screenshots of Actual Interface and Functionalities

Login Page

Allows an existing user to sign in using email and password. The app stores the returned JWT session for protected requests.



The screenshot shows the NSync login page on a mobile device. The status bar at the top displays the time 1:33, signal strength, and 100% battery. The app's logo, a blue square with a white circular arrow and a checkmark, is centered at the top. Below the logo is the text "NSync" and the tagline "Capture knowledge. Review smarter. Level up." The login form is a white rounded rectangle with a shadow. It contains two input fields: "Email Address" with a placeholder "name@example.com" and an envelope icon, and "Password" with a placeholder of seven dots and a lock icon. A blue "Login" button is positioned below the password field. At the bottom of the page, the text "New to NSync?" is followed by a blue link "Create account".

1:33

NSync

Capture knowledge. Review smarter. Level up.

Email Address

name@example.com

Password

.....

Login

New to NSync?

[Create account](#)

Register Page

Allows a new user to create an NSync account with a full name, email address, and password.

1:33

 100%

NSync

Create Account

Start building your personal knowledge base.

Full Name

Email Address

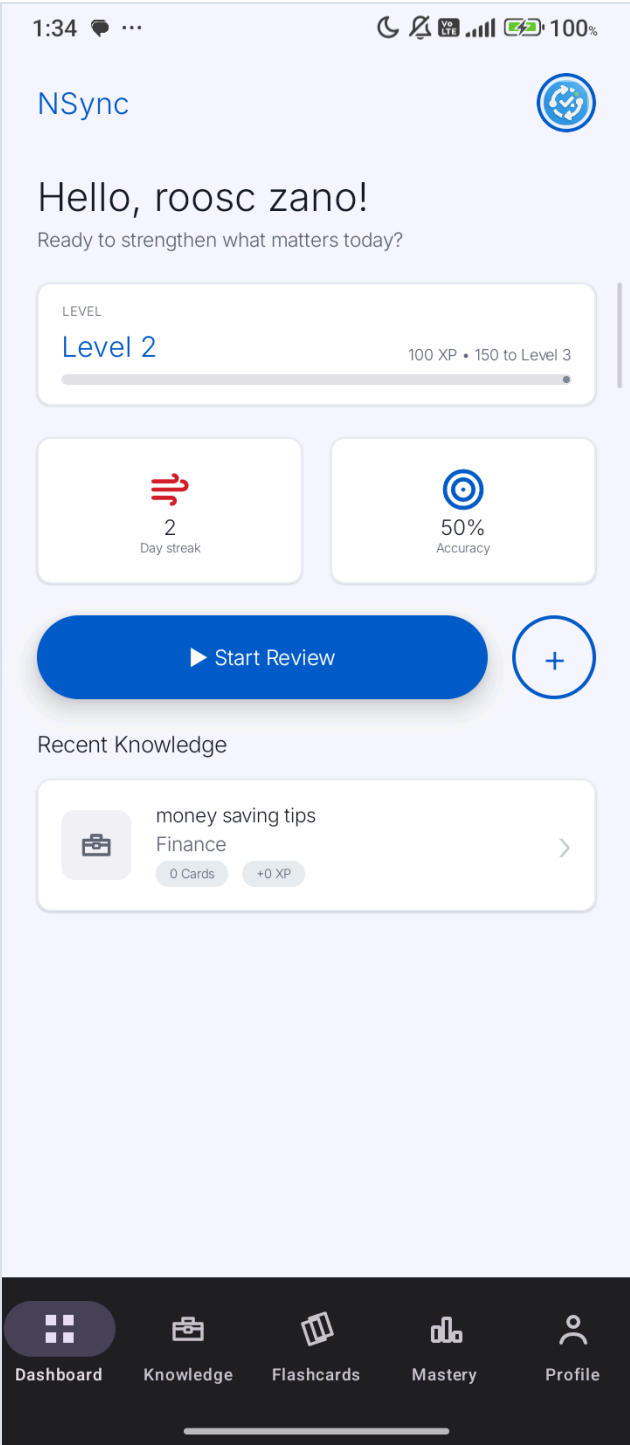
Password

Create Account

Already have an account? [Login](#)

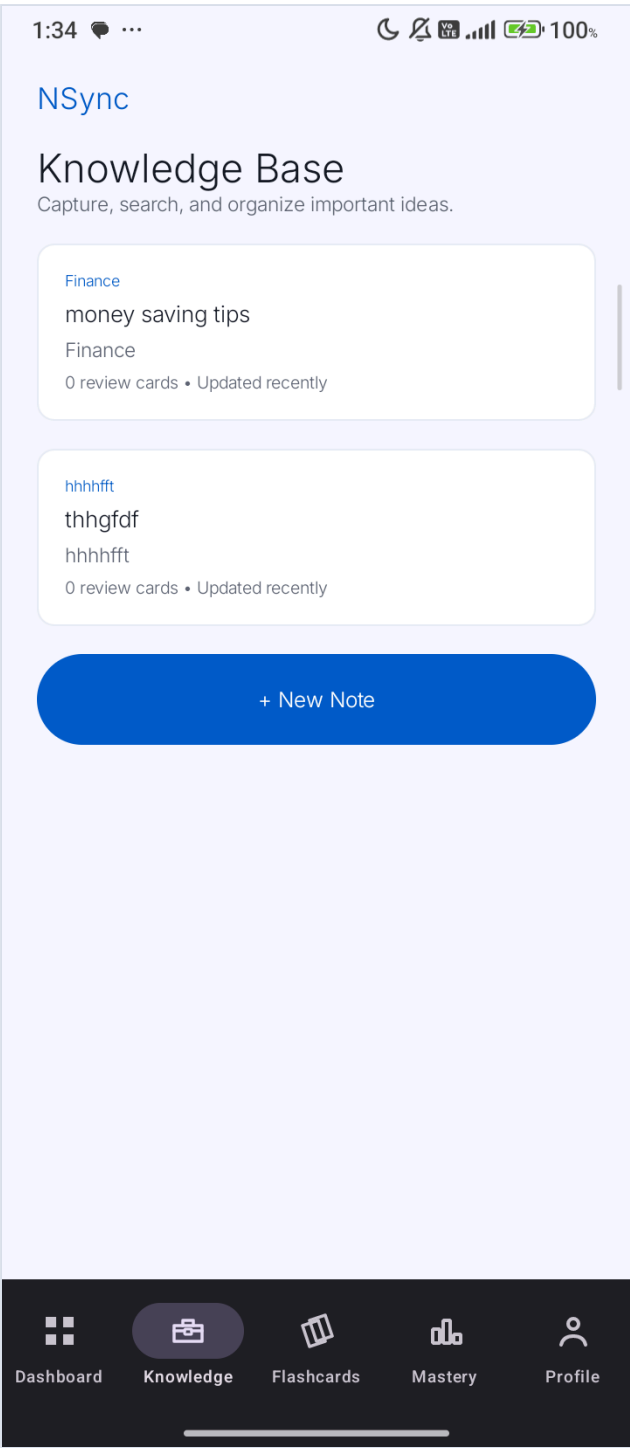
Dashboard Page

Shows the signed-in user greeting, level, XP, streak, accuracy, review action, and recent knowledge.



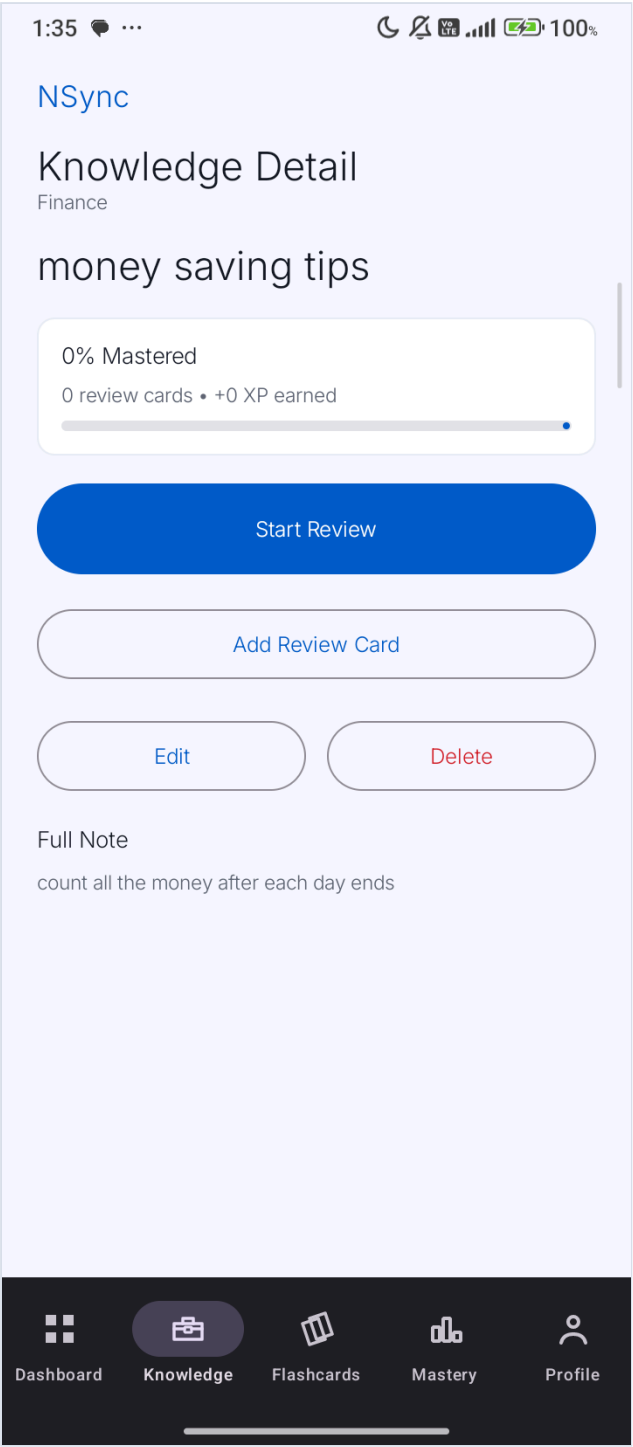
Knowledge Base Page

Lists saved notes from the backend and lets the user create a new note.



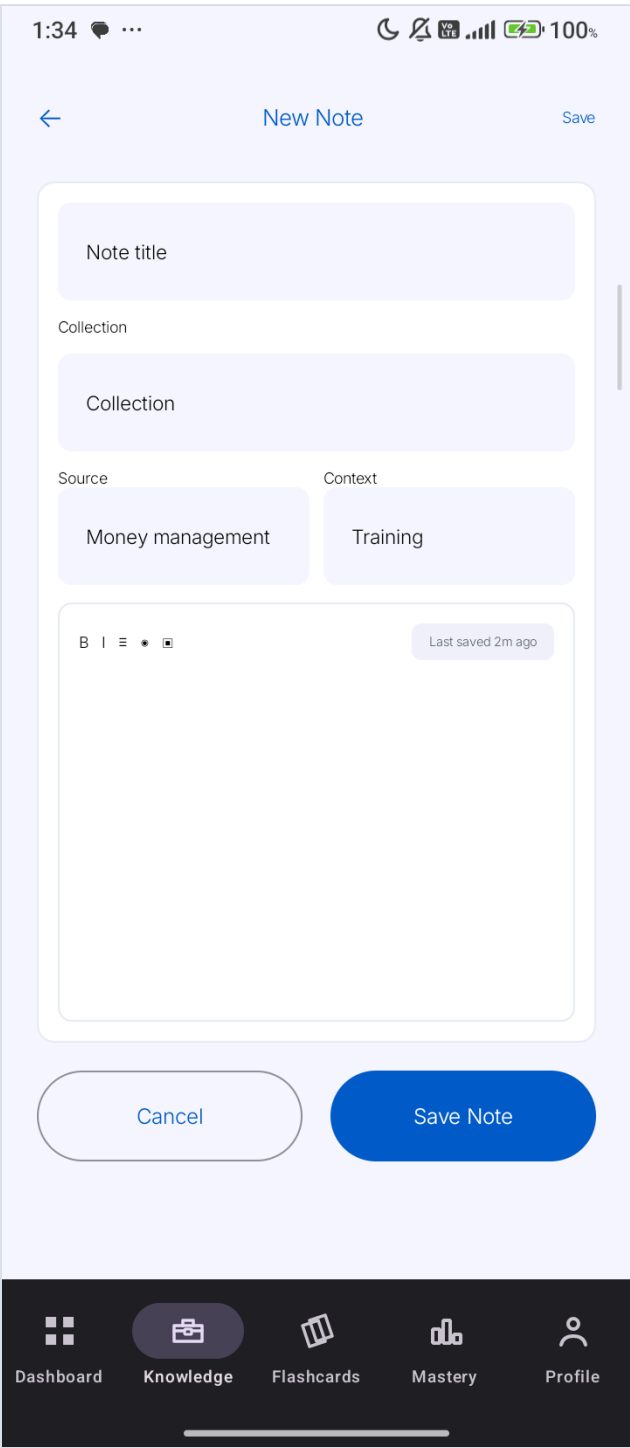
Knowledge Detail Page

Displays the selected note, mastery summary, review-card count, full content, and note actions.



New Note Page

Lets the user create a note with title, collection, source, context, and note content.



New Review Card Page

Lets the user create a flashcard connected to a knowledge note.

1:35

New Review Card

Create a question and answer for this knowledge item.

Question

Answer

Difficulty

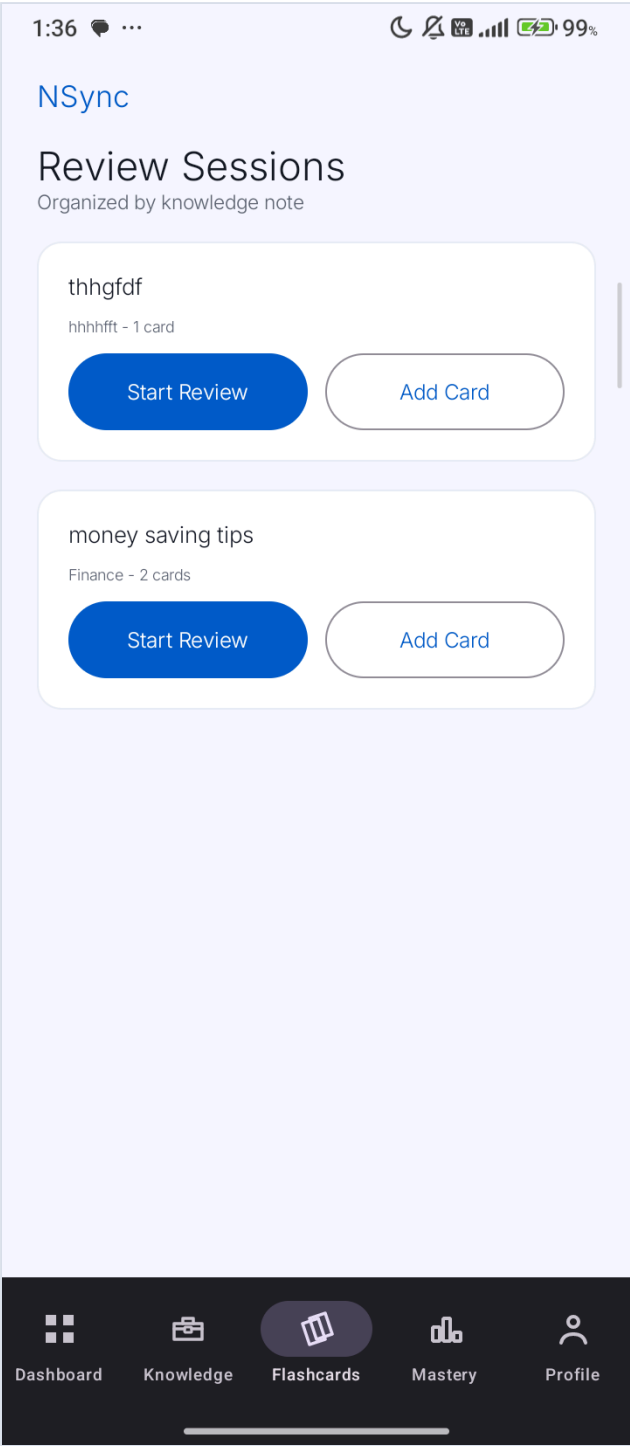
Easy

Cancel

Save Card

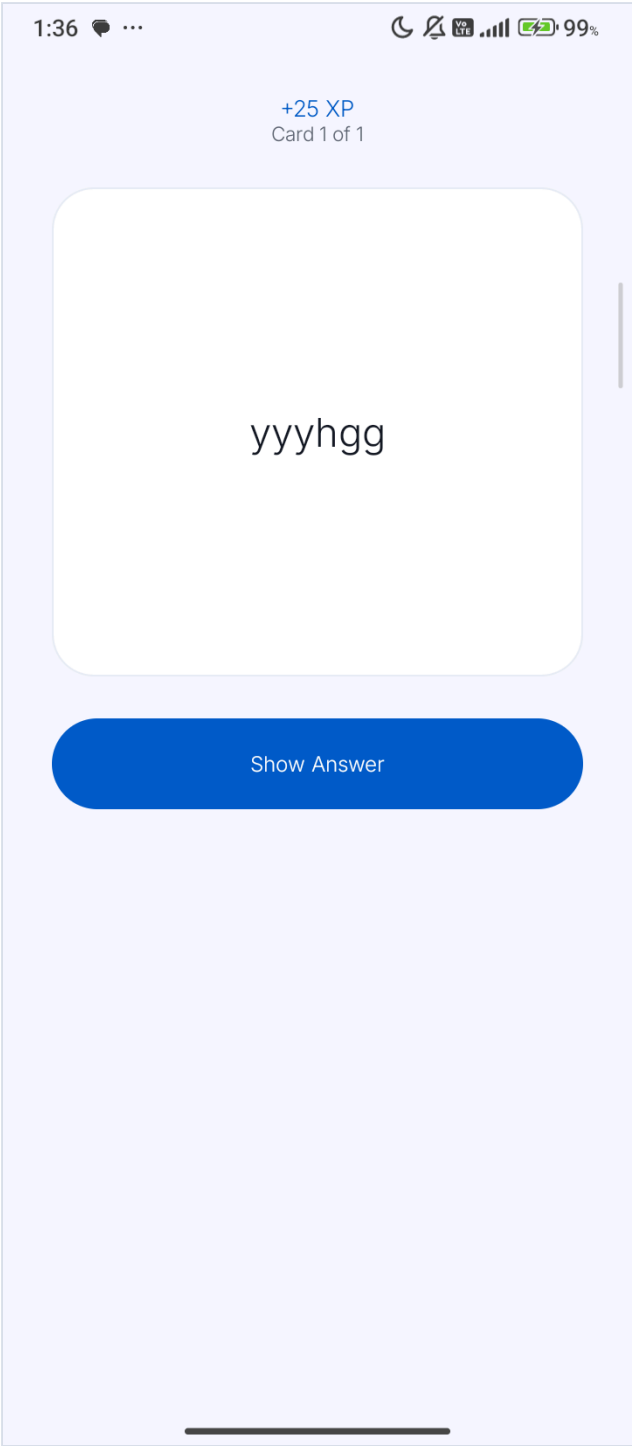
Review Sessions Page

Groups flashcards into review sessions by connected knowledge note.



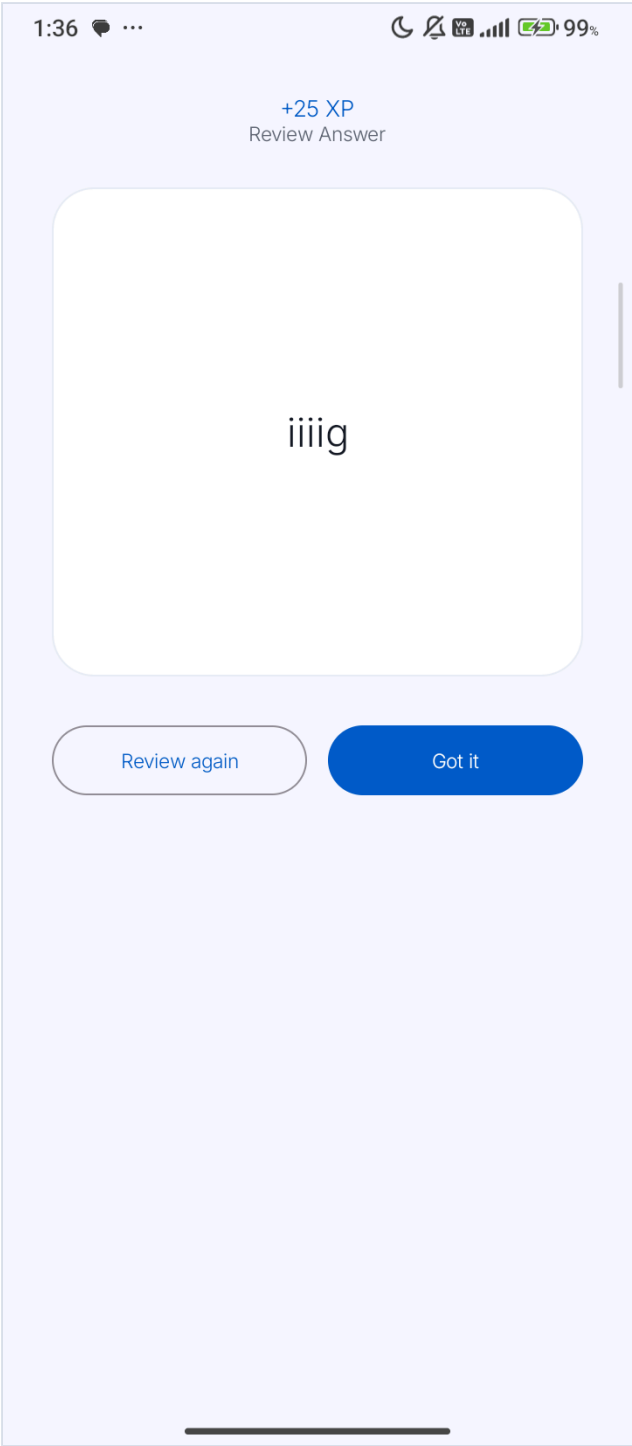
Review Session Question Page

Shows the current review question and lets the user reveal the answer.



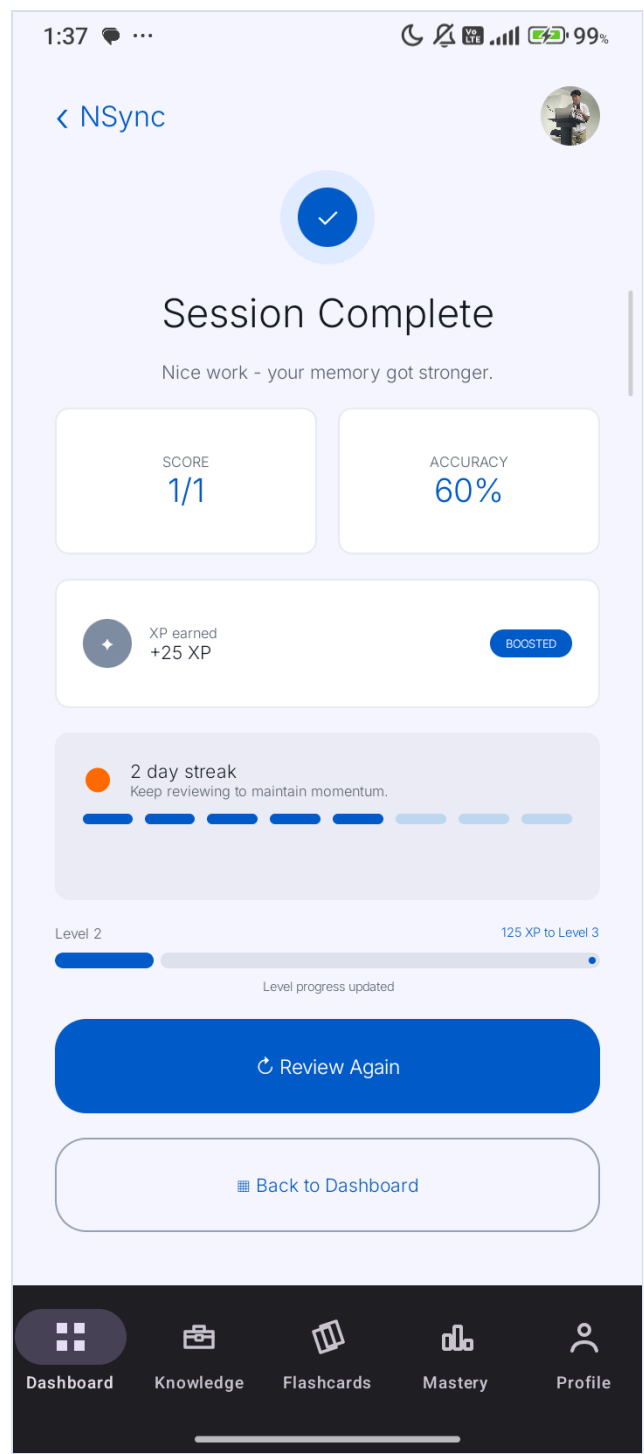
Review Session Answer Page

Shows the answer and lets the user rate recall using Review again or Got it.



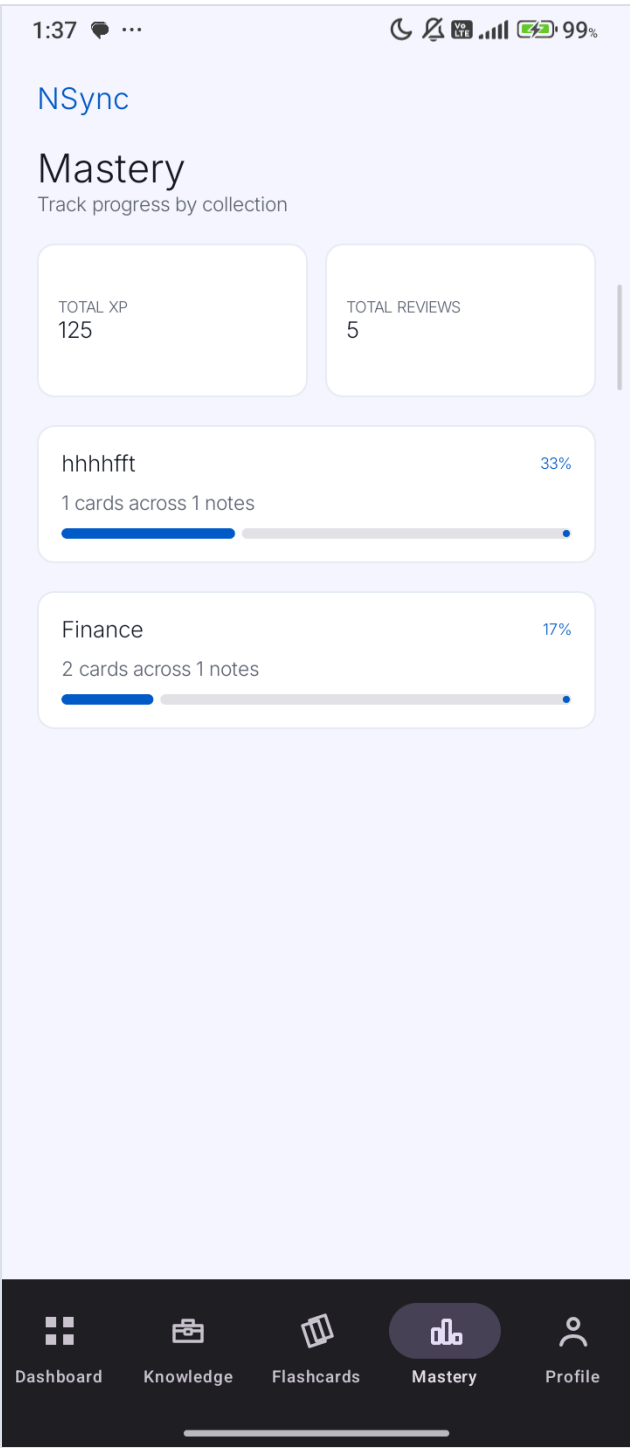
Session Complete Page

Shows score, accuracy, XP earned, streak, level progress, and review/dashboard actions.



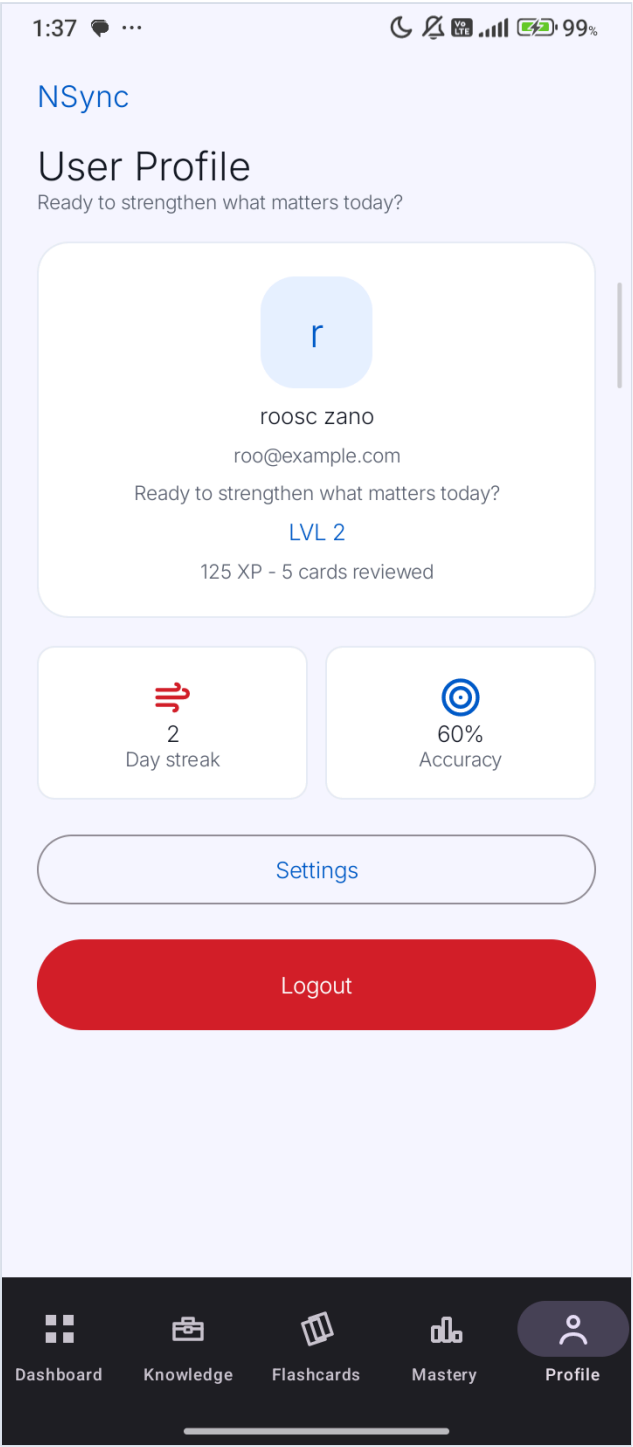
Mastery Page

Shows total XP, total reviews, and mastery progress grouped by collection.



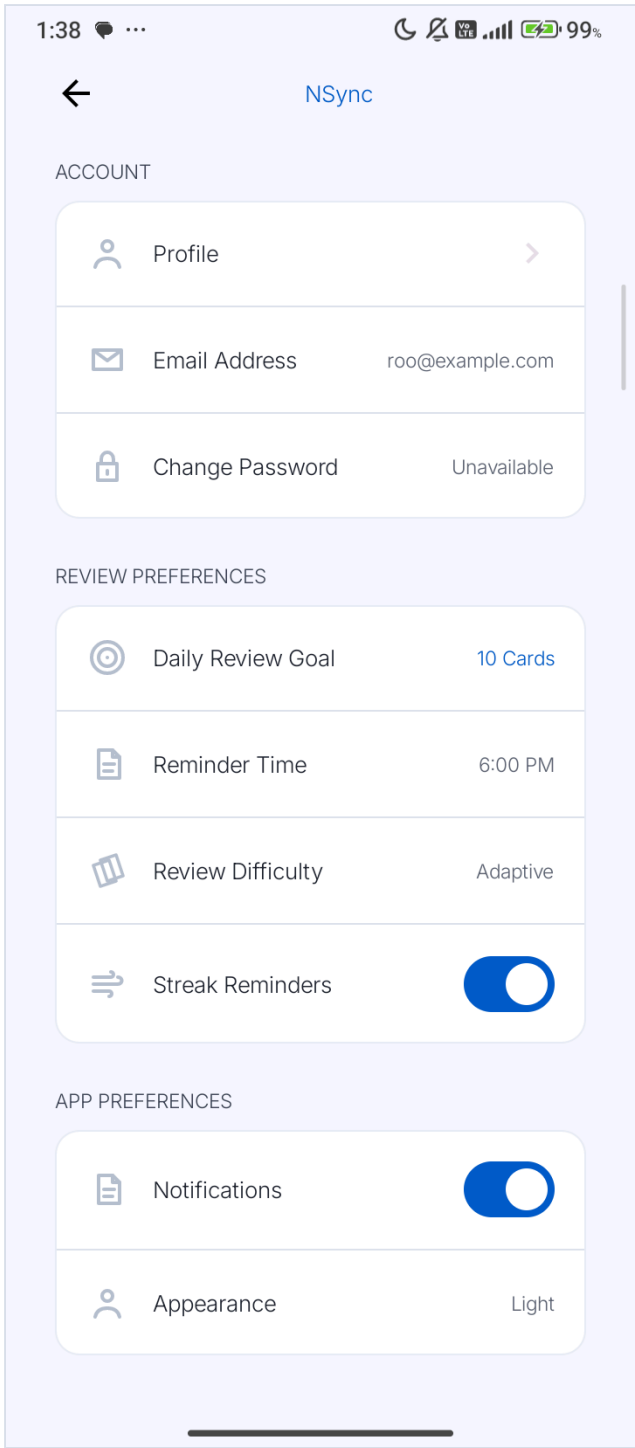
Profile Page

Shows current user identity, level, XP, cards reviewed, streak, accuracy, settings, and logout.



Settings Page

Shows account information and review/app preferences stored locally.



Jetpack Compose Components Used

Compose API	Purpose and Usage
@Composable	Marks functions that describe UI.
Scaffold	Provides a screen layout structure for content and bottom navigation.
NavHost and composable	Define the navigation graph and each screen route.
rememberNavController	Creates the navigation controller used by the app.
LaunchedEffect	Runs side effects such as initial loading and navigation after state changes.
DisposableEffect	Registers and removes lifecycle observers.
remember / rememberSaveable	Stores UI state across recompositions and configuration changes.
mutableStateOf / mutableIntStateOf	Creates observable state that updates the UI.
Column, Row, Box	Lays out UI vertically, horizontally, or in layers.
LazyColumn	Displays scrollable lists such as notes and sessions.
Text	Displays labels, headings, and values.
Button / OutlinedButton	Displays filled and outlined actions.
OutlinedTextField	Accepts user input in forms.
Card	Groups related content visually.
Icon / Image	Displays vector icons and bitmap images.
NavigationBar / NavigationBarItem	Provides bottom tab navigation.
DropDownMenu / DropdownMenuItem	Displays selectable settings options.
Switch	Controls boolean preferences such as notifications.
LinearProgressIndicator	Shows XP and mastery progress.
Modifier	Applies size, padding, background, clipping, click behavior, and layout rules.
MaterialTheme	Applies shared Material colors and typography.

Source Codes + Descriptions

This section follows the source-code format from the previous output: each file has its file name, path, short purpose, and actual source code.

Android Kotlin Source Code

MainActivity.kt

Path: mobile/app/src/main/java/com/example/mobile/MainActivity.kt

Purpose: Application entry point. Initializes Retrofit networking, applies the Compose theme, and loads the navigation graph.

```
0001 package com.example.mobile
0002
0003 import android.os.Bundle
0004 import androidx.activity.ComponentActivity
0005 import androidx.activity.compose.setContent
0006 import androidx.activity.enableEdgeToEdge
0007 import com.example.mobile.data.remote.RetrofitClient
0008 import com.example.mobile.navigation.AppNavigation
0009 import com.example.mobile.ui.theme.MobileTheme
0010
0011 class MainActivity : ComponentActivity() {
0012     override fun onCreate(savedInstanceState: Bundle?) {
0013         super.onCreate(savedInstanceState)
0014         RetrofitClient.initialize(applicationContext)
0015         enableEdgeToEdge()
0016         setContent {
0017             MobileTheme {
0018                 AppNavigation()
0019             }
0020         }
0021     }
0022 }
```

Models.kt

Path: mobile/app/src/main/java/com/example/mobile/data/Models.kt

Purpose: Defines UI-domain models used by screens and ViewModels.

```
0001 package com.example.mobile.data
0002
0003 data class UserProfile(
0004     val name: String,
0005     val email: String,
0006     val learningGoal: String,
0007     val level: Int,
0008     val totalXp: Int,
0009     val xpToNextLevel: Int,
0010     val streakDays: Int,
0011     val accuracyPercent: Int
0012 )
0013
0014 data class KnowledgeItem(
0015     val id: Int,
0016     val title: String,
0017     val collection: String,
0018     val source: String,
0019     val context: String,
0020     val summary: String,
0021     val fullNote: String,
0022     val reviewCardCount: Int,
0023     val updatedLabel: String,
0024     val masteryPercent: Int,
0025     val xpEarned: Int
0026 )
0027
0028 data class ReviewCard(
0029     val id: Int,
0030     val knowledgeItemId: Int,
0031     val collection: String,
0032     val question: String,
0033     val answer: String,
0034     val difficulty: String,
0035     val masteryLabel: String,
0036     val masteryPercent: Int,
0037     val updatedLabel: String,
0038     val sourceNoteTitle: String = collection
0039 )
0040
0041 data class CollectionMastery(
0042     val collection: String,
0043     val context: String,
0044     val masteryPercent: Int
0045 )
0046
0047 data class RecentAttempt(
0048     val title: String,
0049     val timeLabel: String,
0050     val score: String,
0051     val rank: String
0052 )
0053
0054 data class ReviewSessionSummary(
0055     val score: String,
0056     val accuracyPercent: Int,
0057     val xpEarned: Int,
0058     val streakDays: Int,
0059     val levelLabel: String,
0060     val xpToNextLevel: Int
0061 )
0062
0063 data class LoginRequest(
0064     val email: String,
0065     val name: String // Backend User model only has name and email for now
0066 )
0067
0068 data class RegisterRequest(
0069     val email: String,
0070     val name: String
0071 )
```

SampleData.kt

Path: mobile/app/src/main/java/com/example/mobile/data/SampleData.kt

Purpose: Keeps fallback/demo values used where backend data is not required.

```
0001 package com.example.mobile.data
0002
0003 object SampleData {
0004     val userProfile = UserProfile(
0005         name = "Roosc",
0006         email = "zanoroosc@gmail.com",
0007         learningGoal = "Build durable knowledge",
0008         level = 5,
0009         totalXp = 2450,
0010         xpToNextLevel = 550,
0011         streakDays = 12,
0012         accuracyPercent = 92
0013     )
0014
0015     val knowledgeItems = listOf(
0016         KnowledgeItem(
0017             id = 1,
0018             title = "Building a Simple Monthly Budget",
0019             collection = "Personal Finance",
0020             source = "Money management notes",
0021             context = "Money management notes",
0022             summary = "A monthly budget helps organize income, expenses, savings, and spending priorities.",
0023             fullNote = ""
0024             A monthly budget helps organize income, expenses, savings, and spending priorities. It gives a clear view of where money
0025             goes and helps prevent unnecessary overspending.
0026             Management Skills
0027             A simple budget can divide money into needs, wants, savings, and debt payments. This makes financial decisions easier
0028             because each peso has a purpose.
0029             Reviewing a budget regularly helps identify spending habits, prepare for upcoming expenses, and adjust goals when income
0030             or priorities change.
0031             """trimIndent(),
0032             reviewCardCount = 6,
0033             updatedLabel = "Updated yesterday",
0034             masteryPercent = 80,
0035             xpEarned = 120
0036         ),
0037         KnowledgeItem(
0038             id = 2,
0039             title = "Django Rest Framework",
0040             collection = "Programming",
0041             source = "Django Programming",
0042             context = "Backend reference",
0043             summary = "Django REST Framework helps build API endpoints for web and mobile applications.",
0044             fullNote = ""
0045             Django REST Framework is a toolkit for building Web APIs using Django. It provides serializers, viewsets, routers,
0046             authentication helpers, and browsable API tools.
0047             Serializers convert Django model instances into JSON-friendly data. Views and viewsets handle incoming requests and return
0048             API responses.
0049             DRF is useful for connecting a Django backend to a mobile app because the Android client can request and send structured
0050             data through API endpoints.
0051             """trimIndent(),
0052             reviewCardCount = 5,
0053             updatedLabel = "Updated today",
0054             masteryPercent = 72,
0055             xpEarned = 85
0056         ),
0057         KnowledgeItem(
0058             id = 3,
0059             title = "Common Interview Questions",
0060             collection = "Interview Prep",
0061             source = "Personal career notes",
0062             context = "Job preparation",
0063             summary = "Common interview questions help prepare clear answers about skills, projects, and work habits.",
0064             fullNote = ""
0065             Interview preparation helps organize answers before the actual interview. Strong answers are specific, honest, and
0066             connected to real project experience.
0067             Technical interview questions often test core concepts, problem solving, and communication. Behavioral questions focus on
0068             teamwork, responsibility, conflict, and learning from mistakes.
0069             A good answer explains the situation, action, and result without sounding memorized.
0070             """trimIndent(),
0071             reviewCardCount = 8,
0072             updatedLabel = "Updated this week",
0073             masteryPercent = 64,
0074             xpEarned = 64
0075         )
0076     )
0077
0078     val reviewCards = listOf(
0079         ReviewCard(
0080             id = 1,
0081             knowledgeItemId = 1,
0082             collection = "Personal Finance",
0083             question = "What is the purpose of a monthly budget?",
0084             answer = "A monthly budget helps track income, expenses, savings, and spending so money is managed intentionally.",
0085             difficulty = "Easy",
0086             masteryLabel = "Bronze Mastery",
0087             masteryPercent = 52,
0088             updatedLabel = "Edited today"
0089         ),
0090         ReviewCard(
0091             id = 2,
```

```

0091         knowledgeItemId = 2,
0092         collection = "Programming",
0093         question = "What does Django REST Framework help you build?",
0094         answer = "Django REST Framework helps build API endpoints that let web or mobile clients send and receive structured data.",
0095         difficulty = "Medium",
0096         masteryLabel = "Gold Mastery",
0097         masteryPercent = 80,
0098         updatedLabel = "Edited yesterday"
0099     ),
0100     ReviewCard(
0101         id = 3,
0102         knowledgeItemId = 3,
0103         collection = "Interview Prep",
0104         question = "What makes a strong interview answer?",
0105         answer = "A strong interview answer is specific, honest, and connected to real project experience.",
0106         difficulty = "Easy",
0107         masteryLabel = "Silver Mastery",
0108         masteryPercent = 72,
0109         updatedLabel = "Edited this week"
0110     ),
0111     ReviewCard(
0112         id = 4,
0113         knowledgeItemId = 2,
0114         collection = "Programming",
0115         question = "What is the purpose of a Django model?",
0116         answer = "A Django model defines the structure and behavior of data stored in the database.",
0117         difficulty = "Medium",
0118         masteryLabel = "Silver Mastery",
0119         masteryPercent = 72,
0120         updatedLabel = "Edited today"
0121     ),
0122     ReviewCard(
0123         id = 5,
0124         knowledgeItemId = 3,
0125         collection = "Interview Prep",
0126         question = "What is the difference between authentication and authorization?",
0127         answer = "Authentication verifies identity, while authorization determines what actions or resources a user can access.",
0128         difficulty = "Medium",
0129         masteryLabel = "Bronze Mastery",
0130         masteryPercent = 60,
0131         updatedLabel = "Edited this week"
0132     )
0133 )
0134
0135 val collectionMastery = listOf(
0136     CollectionMastery(
0137         collection = "Personal Finance",
0138         context = "Money management",
0139         masteryPercent = 88
0140     ),
0141     CollectionMastery(
0142         collection = "Programming",
0143         context = "Backend development",
0144         masteryPercent = 72
0145     ),
0146     CollectionMastery(
0147         collection = "Interview Prep",
0148         context = "Career preparation",
0149         masteryPercent = 64
0150     ),
0151     CollectionMastery(
0152         collection = "Writing Skills",
0153         context = "Communication",
0154         masteryPercent = 58
0155     )
0156 )
0157
0158 val recentAttempts = listOf(
0159     RecentAttempt(
0160         title = "Finance Review Session",
0161         timeLabel = "2 hours ago",
0162         score = "9/10",
0163         rank = "Gold"
0164     ),
0165     RecentAttempt(
0166         title = "Django API Review",
0167         timeLabel = "Yesterday",
0168         score = "7/10",
0169         rank = "Silver"
0170     ),
0171     RecentAttempt(
0172         title = "Interview Prep Sprint",
0173         timeLabel = "3 days ago",
0174         score = "80%",
0175         rank = "Review"
0176     )
0177 )
0178
0179 val sessionSummary = ReviewSessionSummary(
0180     score = "4/5",
0181     accuracyPercent = 80,
0182     xpEarned = 125,
0183     streakDays = 13,
0184     levelLabel = "Level 5",
0185     xpToNextLevel = 675
0186 )
0187
0188 val reviewSessionCard = reviewCards.first()
0189
0190 val dashboardRecentKnowledge = knowledgeItems.first().copy(
0191     reviewCardCount = 8,
0192     xpEarned = 64
0193 )
0194
0195 val detailKnowledge = knowledgeItems.first().copy(
0196     collection = "Finance",
0197     reviewCardCount = 12,
0198     masteryPercent = 80,
0199     xpEarned = 120
0200 )
0201
0202 val draftReviewCardQuestion = "What is the purpose of a monthly budget?"

```

```
0203
0204     val draftReviewCardAnswer =
0205         "A monthly budget helps track income, expenses, savings, and spending so money is managed intentionally."
0206
0207     val collections = listOf(
0208         "All",
0209         "Personal Finance",
0210         "Programming",
0211         "Interview Prep",
0212         "Writing Skills"
0213     )
0214 }
```

AuthSession.kt

Path: mobile/app/src/main/java/com/example/mobile/data/local/AuthSession.kt

Purpose: Represents the locally stored authenticated user session.

```
0001 package com.example.mobile.data.local
0002
0003 data class AuthSession(
0004     val accessToken: String,
0005     val refreshToken: String,
0006     val userId: Long,
0007     val displayName: String,
0008     val email: String,
0009 )
```


AuthSessionStore.kt

Path: mobile/app/src/main/java/com/example/mobile/data/local/AuthSessionStore.kt

Purpose: Reads, saves, updates, and clears JWT session values using DataStore Preferences.

```
0001 package com.example.mobile.data.local
0002
0003 import android.content.Context
0004 import androidx.datastore.core.DataStore
0005 import androidx.datastore.preferences.core.Preferences
0006 import androidx.datastore.preferences.core.edit
0007 import androidx.datastore.preferences.core.emptyPreferences
0008 import androidx.datastore.preferences.core.longPreferencesKey
0009 import androidx.datastore.preferences.core.stringPreferencesKey
0010 import androidx.datastore.preferences.preferencesDataStore
0011 import java.io.IOException
0012 import kotlinx.coroutines.flow.Flow
0013 import kotlinx.coroutines.flow.catch
0014 import kotlinx.coroutines.flow.map
0015
0016 private val Context.dataStore: DataStore<Preferences> by preferencesDataStore(
0017     name = "nsync_auth_session",
0018 )
0019
0020 class AuthSessionStore(context: Context) {
0021     private val appContext = context.applicationContext
0022
0023     private companion object {
0024         val AccessTokenKey = stringPreferencesKey("access_token")
0025         val RefreshTokenKey = stringPreferencesKey("refresh_token")
0026         val UserIdKey = longPreferencesKey("user_id")
0027         val DisplayNameKey = stringPreferencesKey("display_name")
0028         val EmailKey = stringPreferencesKey("email")
0029     }
0030
0031     private val preferencesFlow: Flow<Preferences> = appContext.dataStore.data
0032         .catch { exception ->
0033             if (exception is IOException) {
0034                 emit(emptyPreferences())
0035             } else {
0036                 throw exception
0037             }
0038         }
0039
0040     val accessTokenFlow: Flow<String?> = preferencesFlow.map { preferences ->
0041         preferences[AccessTokenKey]
0042     }
0043
0044     val refreshTokenFlow: Flow<String?> = preferencesFlow.map { preferences ->
0045         preferences[RefreshTokenKey]
0046     }
0047
0048     val sessionFlow: Flow<AuthSession?> = preferencesFlow.map { preferences ->
0049         val accessToken = preferences[AccessTokenKey]
0050         val refreshToken = preferences[RefreshTokenKey]
0051         val userId = preferences[UserIdKey]
0052         val displayName = preferences[DisplayNameKey]
0053         val email = preferences[EmailKey]
0054
0055         if (
0056             accessToken != null && refreshToken != null && userId != null &&
0057             displayName != null && email != null
0058         ) {
0059             AuthSession(accessToken, refreshToken, userId, displayName, email)
0060         } else {
0061             null
0062         }
0063     }
0064
0065     val isSignedIn: Flow<Boolean> = sessionFlow.map { it != null }
0066
0067     suspend fun saveSession(session: AuthSession) {
0068         appContext.dataStore.edit { preferences ->
0069             preferences[AccessTokenKey] = session.accessToken
0070             preferences[RefreshTokenKey] = session.refreshToken
0071             preferences[UserIdKey] = session.userId
0072             preferences[DisplayNameKey] = session.displayName
0073             preferences[EmailKey] = session.email
0074         }
0075     }
0076
0077     suspend fun updateTokens(accessToken: String, refreshToken: String) {
0078         appContext.dataStore.edit { preferences ->
0079             preferences[AccessTokenKey] = accessToken
0080             preferences[RefreshTokenKey] = refreshToken
0081         }
0082     }
0083
0084     suspend fun clearSession() {
0085         appContext.dataStore.edit { preferences ->
0086             preferences.remove(AccessTokenKey)
0087             preferences.remove(RefreshTokenKey)
0088             preferences.remove(UserIdKey)
0089             preferences.remove(DisplayNameKey)
0090             preferences.remove(EmailKey)
0091         }
0092     }
0093 }
```

RetrofitClient.kt

Path: mobile/app/src/main/java/com/example/mobile/data/remote/RetrofitClient.kt

Purpose: Creates Retrofit, OkHttp, Gson, API service, session store, and auth interceptor instances.

```
0001 package com.example.mobile.data.remote
0002
0003 import android.content.Context
0004 import com.example.mobile.data.local.AuthSessionStore
0005 import com.example.mobile.data.remote.api.ApiService
0006 import com.example.mobile.data.remote.interceptor.AuthInterceptor
0007 import okhttp3.OkHttpClient
0008 import retrofit2.Retrofit
0009 import retrofit2.converter.gson.GsonConverterFactory
0010
0011 object RetrofitClient {
0012     private const val BASE_URL = "http://127.0.0.1:8000/"
0013
0014     private lateinit var initializedSessionStore: AuthSessionStore
0015     private lateinit var initializedApiService: ApiService
0016
0017     @Synchronized
0018     fun initialize(context: Context) {
0019         if (::initializedApiService.isInitialized) {
0020             return
0021         }
0022
0023         initializedSessionStore = AuthSessionStore(context.applicationContext)
0024         val okHttpClient = OkHttpClient.Builder()
0025             .addInterceptor(AuthInterceptor(initializedSessionStore))
0026             .build()
0027
0028         initializedApiService = Retrofit.Builder()
0029             .baseUrl(BASE_URL)
0030             .client(okHttpClient)
0031             .addConverterFactory(GsonConverterFactory.create())
0032             .build()
0033             .create(ApiService::class.java)
0034     }
0035
0036     val sessionStore: AuthSessionStore
0037     get() {
0038         check(::initializedSessionStore.isInitialized) {
0039             "RetrofitClient.initialize must be called before accessing the session store."
0040         }
0041         return initializedSessionStore
0042     }
0043
0044     val apiService: ApiService
0045     get() {
0046         check(::initializedApiService.isInitialized) {
0047             "RetrofitClient.initialize must be called before accessing the API service."
0048         }
0049         return initializedApiService
0050     }
0051 }
```

ApiService.kt

Path: mobile/app/src/main/java/com/example/mobile/data/remote/api/ApiService.kt

Purpose: Defines Retrofit endpoints for authentication, notes, flashcards, review completion, and progress.

```
0001 package com.example.mobile.data.remote.api
0002
0003 import com.example.mobile.data.remote.dto.CreateNoteRequestDto
0004 import com.example.mobile.data.remote.dto.CreateFlashcardRequestDto
0005 import com.example.mobile.data.remote.dto.FlashcardDto
0006 import com.example.mobile.data.remote.dto.NoteDto
0007 import com.example.mobile.data.remote.dto.ReviewCompleteRequestDto
0008 import com.example.mobile.data.remote.dto.ReviewCompleteResponseDto
0009 import com.example.mobile.data.remote.dto.UpdateNoteRequestDto
0010 import com.example.mobile.data.remote.dto.UpdateFlashcardRequestDto
0011 import com.example.mobile.data.remote.dto.UserProgressDto
0012 import com.example.mobile.data.remote.dto.AuthResponseDto
0013 import com.example.mobile.data.remote.dto.RefreshRequestDto
0014 import com.example.mobile.data.remote.dto.RefreshResponseDto
0015 import com.example.mobile.data.remote.dto.AuthenticatedUserDto
0016 import com.example.mobile.data.remote.dto.LoginRequestDto
0017 import com.example.mobile.data.remote.dto.RegisterRequestDto
0018 import retrofit2.Response
0019 import retrofit2.http.Body
0020 import retrofit2.http.GET
0021 import retrofit2.http.POST
0022 import retrofit2.http.PUT
0023 import retrofit2.http.DELETE
0024 import retrofit2.http.Path
0025
0026 interface ApiService {
0027     // Notes API endpoints
0028     // Get notes
0029     @GET("/api/notes/")
0030     suspend fun getNotes(): Response<List<NoteDto>>
0031     @GET("/api/notes/{id}/")
0032     suspend fun getNoteById(@Path("id") id: Int): Response<NoteDto>
0033
0034     // Create note
0035     @POST("/api/notes/")
0036     suspend fun createNote(@Body note: CreateNoteRequestDto): Response<NoteDto>
0037
0038     @PUT("/api/notes/{id}/")
0039     suspend fun updateNote(
0040         @Path("id") id: Int,
0041         @Body note: UpdateNoteRequestDto
0042     ): Response<NoteDto>
0043
0044     // Delete note
0045     @DELETE("/api/notes/{id}/")
0046     suspend fun deleteNote(@Path("id") id: Int): Response<Unit>
0047
0048     // Flashcards API endpoints
0049     // Get flashcards
0050     @GET("/api/flashcards/")
0051     suspend fun getFlashcards(): Response<List<FlashcardDto>>
0052     // Get flashcard by ID
0053     @GET("/api/flashcards/{id}/")
0054     suspend fun getFlashcardById(@Path("id") id: Int): Response<FlashcardDto>
0055
0056     @POST("/api/flashcards/")
0057     suspend fun createFlashcard(
0058         @Body flashcard: CreateFlashcardRequestDto
0059     ): Response<FlashcardDto>
0060
0061     @PUT("/api/flashcards/{id}/")
0062     suspend fun updateFlashcard(
0063         @Path("id") id: Int,
0064         @Body flashcard: UpdateFlashcardRequestDto
0065     ): Response<FlashcardDto>
0066
0067     @DELETE("/api/flashcards/{id}/")
0068     suspend fun deleteFlashcard(@Path("id") id: Int): Response<Unit>
0069
0070     @POST("/api/review/complete/")
0071     suspend fun completeReview(
0072         @Body review: ReviewCompleteRequestDto
0073     ): Response<ReviewCompleteResponseDto>
0074
0075     @GET("/api/progress/")
0076     suspend fun getProgress(): Response<UserProgressDto>
0077
0078     @POST("/api/auth/register/")
0079     suspend fun register(@Body registerRequest: RegisterRequestDto): Response<AuthResponseDto>
0080
0081     @POST("/api/auth/login/")
0082     suspend fun login(@Body loginRequest: LoginRequestDto): Response<AuthResponseDto>
0083
0084     @POST("/api/auth/refresh/")
0085     suspend fun refreshToken(@Body refreshRequest: RefreshRequestDto): Response<RefreshResponseDto>
0086
0087     @POST("/api/auth/logout/")
0088     suspend fun logout(@Body refreshRequest: RefreshRequestDto): Response<Unit>
0089
0090     @GET("/api/auth/me/")
0091     suspend fun getAuthenticatedUser(): Response<AuthenticatedUserDto>
0092
0093 }
```

AuthDtos.kt

Path: mobile/app/src/main/java/com/example/mobile/data/remote/dto/AuthDtos.kt

Purpose: Defines request and response DTOs for register, login, refresh, logout, and current-user data.

```
0001 package com.example.mobile.data.remote.dto
0002
0003 import com.google.gson.annotations.SerializedName
0004
0005 data class RegisterRequestDto(
0006     @SerializedName("display_name") val displayName: String,
0007     @SerializedName("email") val email: String,
0008     @SerializedName("password") val password: String
0009 )
0010
0011 data class LoginRequestDto(
0012     @SerializedName("email") val email: String,
0013     @SerializedName("password") val password: String
0014 )
0015
0016 data class AuthenticatedUserDto(
0017     @SerializedName("id") val id: Long,
0018     @SerializedName("email") val email: String,
0019     @SerializedName("display_name") val displayName: String,
0020 )
0021
0022 data class AuthResponseDto(
0023     @SerializedName("user") val user: AuthenticatedUserDto,
0024     @SerializedName("access") val accessToken: String,
0025     @SerializedName("refresh") val refreshToken: String
0026 )
0027
0028 data class RefreshRequestDto(
0029     @SerializedName("refresh") val refreshToken: String,
0030 )
0031
0032 data class RefreshResponseDto(
0033     @SerializedName("access") val accessToken: String,
0034     @SerializedName("refresh") val refreshToken: String,
0035 )
```

FlashcardDtos.kt

Path: mobile/app/src/main/java/com/example/mobile/data/remote/dto/FlashcardDtos.kt

Purpose: Defines flashcard response and create-flashcard request DTOs.

```
0001 package com.example.mobile.data.remote.dto
0002
0003 import com.google.gson.annotations.SerializedName
0004
0005 data class FlashcardDto(
0006     @SerializedName("id") val id: Int,
0007     @SerializedName("connected_note") val connectedNoteId: Int,
0008     @SerializedName("question") val question: String,
0009     @SerializedName("answer") val answer: String,
0010     @SerializedName("note_title") val noteTitle: String,
0011     @SerializedName("note_tag") val noteTag: String,
0012     @SerializedName("difficulty") val difficulty: String,
0013     @SerializedName("mastery_level") val masteryLevel: Int
0014 )
0015
0016 data class CreateFlashcardRequestDto(
0017     @SerializedName("connected_note") val noteId: Int,
0018     @SerializedName("question") val question: String,
0019     @SerializedName("answer") val answer: String,
0020     @SerializedName("difficulty") val difficulty: String,
0021     @SerializedName("mastery_level") val masteryLevel: Int = 0
0022 )
0023
0024 data class UpdateFlashcardRequestDto(
0025     @SerializedName("connected_note") val noteId: Int,
0026     @SerializedName("question") val question: String,
0027     @SerializedName("answer") val answer: String,
0028     @SerializedName("difficulty") val difficulty: String,
0029     @SerializedName("mastery_level") val masteryLevel: Int
0030 )
```

NoteDtos.kt

Path: mobile/app/src/main/java/com/example/mobile/data/remote/dto/NoteDtos.kt

Purpose: Defines note response, create-note request, and update-note request DTOs.

```
0001 package com.example.mobile.data.remote.dto
0002
0003 import com.google.gson.annotations.SerializedName
0004
0005 data class NoteDto(
0006     @SerializedName("id") val id: Int,
0007     @SerializedName("title") val title: String,
0008     @SerializedName("content") val content: String,
0009     @SerializedName("tag") val tag: String,
0010     @SerializedName("created_at") val created_at: String,
0011     @SerializedName("updated_at") val updated_at: String
0012 )
0013
0014 data class CreateNoteRequestDto(
0015     @SerializedName("title") val title: String,
0016     @SerializedName("content") val content: String,
0017     @SerializedName("tag") val tag: String
0018 )
0019
0020 data class UpdateNoteRequestDto(
0021     @SerializedName("title") val title: String,
0022     @SerializedName("content") val content: String,
0023     @SerializedName("tag") val tag: String
0024 )
```

ReviewDtos.kt

Path: mobile/app/src/main/java/com/example/mobile/data/remote/dto/ReviewDtos.kt

Purpose: Defines review completion request/response DTOs and progress response DTOs.

```
0001 package com.example.mobile.data.remote.dto
0002
0003 import com.google.gson.annotations.SerializedName
0004
0005 data class ReviewCompleteRequestDto(
0006     @SerializedName("answers") val answers: List<ReviewAnswerDto>
0007 )
0008
0009 data class ReviewAnswerDto(
0010     @SerializedName("flashcard_id") val flashcardId: Int,
0011     @SerializedName("recalled") val recalled: Boolean
0012 )
0013
0014 data class QuizAttemptDto(
0015     @SerializedName("id") val id: Int,
0016     @SerializedName("score") val score: Int,
0017     @SerializedName("total_questions") val totalQuestions: Int,
0018     @SerializedName("xp_earned") val xpEarned: Int,
0019     @SerializedName("date_taken") val dateTaken: String
0020 )
0021
0022 data class UserProgressDto(
0023     @SerializedName("id") val id: Int,
0024     @SerializedName("total_xp") val totalXp: Int,
0025     @SerializedName("level") val level: Int,
0026     @SerializedName("streak") val streak: Int,
0027     @SerializedName("total_reviews") val totalReviews: Int,
0028     @SerializedName("correct_reviews") val correctReviews: Int,
0029     @SerializedName("accuracy") val accuracy: Double
0030 )
0031
0032 data class ReviewCompleteResponseDto(
0033     @SerializedName("attempt") val attempt: QuizAttemptDto,
0034     @SerializedName("progress") val progress: UserProgressDto
0035 )
```

AuthInterceptor.kt

Path: mobile/app/src/main/java/com/example/mobile/data/remote/interceptor/AuthInterceptor.kt

Purpose: Adds the JWT bearer token to protected API requests.

```
0001 package com.example.mobile.data.remote.interceptor
0002
0003 import com.example.mobile.data.local.AuthSessionStore
0004 import kotlinx.coroutines.flow.first
0005 import kotlinx.coroutines.runBlocking
0006 import okhttp3.Interceptor
0007 import okhttp3.Response
0008
0009 class AuthInterceptor(
0010     private val authSessionStore: AuthSessionStore,
0011 ) : Interceptor {
0012     private val publicPaths = setOf(
0013         "/api/health/",
0014         "/api/auth/register/",
0015         "/api/auth/login/",
0016         "/api/auth/refresh/"
0017     )
0018
0019     override fun intercept(chain: Interceptor.Chain): Response {
0020         val originalRequest = chain.request()
0021         if (originalRequest.url.encodedPath in publicPaths) {
0022             return chain.proceed(originalRequest)
0023         }
0024
0025         val accessToken = runBlocking {
0026             authSessionStore.accessTokenFlow.first()
0027         }
0028         if (accessToken.isNullOrBlank()) {
0029             return chain.proceed(originalRequest)
0030         }
0031
0032         val authenticatedRequest = originalRequest.newBuilder()
0033             .header("Authorization", "Bearer $accessToken")
0034             .build()
0035         return chain.proceed(authenticatedRequest)
0036     }
0037 }
```


AuthRepository.kt

Path: mobile/app/src/main/java/com/example/mobile/data/repository/AuthRepository.kt

Purpose: Handles register, login, logout, refresh, and current-user API logic.

```
0001 package com.example.mobile.data.repository
0002
0003 import com.example.mobile.data.local.AuthSession
0004 import com.example.mobile.data.local.AuthSessionStore
0005 import com.example.mobile.data.remote.RetrofitClient
0006 import com.example.mobile.data.remote.api.ApiService
0007 import com.example.mobile.data.remote.dto.AuthResponseDto
0008 import com.example.mobile.data.remote.dto.LoginRequestDto
0009 import com.example.mobile.data.remote.dto.RefreshRequestDto
0010 import com.example.mobile.data.remote.dto.RegisterRequestDto
0011 import com.example.mobile.data.remote.dto.AuthenticatedUserDto
0012 import kotlinx.coroutines.flow.Flow
0013 import kotlinx.coroutines.flow.first
0014
0015 class AuthRepository(
0016     private val authSessionStore: AuthSessionStore,
0017     private val apiService: ApiService = RetrofitClient.apiService,
0018 ) {
0019     val sessionFlow: Flow<AuthSession?> = authSessionStore.sessionFlow
0020
0021     suspend fun register(
0022         displayName: String,
0023         email: String,
0024         password: String,
0025     ): Result<AuthSession> {
0026         return try {
0027             val response = apiService.register(
0028                 RegisterRequestDto(
0029                     displayName = displayName,
0030                     email = email,
0031                     password = password,
0032                 ),
0033             )
0034
0035             if (!response.isSuccessful) {
0036                 return Result.failure(
0037                     IllegalStateException("Registration failed with HTTP ${response.code().}"),
0038                 )
0039             }
0040
0041             val authResponse = response.body()
0042             ?: return Result.failure(
0043                 IllegalStateException("Registration returned no session data."),
0044             )
0045
0046             val session = authResponse.toSession()
0047             authSessionStore.saveSession(session)
0048
0049             Result.success(session)
0050         } catch (exception: Exception) {
0051             Result.failure(exception)
0052         }
0053     }
0054
0055     suspend fun login(email: String, password: String): Result<AuthSession> {
0056         return try {
0057             val response = apiService.login(LoginRequestDto(email, password))
0058
0059             if (!response.isSuccessful) {
0060                 return Result.failure(
0061                     IllegalStateException("Login failed with HTTP ${response.code().}"),
0062                 )
0063             }
0064
0065             val authResponse = response.body()
0066             ?: return Result.failure(
0067                 IllegalStateException("Login returned no session data."),
0068             )
0069
0070             val session = authResponse.toSession()
0071             authSessionStore.saveSession(session)
0072
0073             Result.success(session)
0074         } catch (exception: Exception) {
0075             Result.failure(exception)
0076         }
0077     }
0078
0079     suspend fun refreshSession(): Result<Unit> {
0080         return try {
0081             val refreshToken = authSessionStore.refreshTokenFlow.first()
0082             ?: return Result.failure(
0083                 IllegalStateException("No refresh token is available."),
0084             )
0085             val response = apiService.refreshToken(RefreshRequestDto(refreshToken))
0086
0087             if (!response.isSuccessful) {
0088                 return Result.failure(
0089                     IllegalStateException("Token refresh failed with HTTP ${response.code().}"),
0090                 )
0091             }
0092
0093             val refreshedTokens = response.body()
0094             ?: return Result.failure(
0095                 IllegalStateException("Token refresh returned no token data."),
0096             )
0097             authSessionStore.updateTokens(
0098                 accessToken = refreshedTokens.accessToken,
```

```

0099         refreshToken = refreshedTokens.refreshToken,
0100     )
0101
0102     Result.success(Unit)
0103 } catch (exception: Exception) {
0104     Result.failure(exception)
0105 }
0106 }
0107
0108 suspend fun logout(): Result<Unit> {
0109     return try {
0110         val refreshToken = authSessionStore.refreshTokenFlow.first()
0111
0112         if (refreshToken == null) {
0113             Result.success(Unit)
0114         } else {
0115             val response = apiService.logout(RefreshRequestDto(refreshToken))
0116             if (response.isSuccessful) {
0117                 Result.success(Unit)
0118             } else {
0119                 Result.failure(
0120                     IllegalStateException("Logout failed with HTTP ${response.code()}."),
0121                 )
0122             }
0123         }
0124     } catch (exception: Exception) {
0125         Result.failure(exception)
0126     } finally {
0127         authSessionStore.clearSession()
0128     }
0129 }
0130
0131 private fun AuthResponseDto.toSession(): AuthSession {
0132     return AuthSession(
0133         accessToken = accessToken,
0134         refreshToken = refreshToken,
0135         userId = user.id,
0136         displayName = user.displayName,
0137         email = user.email,
0138     )
0139 }
0140
0141 suspend fun verifySession(): Result<AuthSession?> {
0142     return try {
0143         val session = sessionFlow.first()
0144         ?: return Result.success(null)
0145
0146         val response = apiService.getAuthenticatedUser()
0147
0148         if (response.isSuccessful) {
0149             val user = response.body()
0150             ?: return Result.failure(
0151                 IllegalStateException("Session verification returned no user data."),
0152             )
0153
0154             return Result.success(saveVerifiedSession(session, user))
0155         }
0156
0157         if (response.code() != 401) {
0158             return Result.failure(
0159                 IllegalStateException(
0160                     "Session verification failed with HTTP ${response.code()}.",
0161                 ),
0162             )
0163         }
0164
0165         if (refreshSession().isFailure) {
0166             authSessionStore.clearSession()
0167             return Result.success(null)
0168         }
0169
0170         val retryResponse = apiService.getAuthenticatedUser()
0171
0172         if (retryResponse.isSuccessful) {
0173             val user = retryResponse.body()
0174             ?: return Result.failure(
0175                 IllegalStateException("Session retry returned no user data."),
0176             )
0177
0178             return Result.success(saveVerifiedSession(session, user))
0179         }
0180
0181         if (retryResponse.code() == 401) {
0182             authSessionStore.clearSession()
0183             return Result.success(null)
0184         }
0185
0186         Result.failure(
0187             IllegalStateException(
0188                 "Session retry failed with HTTP ${retryResponse.code()}.",
0189             ),
0190         )
0191     } catch (exception: Exception) {
0192         Result.failure(exception)
0193     }
0194 }
0195
0196 private suspend fun saveVerifiedSession(
0197     session: AuthSession,
0198     user: AuthenticatedUserDto,
0199 ): AuthSession {
0200     val verifiedSession = session.copy(
0201         userId = user.id,
0202         displayName = user.displayName,
0203         email = user.email,
0204     )
0205     authSessionStore.saveSession(verifiedSession)
0206     return verifiedSession
0207 }
0208 }

```

NSyncRepository.kt

Path: mobile/app/src/main/java/com/example/mobile/data/repository/NSyncRepository.kt

Purpose: Handles notes, flashcards, review completion, and progress API logic.

```
0001 package com.example.mobile.data.repository
0002
0003 import com.example.mobile.data.KnowledgeItem
0004 import com.example.mobile.data.ReviewCard
0005 import com.example.mobile.data.remote.RetrofitClient
0006 import com.example.mobile.data.remote.dto.CreateNoteRequestDto
0007 import com.example.mobile.data.remote.dto.CreateFlashcardRequestDto
0008 import com.example.mobile.data.remote.dto.NoteDto
0009 import com.example.mobile.data.remote.dto.ReviewCompleteRequestDto
0010 import com.example.mobile.data.remote.dto.ReviewCompleteResponseDto
0011 import com.example.mobile.data.remote.dto.ReviewAnswerDto
0012 import com.example.mobile.data.remote.dto.UpdateNoteRequestDto
0013 import com.example.mobile.data.remote.dto.UpdateFlashcardRequestDto
0014 import com.example.mobile.data.remote.dto.FlashcardDto
0015 import com.example.mobile.data.remote.dto.UserProgressDto
0016
0017 class NSyncRepository {
0018     private val apiService = RetrofitClient.apiService
0019
0020     suspend fun getNotes(): List<NoteDto>? {
0021         val response = apiService.getNotes()
0022         return if (response.isSuccessful) {
0023             response.body()
0024         } else {
0025             null
0026         }
0027     }
0028
0029     suspend fun getNoteById(id: Int): NoteDto? {
0030         val response = apiService.getNoteById(id)
0031         return if (response.isSuccessful) {
0032             response.body()
0033         } else {
0034             null
0035         }
0036     }
0037
0038     suspend fun createNote(title: String, content: String, tag: String): NoteDto? {
0039         val note = CreateNoteRequestDto(
0040             title = title,
0041             content = content,
0042             tag = tag
0043         )
0044         val response = apiService.createNote(note)
0045         return if (response.isSuccessful) {
0046             response.body()
0047         } else {
0048             null
0049         }
0050     }
0051
0052     suspend fun updateNote(id: Int, title: String, content: String, tag: String): NoteDto? {
0053         val note = UpdateNoteRequestDto(
0054             title = title,
0055             content = content,
0056             tag = tag
0057         )
0058         val response = apiService.updateNote(id, note)
0059         return if (response.isSuccessful) {
0060             response.body()
0061         } else {
0062             null
0063         }
0064     }
0065
0066     suspend fun deleteNote(id: Int): Boolean {
0067         val response = apiService.deleteNote(id)
0068         return response.isSuccessful
0069     }
0070
0071     suspend fun getKnowledgeItemById(id: Int): KnowledgeItem? {
0072         return getNoteById(id)?.toKnowledgeItem()
0073     }
0074
0075     suspend fun getKnowledgeItems(): List<KnowledgeItem> {
0076         return try {
0077             getNotes().orEmpty().map { note ->
0078                 note.toKnowledgeItem()
0079             }
0080         } catch (e: Exception) {
0081             emptyList()
0082         }
0083     }
0084
0085     private fun NoteDto.toKnowledgeItem(): KnowledgeItem {
0086         return KnowledgeItem(
0087             id = id,
0088             title = title,
0089             collection = tag.ifBlank { "General" },
0090             source = tag.ifBlank { "Manual note" },
0091             context = tag.ifBlank { "Knowledge base" },
0092             summary = content,
0093             fullNote = content,
0094             reviewCardCount = 0,
0095             updatedLabel = "Updated recently",
0096             masteryPercent = 0,
0097             xpEarned = 0
0098         )
0099     }
0100 }
```

```

0099     }
0100
0101
0102     suspend fun getReviewCards(): List<ReviewCard> {
0103         val response = apiService.getFlashcards()
0104         if (!response.isSuccessful) {
0105             throw IllegalStateException("Request failed with HTTP ${response.code()}.")
0106         }
0107
0108         return response.body().orEmpty().map { flashcard ->
0109             flashcard.toReviewCard()
0110         }
0111     }
0112
0113     suspend fun createFlashcard(
0114         noteId: Int,
0115         question: String,
0116         answer: String,
0117         difficulty: String
0118     ): ReviewCard? {
0119         val request = CreateFlashcardRequestDto(
0120             noteId = noteId,
0121             question = question,
0122             answer = answer,
0123             difficulty = difficulty
0124         )
0125         val response = apiService.createFlashcard(request)
0126         return if (response.isSuccessful) {
0127             response.body()?.toReviewCard()
0128         } else {
0129             null
0130         }
0131     }
0132
0133     suspend fun getReviewCardById(id: Int): ReviewCard? {
0134         val response = apiService.getFlashcardById(id)
0135         return if (response.isSuccessful) response.body()?.toReviewCard() else null
0136     }
0137
0138     suspend fun updateFlashcard(
0139         id: Int,
0140         noteId: Int,
0141         question: String,
0142         answer: String,
0143         difficulty: String,
0144         masteryLevel: Int
0145     ): ReviewCard? {
0146         val request = UpdateFlashcardRequestDto(
0147             noteId = noteId,
0148             question = question,
0149             answer = answer,
0150             difficulty = difficulty,
0151             masteryLevel = masteryLevel
0152         )
0153         val response = apiService.updateFlashcard(id, request)
0154         return if (response.isSuccessful) response.body()?.toReviewCard() else null
0155     }
0156
0157     suspend fun deleteFlashcard(id: Int): Boolean {
0158         return apiService.deleteFlashcard(id).isSuccessful
0159     }
0160
0161     suspend fun completeReview(answers: List<ReviewAnswerDto>): ReviewCompleteResponseDto {
0162         val response = apiService.completeReview(
0163             ReviewCompleteRequestDto(answers)
0164         )
0165         if (!response.isSuccessful) {
0166             throw IllegalStateException("Unable to save review results (HTTP ${response.code()}).")
0167         }
0168         return response.body()
0169             ?: throw IllegalStateException("The server returned no review result.")
0170     }
0171
0172     private fun FlashcardDto.toReviewCard(): ReviewCard {
0173         return ReviewCard(
0174             id = id,
0175             knowledgeItemId = connectedNoteId,
0176             collection = noteTag.ifBlank { noteTitle.ifBlank { "General" } },
0177             sourceNoteTitle = noteTitle.ifBlank { "Untitled note" },
0178             question = question,
0179             answer = answer,
0180             difficulty = difficulty.ifBlank { "Unspecified" },
0181             masteryLabel = masteryLabel(masteryLevel),
0182             masteryPercent = masteryPercent(masteryLevel),
0183             updatedLabel = "From knowledge base"
0184         )
0185     }
0186
0187     private fun masteryLabel(level: Int): String {
0188         return when (level) {
0189             3 -> "Mastered"
0190             2 -> "Familiar"
0191             1 -> "Learning"
0192             else -> "New"
0193         }
0194     }
0195
0196     private fun masteryPercent(level: Int): Int {
0197         return when (level) {
0198             3 -> 100
0199             2 -> 66
0200             1 -> 33
0201             else -> 0
0202         }
0203     }
0204
0205     suspend fun getProgress(): UserProgressDto {
0206         val response = apiService.getProgress()
0207         if (!response.isSuccessful) {
0208             throw IllegalStateException("Unable to load progress (HTTP ${response.code()}).")
0209         }
0210         return response.body()

```

```
0211         ?: throw IllegalStateException("The server returned no progress data.")
0212     }
0213 }
```

AppNavigation.kt

Path: mobile/app/src/main/java/com/example/mobile/navigation/AppNavigation.kt

Purpose: Controls Navigation Compose routes, auth redirects, route arguments, and screen callbacks.

```
0001 package com.example.mobile.navigation
0002
0003 import androidx.compose.runtime.Composable
0004 import androidx.navigation.NavType
0005 import androidx.navigation.compose.NavHost
0006 import androidx.navigation.compose.composable
0007 import androidx.navigation.compose.rememberNavController
0008 import androidx.navigation.navArgument
0009 import com.example.mobile.ui.screens.auth.LoginScreen
0010 import com.example.mobile.ui.screens.auth.RegisterScreen
0011 import com.example.mobile.ui.screens.dashboard.DashboardScreen
0012 import com.example.mobile.ui.screens.knowledge.KnowledgeBaseScreen
0013 import com.example.mobile.ui.screens.knowledge.KnowledgeDetailScreen
0014 import com.example.mobile.ui.screens.knowledge.NewNoteScreen
0015 import com.example.mobile.ui.screens.profile.ProfileScreen
0016 import com.example.mobile.ui.screens.settings.SettingsScreen
0017 import com.example.mobile.ui.screens.progress.MasteryScreen
0018 import com.example.mobile.ui.screens.review.ReviewCardsScreen
0019 import com.example.mobile.ui.screens.review.NewFlashcardScreen
0020 import com.example.mobile.ui.screens.review.FlashcardDetailScreen
0021 import com.example.mobile.ui.screens.review.ReviewSessionScreen
0022 import com.example.mobile.ui.screens.review.SessionCompleteScreen
0023 import androidx.compose.runtime.LaunchedEffect
0024 import androidx.compose.runtime.remember
0025 import androidx.lifecycle.viewmodel.compose.viewModel
0026 import com.example.mobile.data.remote.RetrofitClient
0027 import com.example.mobile.data.repository.AuthRepository
0028 import com.example.mobile.ui.viewmodel.AuthViewModel
0029 import com.example.mobile.ui.viewmodel.AuthViewModelFactory
0030
0031 @Composable
0032 fun AppNavigation() {
0033     val navController = rememberNavController()
0034     val authRepository = remember {
0035         AuthRepository(RetrofitClient.sessionStore)
0036     }
0037     val authViewModel: AuthViewModel = viewModel {
0038         factory = remember { authRepository } {
0039             AuthViewModelFactory(authRepository)
0040         },
0041     }
0042     val authState = authViewModel.uiState
0043
0044     LaunchedEffect(Unit) {
0045         authViewModel.restoreSession()
0046     }
0047     val navigateBottom: (String) -> Unit = { route ->
0048         if (navController.currentDestination?.route != route) {
0049             if (route == Routes.DASHBOARD) {
0050                 val popped = navController.popBackStack(Routes.DASHBOARD, inclusive = false)
0051                 if (!popped) {
0052                     navController.navigate(Routes.DASHBOARD) {
0053                         launchSingleTop = true
0054                     }
0055                 }
0056             } else {
0057                 navController.navigate(route) {
0058                     launchSingleTop = true
0059                     popUpTo(Routes.DASHBOARD) {
0060                         saveState = true
0061                     }
0062                     restoreState = true
0063                 }
0064             }
0065         }
0066     }
0067
0068     LaunchedEffect(authState.hasRestoredSession, authState.session) {
0069         if (!authState.hasRestoredSession) return@LaunchedEffect
0070
0071         val currentRoute = navController.currentDestination?.route
0072         val isAuthRoute = currentRoute == Routes.LOGIN || currentRoute == Routes.REGISTER
0073
0074         if (authState.session != null && isAuthRoute) {
0075             navController.navigate(Routes.DASHBOARD) {
0076                 popUpTo(Routes.LOGIN) {
0077                     inclusive = true
0078                 }
0079                 launchSingleTop = true
0080             }
0081         } else if (authState.session == null && !isAuthRoute) {
0082             navController.navigate(Routes.LOGIN) {
0083                 popUpTo(0)
0084                 launchSingleTop = true
0085             }
0086         }
0087     }
0088
0089     NavHost(
0090         navController = navController,
0091         startDestination = Routes.LOGIN
0092     ) {
0093         composable(Routes.LOGIN) {
0094             LoginScreen(
0095                 onLoginClick = authViewModel::login,
0096                 onRegisterClick = {
0097                     authViewModel.clearError()
0098                     navController.navigate(Routes.REGISTER)
0099                 }
0100             )
0101         }
0102     }
0103 }
```

```

0099         },
0100         isLoading = authState.isLoading,
0101         errorMessage = authState.errorMessage,
0102     )
0103 }
0104
0105 composable(Routes.REGISTER) {
0106     RegisterScreen(
0107         onRegisterClick = authViewModel::register,
0108         onLoginClick = {
0109             authViewModel.clearError()
0110             navController.popBackStack()
0111         },
0112         isLoading = authState.isLoading,
0113         errorMessage = authState.errorMessage,
0114     )
0115 }
0116
0117 composable(Routes.DASHBOARD) {
0118     DashboardScreen(
0119         onStartReviewClick = { navController.navigate(Routes.REVIEW_CARDS) },
0120         onKnowledgeClick = { item -> navController.navigate(Routes.knowledgeDetail(item.id)) },
0121         onAddClick = { navController.navigate(Routes.REVIEW_CARDS) },
0122         onRouteClick = navigateBottom
0123     )
0124 }
0125
0126 composable(Routes.KNOWLEDGE_BASE) {
0127     KnowledgeBaseScreen(
0128         onKnowledgeClick = { item -> navController.navigate(Routes.knowledgeDetail(item.id)) },
0129         onNewNoteClick = { navController.navigate(Routes.NEW_NOTE) },
0130         onRouteClick = navigateBottom
0131     )
0132 }
0133
0134 composable(Routes.NEW_NOTE) {
0135     NewNoteScreen(
0136         onBackClick = { navController.popBackStack() },
0137         onSaveClick = { navController.popBackStack(Routes.KNOWLEDGE_BASE, inclusive = false) },
0138         onRouteClick = navigateBottom
0139     )
0140 }
0141
0142 composable(
0143     route = Routes.EDIT_NOTE,
0144     arguments = listOf(navArgument("noteId") { type = NavType.IntType })
0145 ) { backStackEntry ->
0146     val noteId = backStackEntry.arguments?.getInt("noteId") ?: return@composable
0147     NewNoteScreen(
0148         noteId = noteId,
0149         onBackClick = { navController.popBackStack() },
0150         onSaveClick = { navController.popBackStack(Routes.KNOWLEDGE_BASE, inclusive = false) },
0151         onRouteClick = navigateBottom
0152     )
0153 }
0154
0155 composable(
0156     route = Routes.KNOWLEDGE_DETAIL,
0157     arguments = listOf(navArgument("noteId") { type = NavType.IntType })
0158 ) { backStackEntry ->
0159     val noteId = backStackEntry.arguments?.getInt("noteId") ?: return@composable
0160     KnowledgeDetailScreen(
0161         noteId = noteId,
0162         onBackClick = { navController.popBackStack() },
0163         onEditClick = { navController.navigate(Routes.editNote(noteId)) },
0164         onAddReviewCardClick = { navController.navigate(Routes.newFlashcard(noteId)) },
0165         onDeleted = { navController.popBackStack(Routes.KNOWLEDGE_BASE, inclusive = false) },
0166         onStartReviewClick = { navController.navigate(Routes.REVIEW_SESSION) },
0167         onRouteClick = navigateBottom
0168     )
0169 }
0170
0171 composable(
0172     route = Routes.NEW_FLASHCARD,
0173     arguments = listOf(navArgument("noteId") { type = NavType.IntType })
0174 ) { backStackEntry ->
0175     val noteId = backStackEntry.arguments?.getInt("noteId") ?: return@composable
0176     NewFlashcardScreen(
0177         noteId = noteId,
0178         onBackClick = { navController.popBackStack() },
0179         onSave = { navController.popBackStack() }
0180     )
0181 }
0182
0183 composable(
0184     route = Routes.EDIT_FLASHCARD,
0185     arguments = listOf(navArgument("cardId") { type = NavType.IntType })
0186 ) { backStackEntry ->
0187     val cardId = backStackEntry.arguments?.getInt("cardId") ?: return@composable
0188     NewFlashcardScreen(
0189         cardId = cardId,
0190         onBackClick = { navController.popBackStack() },
0191         onSave = { navController.popBackStack() }
0192     )
0193 }
0194
0195 composable(
0196     route = Routes.FLASHCARD_DETAIL,
0197     arguments = listOf(navArgument("cardId") { type = NavType.IntType })
0198 ) { backStackEntry ->
0199     val cardId = backStackEntry.arguments?.getInt("cardId") ?: return@composable
0200     FlashcardDetailScreen(
0201         cardId = cardId,
0202         onBackClick = { navController.popBackStack() },
0203         onEditClick = { navController.navigate(Routes.editFlashcard(cardId)) },
0204         onStartSessionClick = { noteId ->
0205             navController.navigate(Routes.reviewSessionForNote(noteId))
0206         },
0207         onDeleted = { navController.popBackStack(Routes.REVIEW_CARDS, inclusive = false) },
0208         onRouteClick = navigateBottom
0209     )
0210 }

```

```

0211
0212     composable(Routes.REVIEW_CARDS) {
0213         ReviewCardsScreen(
0214             onStartSessionClick = { noteId ->
0215                 navController.navigate(Routes.reviewSessionForNote(noteId))
0216             },
0217             onAddCardClick = { noteId -> navController.navigate(Routes.newFlashcard(noteId)) },
0218             onRouteClick = navigateBottom
0219         )
0220     }
0221
0222     composable(Routes.REVIEW_SESSION) {
0223         ReviewSessionScreen(
0224             onCompleteClick = { score, totalQuestions, xpEarned ->
0225                 navController.navigate(
0226                     Routes.sessionComplete(score, totalQuestions, xpEarned)
0227                 )
0228             }
0229         )
0230     }
0231
0232     composable(
0233         route = Routes.REVIEW_SESSION_NOTE,
0234         arguments = listOf(navArgument("noteId") { type = NavType.IntType })
0235     ) { backStackEntry ->
0236         val noteId = backStackEntry.arguments?.getInt("noteId") ?: return@composable
0237         ReviewSessionScreen(
0238             noteId = noteId,
0239             onCompleteClick = { score, totalQuestions, xpEarned ->
0240                 navController.navigate(
0241                     Routes.sessionComplete(score, totalQuestions, xpEarned, noteId)
0242                 )
0243             }
0244         )
0245     }
0246
0247     composable(Routes.MASTERY) {
0248         MasteryScreen(onRouteClick = navigateBottom)
0249     }
0250
0251     composable(Routes.PROFILE) {
0252         ProfileScreen(
0253             onRouteClick = navigateBottom,
0254             onSettingsClick = { navController.navigate(Routes.SETTINGS) },
0255             onLogoutClick = authViewModel::logout
0256         )
0257     }
0258
0259     composable(Routes.SETTINGS) {
0260         SettingsScreen(
0261             onBackClick = { navController.popBackStack() },
0262             onProfileClick = { navController.popBackStack(Routes.PROFILE, inclusive = false) }
0263         )
0264     }
0265
0266     composable(
0267         route = Routes.SESSION_COMPLETE,
0268         arguments = listOf(
0269             navArgument("score") { type = NavType.IntType },
0270             navArgument("totalQuestions") { type = NavType.IntType },
0271             navArgument("xpEarned") { type = NavType.IntType },
0272             navArgument("noteId") { type = NavType.IntType }
0273         )
0274     ) { backStackEntry ->
0275         val score = backStackEntry.arguments?.getInt("score") ?: return@composable
0276         val totalQuestions = backStackEntry.arguments?.getInt("totalQuestions") ?: return@composable
0277         val xpEarned = backStackEntry.arguments?.getInt("xpEarned") ?: return@composable
0278         val completedNoteId = backStackEntry.arguments
0279             ?.getInt("noteId")
0280             ?.takeIf { it > 0 }
0281         SessionCompleteScreen(
0282             score = score,
0283             totalQuestions = totalQuestions,
0284             xpEarned = xpEarned,
0285             onRouteClick = navigateBottom,
0286             onReviewAgainClick = {
0287                 navController.navigate(
0288                     completedNoteId?.let { Routes.reviewSessionForNote(it) }
0289                     ?: Routes.REVIEW_SESSION
0290                 )
0291             },
0292             onDashboardClick = {
0293                 navController.navigate(Routes.DASHBOARD) {
0294                     popUpTo(Routes.DASHBOARD) {
0295                         inclusive = true
0296                     }
0297                 }
0298             }
0299         )
0300     }
0301 }
0302 }

```


Routes.kt

Path: mobile/app/src/main/java/com/example/mobile/navigation/Routes.kt

Purpose: Defines route constants, route builders, and bottom navigation routes.

```
0001 package com.example.mobile.navigation
0002
0003 import com.example.mobile.R
0004
0005 object Routes {
0006     const val LOGIN = "login"
0007     const val REGISTER = "register"
0008     const val DASHBOARD = "dashboard"
0009     const val KNOWLEDGE_BASE = "knowledge_base"
0010     const val KNOWLEDGE_DETAIL = "knowledge_detail/{noteId}"
0011     const val NEW_NOTE = "new_note"
0012     const val EDIT_NOTE = "edit_note/{noteId}"
0013     const val NEW_FLASHCARD = "new_flashcard/{noteId}"
0014     const val FLASHCARD_DETAIL = "flashcard_detail/{cardId}"
0015     const val EDIT_FLASHCARD = "edit_flashcard/{cardId}"
0016     const val REVIEW_SESSION_NOTE = "review_session/note/{noteId}"
0017     const val REVIEW_CARDS = "review_cards"
0018     const val REVIEW_SESSION = "review_session"
0019     const val SESSION_COMPLETE = "session_complete/{score}/{totalQuestions}/{xpEarned}/{noteId}"
0020     const val MASTERY = "mastery"
0021     const val PROFILE = "profile"
0022     const val SETTINGS = "settings"
0023
0024     fun knowledgeDetail(noteId: Int): String = "knowledge_detail/$noteId"
0025
0026     fun editNote(noteId: Int): String = "edit_note/$noteId"
0027
0028     fun newFlashcard(noteId: Int): String = "new_flashcard/$noteId"
0029
0030     fun flashcardDetail(cardId: Int): String = "flashcard_detail/$cardId"
0031
0032     fun editFlashcard(cardId: Int): String = "edit_flashcard/$cardId"
0033
0034     fun reviewSessionForNote(noteId: Int): String = "review_session/note/$noteId"
0035
0036     fun sessionComplete(
0037         score: Int,
0038         totalQuestions: Int,
0039         xpEarned: Int,
0040         noteId: Int? = null
0041     ): String = "session_complete/$score/$totalQuestions/$xpEarned/${noteId ?: -1}"
0042 }
0043
0044 data class BottomNavRoute(
0045     val label: String,
0046     val route: String,
0047     val iconRes: Int
0048 )
0049
0050 val bottomNavRoutes = listOf(
0051     BottomNavRoute(
0052         label = "Dashboard",
0053         route = Routes.DASHBOARD,
0054         iconRes = R.drawable.ic_grid
0055     ),
0056     BottomNavRoute(
0057         label = "Knowledge",
0058         route = Routes.KNOWLEDGE_BASE,
0059         iconRes = R.drawable.ic_briefcase
0060     ),
0061     BottomNavRoute(
0062         label = "Flashcards",
0063         route = Routes.REVIEW_CARDS,
0064         iconRes = R.drawable.ic_flashcard
0065     ),
0066     BottomNavRoute(
0067         label = "Mastery",
0068         route = Routes.MASTERY,
0069         iconRes = R.drawable.ic_chart
0070     ),
0071     BottomNavRoute(
0072         label = "Profile",
0073         route = Routes.PROFILE,
0074         iconRes = R.drawable.ic_profile
0075     )
0076 )
```

ReviewReminderReceiver.kt

Path: mobile/app/src/main/java/com/example/mobile/notifications/ReviewReminderReceiver.kt

Purpose: Broadcast receiver for review reminder notifications.

```
0001 package com.example.mobile.notifications
0002
0003 import android.app.NotificationChannel
0004 import android.app.NotificationManager
0005 import android.app.PendingIntent
0006 import android.content.BroadcastReceiver
0007 import android.content.Context
0008 import android.content.Intent
0009 import android.os.Build
0010 import androidx.core.app.NotificationCompat
0011 import androidx.core.app.NotificationManagerCompat
0012 import androidx.core.content.ContextCompat
0013 import com.example.mobile.MainActivity
0014 import com.example.mobile.R
0015 import com.example.mobile.ui.screens.settings.SettingsPreferences
0016
0017 class ReviewReminderReceiver : BroadcastReceiver() {
0018     override fun onReceive(context: Context, intent: Intent) {
0019         if (
0020             Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU &&
0021             ContextCompat.checkSelfPermission(
0022                 context,
0023                 android.Manifest.permission.POST_NOTIFICATIONS
0024             ) != android.content.pm.PackageManager.PERMISSION_GRANTED
0025         ) {
0026             ReviewReminderScheduler.cancel(context)
0027             return
0028         }
0029
0030         createChannel(context)
0031         val preferences = SettingsPreferences.from(context)
0032         val dailyGoal = preferences.getInt(SettingsPreferences.DAILY_GOAL, 50)
0033         val streakReminders = preferences.getBoolean(SettingsPreferences.STREAK_REMINDERS, true)
0034         val message = if (streakReminders) {
0035             "Review $dailyGoal cards to protect your streak."
0036         } else {
0037             "Your $dailyGoal-card review goal is ready."
0038         }
0039         val contentIntent = PendingIntent.getActivity(
0040             context,
0041             0,
0042             Intent(context, MainActivity::class.java),
0043             PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE
0044         )
0045         val notification = NotificationCompat.Builder(context, CHANNEL_ID)
0046             .setSmallIcon(android.R.drawable.ic_dialog_info)
0047             .setContentTitle("NSync review reminder")
0048             .setContentText(message)
0049             .setContentIntent(contentIntent)
0050             .setAutoCancel(true)
0051             .build()
0052
0053         NotificationManagerCompat.from(context).notify(NOTIFICATION_ID, notification)
0054         ReviewReminderScheduler.schedule(context)
0055     }
0056
0057     private fun createChannel(context: Context) {
0058         if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
0059             val channel = NotificationChannel(
0060                 CHANNEL_ID,
0061                 "Review reminders",
0062                 NotificationManager.IMPORTANCE_DEFAULT
0063             )
0064             context.getSystemService(NotificationManager::class.java).createNotificationChannel(channel)
0065         }
0066     }
0067
0068     private companion object {
0069         const val CHANNEL_ID = "review_reminders"
0070         const val NOTIFICATION_ID = 701
0071     }
0072 }
```

ReviewReminderScheduler.kt

Path: mobile/app/src/main/java/com/example/mobile/notifications/ReviewReminderScheduler.kt

Purpose: Schedules or cancels local review reminder notifications.

```
0001 package com.example.mobile.notifications
0002
0003 import android.app.AlarmManager
0004 import android.app.PendingIntent
0005 import android.content.Context
0006 import android.content.Intent
0007 import com.example.mobile.ui.screens.settings.SettingsPreferences
0008 import java.util.Calendar
0009
0010 object ReviewReminderScheduler {
0011     private const val REQUEST_CODE = 401
0012
0013     fun schedule(context: Context) {
0014         val preferences = SettingsPreferences.from(context)
0015         if (!preferences.getBoolean(SettingsPreferences.NOTIFICATIONS_ENABLED, true)) {
0016             cancel(context)
0017             return
0018         }
0019
0020         val triggerAtMillis = reminderCalendar(
0021             preferences.getString(SettingsPreferences.REMINDER_TIME, "8:00 AM") ?: "8:00 AM"
0022         ).timeInMillis
0023         val alarmManager = context.getSystemService(AlarmManager::class.java)
0024         alarmManager.setAndAllowWhileIdle(
0025             AlarmManager.RTC_WAKEUP,
0026             triggerAtMillis,
0027             pendingIntent(context)
0028         )
0029     }
0030
0031     fun cancel(context: Context) {
0032         context.getSystemService(AlarmManager::class.java).cancel(pendingIntent(context))
0033     }
0034
0035     private fun reminderCalendar(reminderTime: String): Calendar {
0036         val (hour, minute) = when (reminderTime) {
0037             "12:00 PM" -> 12 to 0
0038             "6:00 PM" -> 18 to 0
0039             else -> 8 to 0
0040         }
0041         return Calendar.getInstance().apply {
0042             set(Calendar.HOUR_OF_DAY, hour)
0043             set(Calendar.MINUTE, minute)
0044             set(Calendar.SECOND, 0)
0045             set(Calendar.MILLISECOND, 0)
0046             if (timeInMillis <= System.currentTimeMillis()) {
0047                 add(Calendar.DAY_OF_YEAR, 1)
0048             }
0049         }
0050     }
0051
0052     private fun pendingIntent(context: Context): PendingIntent = PendingIntent.getBroadcast(
0053         context,
0054         REQUEST_CODE,
0055         Intent(context, ReviewReminderReceiver::class.java),
0056         PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE
0057     )
0058 }
```

AuthTextField.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/components/AuthTextField.kt

Purpose: Reusable authentication text field with consistent colors and password support.

```
0001 package com.example.mobile.ui.components
0002
0003 import androidx.compose.foundation.layout.fillMaxWidth
0004 import androidx.compose.foundation.layout.height
0005 import androidx.compose.foundation.layout.size
0006 import androidx.compose.foundation.shape.RoundedCornerShape
0007 import androidx.compose.material3.Icon
0008 import androidx.compose.material3.OutlinedTextField
0009 import androidx.compose.material3.OutlinedTextFieldDefaults
0010 import androidx.compose.material3.Text
0011 import androidx.compose.runtime.Composable
0012 import androidx.compose.ui.Modifier
0013 import androidx.compose.ui.graphics.Color
0014 import androidx.compose.ui.res.painterResource
0015 import androidx.compose.ui.text.TextStyle
0016 import androidx.compose.ui.text.font.FontWeight
0017 import androidx.compose.ui.text.input.PasswordVisualTransformation
0018 import androidx.compose.ui.text.input.VisualTransformation
0019 import androidx.compose.ui.unit.dp
0020 import androidx.compose.ui.unit.sp
0021 import com.example.mobile.ui.theme.InterFontFamily
0022 import com.example.mobile.ui.theme.NSyncBlue
0023 import com.example.mobile.ui.theme.NSyncCardWhite
0024 import com.example.mobile.ui.theme.nsyncTextFieldColors
0025
0026 @Composable
0027 fun AuthTextField(
0028     value: String,
0029     onChange: (String) -> Unit,
0030     placeholder: String,
0031     leadingIcon: Int? = null,
0032     isPassword: Boolean = false
0033 ) {
0034     OutlinedTextField(
0035         value = value,
0036         onChange = onChange,
0037         modifier = Modifier
0038             .fillMaxWidth()
0039             .height(56.dp),
0040         leadingIcon = leadingIcon?.let {
0041             Icon(
0042                 painter = painterResource(id = it),
0043                 contentDescription = null,
0044                 modifier = Modifier.size(18.dp),
0045                 tint = AuthIcon
0046             )
0047         },
0048         placeholder = {
0049             Text(
0050                 text = placeholder,
0051                 color = AuthPlaceholder,
0052                 style = AuthFieldTextStyle
0053             )
0054         },
0055         textStyle = AuthFieldTextStyle,
0056         singleLine = true,
0057         visualTransformation = if (isPassword) PasswordVisualTransformation() else VisualTransformation.None,
0058         shape = RoundedCornerShape(10.dp),
0059         colors = OutlinedTextFieldDefaults.colors(
0060             focusedTextColor = AuthInputText,
0061             unfocusedTextColor = AuthInputText,
0062             disabledTextColor = AuthInputText.copy(alpha = 0.6f),
0063             focusedBorderColor = NSyncBlue,
0064             unfocusedBorderColor = AuthBorder,
0065             focusedContainerColor = NSyncCardWhite,
0066             unfocusedContainerColor = NSyncCardWhite,
0067             disabledContainerColor = NSyncCardWhite,
0068             cursorColor = NSyncBlue,
0069             focusedPlaceholderColor = AuthPlaceholder,
0070             unfocusedPlaceholderColor = AuthPlaceholder,
0071             focusedLeadingIconColor = AuthIcon,
0072             unfocusedLeadingIconColor = AuthIcon
0073         )
0074     )
0075 }
0076
0077 private val AuthInputText = Color(0xFF151927)
0078 private val AuthPlaceholder = Color(0xFFB5BBC6)
0079 private val AuthBorder = Color(0xFFE3E7F0)
0080 private val AuthIcon = Color(0xFF747B8A)
0081
0082 private val AuthFieldTextStyle = TextStyle(
0083     fontFamily = InterFontFamily,
0084     fontSize = 13.sp,
0085     lineHeight = 18.sp,
0086     fontWeight = FontWeight.Normal,
0087     color = AuthInputText,
0088     letterSpacing = 0.sp
0089 )
0090
0091 }
```

BottomNavBar.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/components/BottomNavBar.kt

Purpose: Reusable bottom navigation bar for the main app sections.

```
0001 package com.example.mobile.ui.components
0002
0003 import androidx.compose.foundation.layout.size
0004 import androidx.compose.material3.Icon
0005 import androidx.compose.material3.NavigationBar
0006 import androidx.compose.material3.NavigationBarItem
0007 import androidx.compose.material3.Text
0008 import androidx.compose.runtime.Composable
0009 import androidx.compose.ui.Modifier
0010 import androidx.compose.ui.res.painterResource
0011 import androidx.compose.ui.unit.dp
0012 import com.example.mobile.navigation.bottomNavRoutes
0013
0014 @Composable
0015 fun BottomNavBar(
0016     currentRoute: String,
0017     onRouteClick: (String) -> Unit
0018 ) {
0019     NavigationBar {
0020         bottomNavRoutes.forEach { item ->
0021             NavigationBarItem(
0022                 selected = currentRoute == item.route,
0023                 onClick = {
0024                     onRouteClick(item.route)
0025                 },
0026                 icon = {
0027                     Icon(
0028                         painter = painterResource(id = item.iconRes),
0029                         contentDescription = item.label,
0030                         modifier = Modifier.size(24.dp)
0031                     )
0032                 },
0033                 label = {
0034                     Text(item.label)
0035                 }
0036             )
0037         }
0038     }
0039 }
```

KnowledgeListCard.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/components/KnowledgeListCard.kt

Purpose: Reusable card for knowledge-base note list items.

```
0001 package com.example.mobile.ui.components
0002
0003 import androidx.compose.foundation.clickable
0004 import androidx.compose.foundation.layout.Arrangement
0005 import androidx.compose.foundation.layout.Column
0006 import androidx.compose.foundation.layout.fillMaxWidth
0007 import androidx.compose.foundation.layout.padding
0008 import androidx.compose.foundation.shape.RoundedCornerShape
0009 import androidx.compose.material3.Card
0010 import androidx.compose.material3.CardDefaults
0011 import androidx.compose.material3.Text
0012 import androidx.compose.runtime.Composable
0013 import androidx.compose.ui.Modifier
0014 import androidx.compose.ui.unit.dp
0015 import com.example.mobile.data.KnowledgeItem
0016 import com.example.mobile.ui.theme.NSyncBlue
0017 import com.example.mobile.ui.theme.NSyncCardWhite
0018 import com.example.mobile.ui.theme.NSyncMutedText
0019 import com.example.mobile.ui.theme.ScreenBodyStyle
0020 import com.example.mobile.ui.theme.ScreenCardBorder
0021 import com.example.mobile.ui.theme.ScreenSectionStyle
0022 import com.example.mobile.ui.theme.ScreenSmallBoldStyle
0023 import com.example.mobile.ui.theme.ScreenSmallStyle
0024 import com.example.mobile.ui.theme.ScreenTitle
0025
0026 @Composable
0027 fun KnowledgeListCard(
0028     item: KnowledgeItem,
0029     onClick: () -> Unit
0030 ) {
0031     Card(
0032         modifier = Modifier
0033             .fillMaxWidth()
0034             .clickable(onClick = onClick),
0035         colors = CardDefaults.cardColors(containerColor = NSyncCardWhite),
0036         shape = RoundedCornerShape(10.dp),
0037         border = ScreenCardBorder
0038     ) {
0039         Column(
0040             modifier = Modifier.padding(16.dp),
0041             verticalArrangement = Arrangement.spacedBy(6.dp)
0042         ) {
0043             Text(item.collection, color = NSyncBlue, style = ScreenSmallBoldStyle)
0044             Text(item.title, color = ScreenTitle, style = ScreenSectionStyle)
0045             Text(item.source, color = NSyncMutedText, style = ScreenBodyStyle)
0046             Text(
0047                 text = "${item.reviewCardCount} review cards • ${item.updatedLabel}",
0048                 color = NSyncMutedText,
0049                 style = ScreenSmallStyle
0050             )
0051         }
0052     }
0053 }
```

MainScreenScaffold.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/components/MainScreenScaffold.kt

Purpose: Shared Scaffold wrapper for main screens with bottom navigation.

```
0001 package com.example.mobile.ui.components
0002
0003 import androidx.compose.foundation.layout.Arrangement
0004 import androidx.compose.foundation.layout.Spacer
0005 import androidx.compose.foundation.layout.fillMaxSize
0006 import androidx.compose.foundation.layout.height
0007 import androidx.compose.foundation.layout.padding
0008 import androidx.compose.foundation.lazy.LazyColumn
0009 import androidx.compose.foundation.lazy.LazyListScope
0010 import androidx.compose.material3.Scaffold
0011 import androidx.compose.material3.Text
0012 import androidx.compose.runtime.Composable
0013 import androidx.compose.ui.Modifier
0014 import androidx.compose.ui.unit.dp
0015 import com.example.mobile.ui.theme.NSyncBlue
0016 import com.example.mobile.ui.theme.NSyncLightBackground
0017 import com.example.mobile.ui.theme.NSyncMutedText
0018 import com.example.mobile.ui.theme.ScreenBodyStyle
0019 import com.example.mobile.ui.theme.ScreenHeroStyle
0020 import com.example.mobile.ui.theme.ScreenLogoStyle
0021 import com.example.mobile.ui.theme.ScreenTitle
0022
0023 @Composable
0024 fun MainScreenScaffold(
0025     currentRoute: String,
0026     title: String,
0027     subtitle: String,
0028     onRouteClick: (String) -> Unit,
0029     content: LazyListScope() -> Unit
0030 ) {
0031     Scaffold(
0032         containerColor = NSyncLightBackground,
0033         bottomBar = {
0034             BottomNavBar(
0035                 currentRoute = currentRoute,
0036                 onRouteClick = onRouteClick
0037             )
0038         }
0039     ) { innerPadding ->
0040         LazyColumn(
0041             modifier = Modifier
0042                 .fillMaxSize()
0043                 .padding(innerPadding)
0044                 .padding(horizontal = 20.dp, vertical = 18.dp),
0045             verticalArrangement = Arrangement.spacedBy(16.dp)
0046         ) {
0047             item {
0048                 Text("NSync", color = NSyncBlue, style = ScreenLogoStyle)
0049                 Spacer(modifier = Modifier.height(16.dp))
0050                 Text(title, color = ScreenTitle, style = ScreenHeroStyle)
0051                 Text(subtitle, color = NSyncMutedText, style = ScreenBodyStyle)
0052             }
0053             content()
0054             item { Spacer(modifier = Modifier.height(74.dp)) }
0055         }
0056     }
0057 }
```

ReviewCardListItem.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/components/ReviewCardListItem.kt

Purpose: Reusable list item for individual review cards.

```
0001 package com.example.mobile.ui.components
0002
0003 import androidx.compose.foundation.clickable
0004 import androidx.compose.foundation.layout.Arrangement
0005 import androidx.compose.foundation.layout.Column
0006 import androidx.compose.foundation.layout.fillMaxWidth
0007 import androidx.compose.foundation.layout.padding
0008 import androidx.compose.foundation.shape.RoundedCornerShape
0009 import androidx.compose.material3.Card
0010 import androidx.compose.material3.CardDefaults
0011 import androidx.compose.material3.Text
0012 import androidx.compose.runtime.Composable
0013 import androidx.compose.ui.Modifier
0014 import androidx.compose.ui.text.style.TextOverflow
0015 import androidx.compose.ui.unit.dp
0016 import com.example.mobile.data.ReviewCard
0017 import com.example.mobile.ui.theme.NSyncBlue
0018 import com.example.mobile.ui.theme.NSyncCardWhite
0019 import com.example.mobile.ui.theme.NSyncMutedText
0020 import com.example.mobile.ui.theme.ScreenBodyStyle
0021 import com.example.mobile.ui.theme.ScreenCardBorder
0022 import com.example.mobile.ui.theme.ScreenSectionStyle
0023 import com.example.mobile.ui.theme.ScreenSmallBoldStyle
0024 import com.example.mobile.ui.theme.ScreenTitle
0025
0026 @Composable
0027 fun ReviewCardListItem(card: ReviewCard, onClick: () -> Unit) {
0028     Card(
0029         modifier = Modifier
0030             .fillMaxWidth()
0031             .clickable(onClick = onClick),
0032         colors = CardDefaults.cardColors(containerColor = NSyncCardWhite),
0033         shape = RoundedCornerShape(10.dp),
0034         border = ScreenCardBorder
0035     ) {
0036         Column(
0037             modifier = Modifier.padding(16.dp),
0038             verticalArrangement = Arrangement.spacedBy(6.dp)
0039         ) {
0040             Text(card.masteryLabel, color = NSyncBlue, style = ScreenSmallBoldStyle)
0041             Text(
0042                 text = card.question,
0043                 color = ScreenTitle,
0044                 style = ScreenSectionStyle,
0045                 maxLines = 2,
0046                 overflow = TextOverflow.Ellipsis
0047             )
0048             Text(card.collection, color = NSyncMutedText, style = ScreenBodyStyle)
0049             ProgressSummaryCard(
0050                 title = card.difficulty,
0051                 subtitle = "${card.masteryPercent}% mastery",
0052                 progress = card.masteryPercent / 100f
0053             )
0054         }
0055     }
0056 }
```


ReviewSessionListItem.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/components/ReviewSessionListItem.kt

Purpose: Reusable list item for grouped review sessions.

```
0001 package com.example.mobile.ui.components
0002
0003 import androidx.compose.foundation.layout.Arrangement
0004 import androidx.compose.foundation.layout.Column
0005 import androidx.compose.foundation.layout.Row
0006 import androidx.compose.foundation.layout.fillMaxWidth
0007 import androidx.compose.foundation.layout.height
0008 import androidx.compose.foundation.layout.padding
0009 import androidx.compose.foundation.shape.RoundedCornerShape
0010 import androidx.compose.material3.Button
0011 import androidx.compose.material3.ButtonDefaults
0012 import androidx.compose.material3.Card
0013 import androidx.compose.material3.CardDefaults
0014 import androidx.compose.material3.OutlinedButton
0015 import androidx.compose.material3.Text
0016 import androidx.compose.runtime.Composable
0017 import androidx.compose.ui.Modifier
0018 import androidx.compose.ui.graphics.Color
0019 import androidx.compose.ui.unit.dp
0020 import com.example.mobile.ui.theme.NSyncBlue
0021 import com.example.mobile.ui.theme.NSyncCardWhite
0022 import com.example.mobile.ui.theme.NSyncMutedText
0023 import com.example.mobile.ui.theme.ScreenButtonStyle
0024 import com.example.mobile.ui.theme.ScreenCardBorder
0025 import com.example.mobile.ui.theme.ScreenSectionStyle
0026 import com.example.mobile.ui.theme.ScreenSmallBoldStyle
0027 import com.example.mobile.ui.theme.ScreenTitle
0028
0029 @Composable
0030 fun ReviewSessionListItem(
0031     title: String,
0032     tag: String,
0033     cardCount: Int,
0034     onStartSessionClick: () -> Unit,
0035     onAddCardClick: () -> Unit
0036 ) {
0037     Card(
0038         modifier = Modifier.fillMaxWidth(),
0039         colors = CardDefaults.cardColors(containerColor = NSyncCardWhite),
0040         shape = RoundedCornerShape(14.dp),
0041         border = ScreenCardBorder
0042     ) {
0043         Column(
0044             modifier = Modifier.padding(18.dp),
0045             verticalArrangement = Arrangement.spacedBy(10.dp)
0046         ) {
0047             Text(title, color = ScreenTitle, style = ScreenSectionStyle)
0048             Text(
0049                 text = "$tag - $cardCount card${if (cardCount == 1) "" else "s"}",
0050                 color = NSyncMutedText,
0051                 style = ScreenSmallBoldStyle
0052             )
0053             Row(
0054                 modifier = Modifier.fillMaxWidth(),
0055                 horizontalArrangement = Arrangement.spacedBy(10.dp)
0056             ) {
0057                 Button(
0058                     onClick = onStartSessionClick,
0059                     modifier = Modifier
0060                         .weight(1f)
0061                         .height(44.dp),
0062                     shape = RoundedCornerShape(999.dp),
0063                     colors = ButtonDefaults.buttonColors(containerColor = NSyncBlue)
0064                 ) {
0065                     Text("Start Review", color = Color.White, style = ScreenButtonStyle)
0066                 }
0067                 OutlinedButton(
0068                     onClick = onAddCardClick,
0069                     modifier = Modifier
0070                         .weight(1f)
0071                         .height(44.dp),
0072                     shape = RoundedCornerShape(999.dp)
0073                 ) {
0074                     Text("Add Card", color = NSyncBlue, style = ScreenButtonStyle)
0075                 }
0076             }
0077         }
0078     }
0079 }
```

ScreenButtons.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/components/ScreenButtons.kt

Purpose: Shared button styling and reusable action buttons.

```
0001 package com.example.mobile.ui.components
0002
0003 import androidx.compose.foundation.layout.fillMaxWidth
0004 import androidx.compose.foundation.layout.height
0005 import androidx.compose.foundation.shape.RoundedCornerShape
0006 import androidx.compose.material3.Button
0007 import androidx.compose.material3.ButtonDefaults
0008 import androidx.compose.material3.Text
0009 import androidx.compose.runtime.Composable
0010 import androidx.compose.ui.Modifier
0011 import androidx.compose.ui.unit.dp
0012 import com.example.mobile.ui.theme.NSyncBlue
0013 import com.example.mobile.ui.theme.ScreenButtonStyle
0014
0015 @Composable
0016 fun PrimaryScreenButton(
0017     text: String,
0018     onClick: () -> Unit,
0019     modifier: Modifier = Modifier
0020 ) {
0021     Button(
0022         onClick = onClick,
0023         modifier = modifier
0024             .fillMaxWidth()
0025             .height(52.dp),
0026         shape = RoundedCornerShape(999.dp),
0027         colors = ButtonDefaults.buttonColors(containerColor = NSyncBlue)
0028     ) {
0029         Text(text, style = ScreenButtonStyle)
0030     }
0031 }
```

ScreenCards.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/components/ScreenCards.kt

Purpose: Shared card styling and reusable metric cards.

```
0001 package com.example.mobile.ui.components
0002
0003 import androidx.compose.foundation.layout.Arrangement
0004 import androidx.compose.foundation.layout.Box
0005 import androidx.compose.foundation.layout.Column
0006 import androidx.compose.foundation.layout.Row
0007 import androidx.compose.foundation.layout.fillMaxSize
0008 import androidx.compose.foundation.layout.fillMaxWidth
0009 import androidx.compose.foundation.layout.height
0010 import androidx.compose.foundation.layout.padding
0011 import androidx.compose.foundation.shape.RoundedCornerShape
0012 import androidx.compose.material3.Card
0013 import androidx.compose.material3.CardDefaults
0014 import androidx.compose.material3.LinearProgressIndicator
0015 import androidx.compose.material3.Text
0016 import androidx.compose.runtime.Composable
0017 import androidx.compose.ui.Alignment
0018 import androidx.compose.ui.Modifier
0019 import androidx.compose.ui.draw.clip
0020 import androidx.compose.ui.graphics.Color
0021 import androidx.compose.ui.unit.dp
0022 import com.example.mobile.ui.theme.NSyncBlue
0023 import com.example.mobile.ui.theme.NSyncCardWhite
0024 import com.example.mobile.ui.theme.NSyncMutedText
0025 import com.example.mobile.ui.theme.ScreenBodyStyle
0026 import com.example.mobile.ui.theme.ScreenCardBorder
0027 import com.example.mobile.ui.theme.ScreenSectionStyle
0028 import com.example.mobile.ui.theme.ScreenSmallBoldStyle
0029 import com.example.mobile.ui.theme.ScreenTitle
0030
0031 @Composable
0032 fun SummaryMetric(
0033     label: String,
0034     value: String,
0035     modifier: Modifier = Modifier
0036 ) {
0037     Card(
0038         modifier = modifier.height(88.dp),
0039         colors = CardDefaults.cardColors(containerColor = NSyncCardWhite),
0040         shape = RoundedCornerShape(10.dp),
0041         border = ScreenCardBorder
0042     ) {
0043         Column(
0044             modifier = Modifier
0045                 .fillMaxSize()
0046                 .padding(12.dp),
0047             verticalArrangement = Arrangement.Center
0048         ) {
0049             Text(label, color = NSyncMutedText, style = ScreenSmallBoldStyle)
0050             Text(value, color = ScreenTitle, style = ScreenSectionStyle)
0051         }
0052     }
0053 }
0054
0055 @Composable
0056 fun ProgressSummaryCard(
0057     title: String,
0058     subtitle: String,
0059     progress: Float,
0060     trailing: String? = null
0061 ) {
0062     Card(
0063         modifier = Modifier.fillMaxWidth(),
0064         colors = CardDefaults.cardColors(containerColor = NSyncCardWhite),
0065         shape = RoundedCornerShape(10.dp),
0066         border = ScreenCardBorder
0067     ) {
0068         Column(
0069             modifier = Modifier.padding(14.dp),
0070             verticalArrangement = Arrangement.spacedBy(8.dp)
0071         ) {
0072             Row(
0073                 modifier = Modifier.fillMaxWidth(),
0074                 horizontalArrangement = Arrangement.SpaceBetween,
0075                 verticalAlignment = Alignment.CenterVertically
0076             ) {
0077                 Text(title, color = ScreenTitle, style = ScreenSectionStyle)
0078                 trailing?.let { Text(it, color = NSyncBlue, style = ScreenSmallBoldStyle) }
0079             }
0080             Text(subtitle, color = NSyncMutedText, style = ScreenBodyStyle)
0081             LinearProgressIndicator(
0082                 progress = { progress },
0083                 modifier = Modifier
0084                     .fillMaxWidth()
0085                     .height(6.dp)
0086                     .clip(RoundedCornerShape(999.dp)),
0087                 color = NSyncBlue,
0088                 trackColor = Color(0xFFE2E5EA)
0089             )
0090         }
0091     }
0092 }
0093
0094 @Composable
0095 fun CenteredCard(
0096     modifier: Modifier = Modifier,
0097     content: @Composable () -> Unit
0098 ) {
```

```
0099     Card(  
0100         modifier = modifier,  
0101         colors = CardDefaults.cardColors(containerColor = NSyncCardWhite),  
0102         shape = RoundedCornerShape(24.dp),  
0103         border = ScreenCardBorder  
0104     ) {  
0105         Box(  
0106             modifier = Modifier  
0107                 .fillMaxSize()  
0108                 .padding(24.dp),  
0109             contentAlignment = Alignment.Center  
0110         ) {  
0111             content()  
0112         }  
0113     }  
0114 }
```

SettingsComponents.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/components/SettingsComponents.kt

Purpose: Reusable settings rows, grouped sections, dropdowns, and switches.

```
0001 package com.example.mobile.ui.components
0002
0003 import androidx.compose.foundation.clickable
0004 import androidx.compose.foundation.layout.Arrangement
0005 import androidx.compose.foundation.layout.Box
0006 import androidx.compose.foundation.layout.Column
0007 import androidx.compose.foundation.layout.Row
0008 import androidx.compose.foundation.layout.fillMaxWidth
0009 import androidx.compose.foundation.layout.padding
0010 import androidx.compose.foundation.shape.RoundedCornerShape
0011 import androidx.compose.material3.Card
0012 import androidx.compose.material3.CardDefaults
0013 import androidx.compose.material3.DropdownMenu
0014 import androidx.compose.material3.DropdownMenuItem
0015 import androidx.compose.material3.HorizontalDivider
0016 import androidx.compose.material3.Icon
0017 import androidx.compose.material3.Switch
0018 import androidx.compose.material3.Text
0019 import androidx.compose.runtime.Composable
0020 import androidx.compose.runtime.getValue
0021 import androidx.compose.runtime.mutableStateOf
0022 import androidx.compose.runtime.remember
0023 import androidx.compose.runtime.setValue
0024 import androidx.compose.ui.Alignment
0025 import androidx.compose.ui.Modifier
0026 import androidx.compose.ui.graphics.Color
0027 import androidx.compose.ui.res.painterResource
0028 import androidx.compose.ui.text.font.FontWeight
0029 import androidx.compose.ui.unit.dp
0030 import com.example.mobile.R
0031 import com.example.mobile.ui.theme.NSyncCardWhite
0032 import com.example.mobile.ui.theme.NSyncMutedText
0033 import com.example.mobile.ui.theme.ScreenBodyStyle
0034 import com.example.mobile.ui.theme.ScreenCardBorder
0035 import com.example.mobile.ui.theme.ScreenSectionStyle
0036
0037 @Composable
0038 fun SettingsSection(title: String, content: @Composable () -> Unit) {
0039     Column(verticalArrangement = Arrangement.spacedBy(10.dp)) {
0040         Text(
0041             text = title,
0042             color = Color(0xFF3D4556),
0043             style = ScreenBodyStyle,
0044             fontWeight = FontWeight.SemiBold
0045         )
0046         Card(
0047             modifier = Modifier.fillMaxWidth(),
0048             colors = CardDefaults.cardColors(containerColor = NSyncCardWhite),
0049             shape = RoundedCornerShape(14.dp),
0050             border = ScreenCardBorder
0051         ) {
0052             Column(content = { content() })
0053         }
0054     }
0055 }
0056
0057 @Composable
0058 fun SettingsRow(
0059     label: String,
0060     value: String = "",
0061     iconRes: Int,
0062     valueColor: Color = NSyncMutedText,
0063     onClick: (() -> Unit)? = null,
0064     showChevron: Boolean = false
0065 ) {
0066     Row(
0067         modifier = Modifier
0068             .fillMaxWidth()
0069             .then(if (onClick != null) Modifier.clickable(onClick = onClick) else Modifier)
0070             .padding(horizontal = 18.dp, vertical = 18.dp),
0071         horizontalArrangement = Arrangement.SpaceBetween,
0072         verticalAlignment = Alignment.CenterVertically
0073     ) {
0074         Row(
0075             verticalAlignment = Alignment.CenterVertically,
0076             horizontalArrangement = Arrangement.spacedBy(14.dp)
0077         ) {
0078             Icon(
0079                 painter = painterResource(iconRes),
0080                 contentDescription = null,
0081                 tint = Color(0xFFB8C0D0)
0082             )
0083             Text(label, color = Color(0xFF202431), style = ScreenSectionStyle)
0084         }
0085         if (showChevron) {
0086             Icon(
0087                 painter = painterResource(R.drawable.ic_chevron_right),
0088                 contentDescription = null
0089             )
0090         } else {
0091             Text(value, color = valueColor, style = ScreenBodyStyle)
0092         }
0093     }
0094 }
0095
0096 @Composable
0097 fun SettingsDropdownRow(
0098     label: String,
```

```

0099     value: String,
0100     iconRes: Int,
0101     options: List<String>,
0102     valueColor: Color = NSyncMutedText,
0103     onOptionSelected: (String) -> Unit
0104 ) {
0105     var expanded by remember { mutableStateOf(false) }
0106
0107     Box {
0108         SettingsRow(
0109             label = label,
0110             value = value,
0111             iconRes = iconRes,
0112             valueColor = valueColor,
0113             onClick = { expanded = true }
0114         )
0115         DropdownMenu(
0116             expanded = expanded,
0117             onDismissRequest = { expanded = false }
0118         ) {
0119             options.forEach { option ->
0120                 DropdownMenuItem(
0121                     text = { Text(option, style = ScreenBodyStyle) },
0122                     onClick = {
0123                         onOptionSelected(option)
0124                         expanded = false
0125                     }
0126                 )
0127             }
0128         }
0129     }
0130 }
0131
0132 @Composable
0133 fun SettingsToggleRow(
0134     label: String,
0135     iconRes: Int,
0136     checked: Boolean,
0137     onCheckedChange: (Boolean) -> Unit
0138 ) {
0139     Row(
0140         modifier = Modifier
0141             .fillMaxWidth()
0142             .padding(horizontal = 18.dp, vertical = 10.dp),
0143         horizontalArrangement = Arrangement.SpaceBetween,
0144         verticalAlignment = Alignment.CenterVertically
0145     ) {
0146         Row(
0147             verticalAlignment = Alignment.CenterVertically,
0148             horizontalArrangement = Arrangement.spacedBy(14.dp)
0149         ) {
0150             Icon(
0151                 painter = painterResource(iconRes),
0152                 contentDescription = null,
0153                 tint = Color(0xFFB8C0D0)
0154             )
0155             Text(label, color = Color(0xFF202431), style = ScreenSectionStyle)
0156         }
0157         Switch(checked = checked, onCheckedChange = onCheckedChange)
0158     }
0159 }
0160
0161 @Composable
0162 fun SettingsDivider() {
0163     HorizontalDivider(color = Color(0xFFE0E4ED))
0164 }

```

LoginScreen.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/screens/auth/LoginScreen.kt

Purpose: Login UI that collects credentials and calls the auth ViewModel.

```
0001 package com.example.mobile.ui.screens.auth
0002
0003 import androidx.compose.foundation.background
0004 import androidx.compose.foundation.Image
0005 import androidx.compose.foundation.border
0006 import androidx.compose.foundation.layout.Arrangement
0007 import androidx.compose.foundation.layout.Box
0008 import androidx.compose.foundation.layout.Column
0009 import androidx.compose.foundation.layout.Spacer
0010 import androidx.compose.foundation.layout.fillMaxSize
0011 import androidx.compose.foundation.layout.fillMaxWidth
0012 import androidx.compose.foundation.layout.height
0013 import androidx.compose.foundation.layout.offset
0014 import androidx.compose.foundation.layout.padding
0015 import androidx.compose.foundation.layout.size
0016 import androidx.compose.foundation.shape.RoundedCornerShape
0017 import androidx.compose.foundation.clickable
0018 import androidx.compose.material3.Button
0019 import androidx.compose.material3.ButtonDefaults
0020 import androidx.compose.material3.Card
0021 import androidx.compose.material3.CardDefaults
0022 import androidx.compose.material3.Text
0023 import androidx.compose.runtime.Composable
0024 import androidx.compose.runtime.getValue
0025 import androidx.compose.runtime.mutableStateOf
0026 import androidx.compose.runtime.remember
0027 import androidx.compose.runtime.setValue
0028 import androidx.compose.ui.Alignment
0029 import androidx.compose.ui.Modifier
0030 import androidx.compose.ui.draw.clip
0031 import androidx.compose.ui.draw.shadow
0032 import androidx.compose.ui.graphics.Color
0033 import androidx.compose.ui.layout.ContentScale
0034 import androidx.compose.ui.res.painterResource
0035 import androidx.compose.ui.text.TextStyle
0036 import androidx.compose.ui.text.font.FontWeight
0037 import androidx.compose.ui.text.style.TextAlign
0038 import androidx.compose.ui.unit.dp
0039 import androidx.compose.ui.unit.sp
0040 import com.example.mobile.R
0041 import com.example.mobile.ui.components.AuthTextField
0042 import com.example.mobile.ui.theme.InterFontFamily
0043 import com.example.mobile.ui.theme.NSyncBlue
0044 import com.example.mobile.ui.theme.NSyncCardWhite
0045
0046
0047 @Composable
0048 fun LoginScreen(
0049     onLoginClick: (String, String) -> Unit,
0050     onRegisterClick: () -> Unit,
0051     isLoading: Boolean,
0052     errorMessage: String?,
0053 ) {
0054     var email by remember { mutableStateOf("") }
0055     var password by remember { mutableStateOf("") }
0056
0057     Column(
0058         modifier = Modifier
0059             .fillMaxSize()
0060             .background(LoginBackground)
0061             .padding(horizontal = 20.dp, vertical = 46.dp),
0062         horizontalAlignment = Alignment.CenterHorizontally
0063     ) {
0064         Spacer(modifier = Modifier.height(0.dp))
0065
0066         AppLogo()
0067
0068         Spacer(modifier = Modifier.height(24.dp))
0069
0070         Text(
0071             text = "NSync",
0072             color = NSyncBlue,
0073             fontWeight = FontWeight.ExtraBold,
0074             style = TextStyle(
0075                 fontFamily = InterFontFamily,
0076                 fontSize = 29.sp,
0077                 lineHeight = 34.sp,
0078                 letterSpacing = 0.sp
0079             )
0080         )
0081
0082         Text(
0083             text = "Capture knowledge. Review smarter. Level up.",
0084             modifier = Modifier.padding(top = 10.dp),
0085             color = LoginBody,
0086             style = TextStyle(
0087                 fontFamily = InterFontFamily,
0088                 fontSize = 13.sp,
0089                 lineHeight = 18.sp,
0090                 fontWeight = FontWeight.Normal,
0091                 letterSpacing = 0.sp
0092             ),
0093             textAlign = TextAlign.Center
0094         )
0095
0096         Spacer(modifier = Modifier.height(28.dp))
0097
0098         Card(
```

```

0099         modifier = Modifier.fillMaxWidth(),
0100         colors = CardDefaults.cardColors(containerColor = NSyncCardWhite),
0101         shape = RoundedCornerShape(26.dp),
0102         elevation = CardDefaults.cardElevation(defaultElevation = 10.dp)
0103     ) {
0104         Column(
0105             modifier = Modifier.padding(horizontal = 28.dp, vertical = 28.dp),
0106             verticalArrangement = Arrangement.spacedBy(10.dp)
0107         ) {
0108             Text(
0109                 text = "Email Address",
0110                 color = LoginBody,
0111                 style = LoginLabelStyle
0112             )
0113
0114             AuthTextField(
0115                 value = email,
0116                 onValueChange = { email = it },
0117                 placeholder = "name@example.com",
0118                 leadingIcon = R.drawable.ic_mail_outline
0119             )
0120
0121             Spacer(modifier = Modifier.height(8.dp))
0122
0123             Text(
0124                 text = "Password",
0125                 color = LoginBody,
0126                 style = LoginLabelStyle
0127             )
0128
0129             AuthTextField(
0130                 value = password,
0131                 onValueChange = { password = it },
0132                 placeholder = ".....",
0133                 leadingIcon = R.drawable.ic_lock_outline,
0134                 isPassword = true
0135             )
0136
0137             Spacer(modifier = Modifier.height(10.dp))
0138
0139             Button(
0140                 onClick = { onLoginClick(email.trim(), password) },
0141                 enabled = !isLoading && email.isNotBlank() && password.isNotBlank(),
0142                 modifier = Modifier
0143                     .fillMaxWidth()
0144                     .height(56.dp)
0145                     .shadow(8.dp, RoundedCornerShape(999.dp)),
0146                 shape = RoundedCornerShape(999.dp),
0147                 colors = ButtonDefaults.buttonColors(
0148                     containerColor = NSyncBlue,
0149                     disabledContainerColor = NSyncBlue.copy(alpha = 0.45f),
0150                     disabledContentColor = Color.White
0151                 ),
0152                 elevation = ButtonDefaults.buttonElevation(defaultElevation = 0.dp)
0153             ) {
0154                 Text(
0155                     text = if (isLoading) "Logging in..." else "Login",
0156                     style = TextStyle(
0157                         fontFamily = InterFontFamily,
0158                         color = Color.White,
0159                         fontSize = 15.sp,
0160                         lineHeight = 20.sp,
0161                         fontWeight = FontWeight.SemiBold,
0162                         letterSpacing = 0.sp
0163                     )
0164                 )
0165             }
0166
0167             if (!errorMessage.isNullOrBlank()) {
0168                 Text(
0169                     text = errorMessage,
0170                     color = Color(0xFFB3261E),
0171                     modifier = Modifier.padding(top = 8.dp),
0172                 )
0173             }
0174         }
0175     }
0176
0177     Spacer(modifier = Modifier.weight(1f))
0178
0179     Text(
0180         text = "New to NSync?",
0181         color = LoginBody,
0182         style = LoginFooterStyle,
0183     )
0184
0185     Text(
0186         text = "Create account",
0187         modifier = Modifier
0188             .padding(top = 8.dp)
0189             .clickable(onClick = onRegisterClick),
0190         color = NSyncBlue,
0191         style = LoginCreateAccountStyle
0192     )
0193 }
0194
0195 @Composable
0196 private fun AppLogo() {
0197     Box(
0198         modifier = Modifier
0199             .size(84.dp)
0200             .offset(y = 4.dp)
0201             .shadow(12.dp, RoundedCornerShape(24.dp))
0202             .clip(RoundedCornerShape(24.dp))
0203             .background(NSyncCardWhite)
0204             .border(2.dp, NSyncBlue, RoundedCornerShape(24.dp)),
0205         contentAlignment = Alignment.Center
0206     ) {
0207         Image(
0208             painter = painterResource(id = R.drawable.nsync_logo),
0209             contentDescription = "NSync logo",
0210

```



```
0211         modifier = Modifier.fillMaxSize(),
0212         contentScale = ContentScale.Fit
0213     )
0214 }
0215 }
0216
0217 private val LoginBackground = Color(0xFFEFF4FF)
0218 private val LoginBody = Color(0xFF424858)
0219
0220 private val LoginLabelStyle = TextStyle(
0221     fontFamily = InterFontFamily,
0222     fontSize = 18.sp,
0223     lineHeight = 15.sp,
0224     fontWeight = FontWeight.Bold,
0225     letterSpacing = 0.2.sp
0226 )
0227
0228 private val LoginFooterStyle = TextStyle(
0229     fontFamily = InterFontFamily,
0230     fontSize = 15.sp,
0231     lineHeight = 11.sp,
0232     fontWeight = FontWeight.Bold,
0233     letterSpacing = 0.sp
0234 )
0235
0236 private val LoginCreateAccountStyle = TextStyle(
0237     fontFamily = InterFontFamily,
0238     fontSize = 15.sp,
0239     lineHeight = 11.sp,
0240     fontWeight = FontWeight.Normal,
0241     letterSpacing = 0.sp
0242 )
```

RegisterScreen.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/screens/auth/RegisterScreen.kt

Purpose: Registration UI that collects account details and calls the auth ViewModel.

```
0001 package com.example.mobile.ui.screens.auth
0002
0003 import androidx.compose.foundation.background
0004 import androidx.compose.foundation.layout.Column
0005 import androidx.compose.foundation.layout.Row
0006 import androidx.compose.foundation.layout.Spacer
0007 import androidx.compose.foundation.layout.fillMaxSize
0008 import androidx.compose.foundation.layout.fillMaxWidth
0009 import androidx.compose.foundation.layout.height
0010 import androidx.compose.foundation.layout.padding
0011 import androidx.compose.foundation.shape.RoundedCornerShape
0012 import androidx.compose.material3.Button
0013 import androidx.compose.material3.ButtonDefaults
0014 import androidx.compose.material3.Text
0015 import androidx.compose.material3.TextButton
0016 import androidx.compose.runtime.Composable
0017 import androidx.compose.runtime.getValue
0018 import androidx.compose.runtime.mutableStateOf
0019 import androidx.compose.runtime.remember
0020 import androidx.compose.runtime.setValue
0021 import androidx.compose.ui.Alignment
0022 import androidx.compose.ui.Modifier
0023 import androidx.compose.ui.draw.shadow
0024 import androidx.compose.ui.graphics.Color
0025 import androidx.compose.ui.text.TextStyle
0026 import androidx.compose.ui.text.font.FontWeight
0027 import androidx.compose.ui.text.style.TextAlign
0028 import androidx.compose.ui.unit.dp
0029 import androidx.compose.ui.unit.sp
0030 import com.example.mobile.ui.components.AuthTextField
0031 import com.example.mobile.ui.theme.InterFontFamily
0032 import com.example.mobile.ui.theme.NSyncBlue
0033 import com.example.mobile.ui.theme.NSyncLightBackground
0034 import com.example.mobile.ui.theme.NSyncMutedText
0035
0036
0037 @Composable
0038 fun RegisterScreen(
0039     onRegisterClick: (String, String, String) -> Unit,
0040     onLoginClick: () -> Unit,
0041     isLoading: Boolean,
0042     errorMessage: String?,
0043 ) {
0044     var name by remember { mutableStateOf("") }
0045     var email by remember { mutableStateOf("") }
0046     var password by remember { mutableStateOf("") }
0047
0048     Column(
0049         modifier = Modifier
0050             .fillMaxSize()
0051             .background(NSyncLightBackground)
0052             .padding(horizontal = 24.dp, vertical = 36.dp),
0053         horizontalAlignment = Alignment.CenterHorizontally
0054     ) {
0055         Spacer(modifier = Modifier.height(28.dp))
0056
0057         Text(
0058             text = "NSync",
0059             color = NSyncBlue,
0060             style = RegisterLogoStyle
0061         )
0062
0063         Spacer(modifier = Modifier.height(42.dp))
0064
0065         Text(
0066             text = "Create Account",
0067             modifier = Modifier
0068                 .fillMaxWidth()
0069                 .padding(top = 10.dp, bottom = 10.dp),
0070             color = RegisterTitle,
0071             style = RegisterTitleStyle
0072         )
0073
0074         Text(
0075             text = "Start building your personal knowledge base.",
0076             modifier = Modifier.fillMaxWidth(),
0077             color = NSyncMutedText,
0078             style = RegisterBodyStyle
0079         )
0080
0081         Spacer(modifier = Modifier.height(30.dp))
0082
0083         RegisterLabel("Full Name")
0084         AuthTextField(
0085             value = name,
0086             onValueChange = { name = it },
0087             placeholder = "Enter your name"
0088         )
0089
0090         Spacer(modifier = Modifier.height(14.dp))
0091
0092         RegisterLabel("Email Address")
0093         AuthTextField(
0094             value = email,
0095             onValueChange = { email = it },
0096             placeholder = "name@example.com"
0097         )
0098     }
```

```

0099         Spacer(modifier = Modifier.height(14.dp))
0100
0101         RegisterLabel("Password")
0102         AuthTextField(
0103             value = password,
0104             onValueChange = { password = it },
0105             placeholder = "Password",
0106             isPassword = true
0107         )
0108
0109         Spacer(modifier = Modifier.height(28.dp))
0110
0111         Button(
0112             onClick = { onRegisterClick(name.trim(), email.trim(), password) },
0113             enabled = !isLoading &&
0114                 name.isNotBlank() &&
0115                 email.isNotBlank() &&
0116                 password.isNotBlank(),
0117             modifier = Modifier
0118                 .fillMaxWidth()
0119                 .height(56.dp)
0120                 .shadow(8.dp, RoundedCornerShape(999.dp)),
0121             shape = RoundedCornerShape(999.dp),
0122             colors = ButtonDefaults.buttonColors(
0123                 containerColor = NSyncBlue,
0124                 contentColor = Color.White,
0125                 disabledContainerColor = NSyncBlue.copy(alpha = 0.45f),
0126                 disabledContentColor = Color.White
0127             ),
0128             elevation = ButtonDefaults.buttonElevation(defaultElevation = 0.dp)
0129         ) {
0130             Text(
0131                 text = if (isLoading) "Creating account..." else "Create Account",
0132                 style = RegisterButtonStyle
0133             )
0134         }
0135
0136         if (!errorMessage.isNullOrBlank()) {
0137             Text(
0138                 text = errorMessage,
0139                 color = Color(0xFFB3261E),
0140                 modifier = Modifier.padding(top = 8.dp),
0141             )
0142         }
0143
0144         Spacer(modifier = Modifier.height(28.dp))
0145
0146         Row(verticalAlignment = Alignment.CenterVertically) {
0147             Text(
0148                 text = "Already have an account?",
0149                 color = NSyncMutedText,
0150                 style = RegisterSmallStyle,
0151                 textAlign = TextAlign.Center
0152             )
0153
0154             TextButton(onClick = onLoginClick) {
0155                 Text(
0156                     text = "Login",
0157                     color = NSyncBlue,
0158                     style = RegisterLinkStyle
0159                 )
0160             }
0161         }
0162     }
0163 }
0164
0165 @Composable
0166 private fun RegisterLabel(text: String) {
0167     Text(
0168         text = text,
0169         modifier = Modifier.fillMaxWidth(),
0170         color = NSyncMutedText,
0171         style = RegisterLabelStyle
0172     )
0173 }
0174
0175 private val RegisterTitle = Color(0xFF151927)
0176
0177 private val RegisterLogoStyle = TextStyle(
0178     fontFamily = InterFontFamily,
0179     fontSize = 35.sp,
0180     lineHeight = 42.sp,
0181     fontWeight = FontWeight.ExtraBold
0182 )
0183
0184 private val RegisterTitleStyle = TextStyle(
0185     fontFamily = InterFontFamily,
0186     fontSize = 32.sp,
0187     lineHeight = 38.sp,
0188     fontWeight = FontWeight.ExtraBold
0189 )
0190
0191 private val RegisterBodyStyle = TextStyle(
0192     fontFamily = InterFontFamily,
0193     fontSize = 14.sp,
0194     lineHeight = 20.sp,
0195     fontWeight = FontWeight.Normal
0196 )
0197
0198 private val RegisterLabelStyle = TextStyle(
0199     fontFamily = InterFontFamily,
0200     fontSize = 13.sp,
0201     lineHeight = 18.sp,
0202     fontWeight = FontWeight.SemiBold
0203 )
0204
0205 private val RegisterButtonStyle = TextStyle(
0206     fontFamily = InterFontFamily,
0207     fontSize = 16.sp,
0208     lineHeight = 20.sp,
0209     fontWeight = FontWeight.Bold
0210 )

```

```
0211
0212 private val RegisterSmallStyle = TextStyle(
0213     fontFamily = InterFontFamily,
0214     fontSize = 13.sp,
0215     lineHeight = 18.sp,
0216     fontWeight = FontWeight.Normal
0217 )
0218
0219 private val RegisterLinkStyle = TextStyle(
0220     fontFamily = InterFontFamily,
0221     fontSize = 13.sp,
0222     lineHeight = 18.sp,
0223     fontWeight = FontWeight.Bold
0224 )
```

DashboardScreen.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/screens/dashboard/DashboardScreen.kt

Purpose: Dashboard UI showing user progress, recent knowledge, and quick review actions.

```
0001 package com.example.mobile.ui.screens.dashboard
0002
0003 import androidx.compose.foundation.BorderStroke
0004 import androidx.compose.foundation.Image
0005 import androidx.compose.foundation.background
0006 import androidx.compose.foundation.border
0007 import androidx.compose.foundation.clickable
0008 import androidx.compose.foundation.layout.Arrangement
0009 import androidx.compose.foundation.layout.Box
0010 import androidx.compose.foundation.layout.Column
0011 import androidx.compose.foundation.layout.Row
0012 import androidx.compose.foundation.layout.Spacer
0013 import androidx.compose.foundation.layout.fillMaxSize
0014 import androidx.compose.foundation.layout.fillMaxWidth
0015 import androidx.compose.foundation.layout.height
0016 import androidx.compose.foundation.layout.padding
0017 import androidx.compose.foundation.layout.size
0018 import androidx.compose.foundation.layout.width
0019 import androidx.compose.foundation.lazy.LazyColumn
0020 import androidx.compose.foundation.shape.CircleShape
0021 import androidx.compose.foundation.shape.RoundedCornerShape
0022 import androidx.compose.material3.Card
0023 import androidx.compose.material3.CardDefaults
0024 import androidx.compose.material3.Icon
0025 import androidx.compose.material3.LinearProgressIndicator
0026 import androidx.compose.material3.Scaffold
0027 import androidx.compose.material3.Surface
0028 import androidx.compose.material3.Text
0029 import androidx.compose.runtime.Composable
0030 import androidx.compose.ui.Alignment
0031 import androidx.compose.ui.Modifier
0032 import androidx.compose.ui.draw.clip
0033 import androidx.compose.ui.draw.shadow
0034 import androidx.compose.ui.graphics.Color
0035 import androidx.compose.ui.layout.ContentScale
0036 import androidx.compose.ui.res.painterResource
0037 import androidx.compose.ui.text.TextStyle
0038 import androidx.compose.ui.text.font.FontWeight
0039 import androidx.compose.ui.text.style.TextOverflow
0040 import androidx.compose.ui.unit.dp
0041 import androidx.compose.ui.unit.sp
0042 import androidx.lifecycle.viewmodel.compose.viewModel
0043 import com.example.mobile.R
0044 import com.example.mobile.data.KnowledgeItem
0045 import com.example.mobile.navigation.Routes
0046 import com.example.mobile.ui.components.BottomNavBar
0047 import com.example.mobile.ui.theme.InterFontFamily
0048 import com.example.mobile.ui.theme.NSyncBlue
0049 import com.example.mobile.ui.theme.NSyncCardWhite
0050 import com.example.mobile.ui.theme.NSyncLightBackground
0051 import com.example.mobile.ui.theme.NSyncMutedText
0052 import com.example.mobile.ui.theme.NSyncRed
0053 import com.example.mobile.ui.viewmodel.DashboardViewModel
0054 import com.example.mobile.data.remote.dto.UserProgressDto
0055 import kotlin.math.roundToInt
0056
0057
0058 @Composable
0059 fun DashboardScreen(
0060     onStartReviewClick: () -> Unit = {},
0061     onKnowledgeClick: (KnowledgeItem) -> Unit = {},
0062     onAddClick: () -> Unit = {},
0063     onRouteClick: (String) -> Unit = {},
0064     dashboardViewModel: DashboardViewModel = viewModel()
0065 ) {
0066     val uiState = dashboardViewModel.uiState
0067
0068     Scaffold(
0069         containerColor = NSyncLightBackground,
0070         bottomBar = {
0071             BottomNavBar(
0072                 currentRoute = Routes.DASHBOARD,
0073                 onRouteClick = onRouteClick
0074             )
0075         }
0076     ) { innerPadding ->
0077         LazyColumn(
0078             modifier = Modifier
0079                 .fillMaxSize()
0080                 .padding(innerPadding)
0081                 .padding(horizontal = 20.dp),
0082             verticalArrangement = Arrangement.spacedBy(18.dp)
0083         ) {
0084             item {
0085                 Spacer(modifier = Modifier.height(10.dp))
0086                 DashboardTopNav()
0087             }
0088
0089             item {
0090                 DashboardGreeting(uiState.displayName)
0091             }
0092
0093             if (uiState.isLoading) {
0094                 item {
0095                     Text(
0096                         text = "Loading dashboard...",
0097                         color = NSyncMutedText,
0098                         style = DashboardBodyStyle
```

```

0099         )
0100     }
0101 }
0102
0103 uiState.error?.let { message ->
0104     item {
0105         Text(
0106             text = "Unable to load dashboard: $message",
0107             color = Color(0xFFD21F2B),
0108             style = DashboardBodyStyle
0109         )
0110     }
0111 }
0112
0113 item {
0114     LevelProgressCard(uiState.progress)
0115 }
0116
0117 item {
0118     DashboardStatsRow(
0119         streakDays = uiState.progress?.streak ?: 0,
0120         accuracyPercent = uiState.progress?.accuracy?.roundToInt() ?: 0
0121     )
0122 }
0123
0124 item {
0125     DashboardActions(
0126         onStartReviewClick = onStartReviewClick,
0127         onAddClick = onAddClick
0128     )
0129 }
0130
0131 item {
0132     RecentKnowledgeSection(
0133         recentKnowledge = uiState.recentKnowledge.firstOrNull(),
0134         onKnowledgeClick = onKnowledgeClick
0135     )
0136 }
0137
0138 item {
0139     Spacer(modifier = Modifier.height(80.dp))
0140 }
0141 }
0142 }
0143 }
0144
0145 @Composable
0146 private fun DashboardTopNav() {
0147     Row(
0148         modifier = Modifier
0149             .fillMaxWidth()
0150             .height(44.dp),
0151         horizontalArrangement = Arrangement.SpaceBetween,
0152         verticalAlignment = Alignment.CenterVertically
0153     ) {
0154         Text(
0155             text = "NSync",
0156             color = NSyncBlue,
0157             style = DashboardLogoStyle
0158         )
0159
0160         Box(
0161             modifier = Modifier
0162                 .size(34.dp)
0163                 .clip(CircleShape)
0164                 .background(Color(0xFFE8F1FF))
0165                 .border(2.dp, NSyncBlue, CircleShape),
0166             contentAlignment = Alignment.Center
0167         ) {
0168             Image(
0169                 painter = painterResource(id = R.drawable.nsync_logo),
0170                 contentDescription = "User profile",
0171                 modifier = Modifier
0172                     .size(26.dp)
0173                     .clip(CircleShape),
0174                 contentScale = ContentScale.Fit
0175             )
0176         }
0177     }
0178 }
0179
0180 @Composable
0181 private fun DashboardGreeting(name: String) {
0182     Column(verticalArrangement = Arrangement.spacedBy(4.dp)) {
0183         Text(
0184             text = "Hello, $name!",
0185             color = DashboardTitle,
0186             style = DashboardHeadlineStyle
0187         )
0188         Text(
0189             text = "Ready to strengthen what matters today?",
0190             color = NSyncMutedText,
0191             style = DashboardBodyStyle
0192         )
0193     }
0194 }
0195
0196 @Composable
0197 private fun LevelProgressCard(progress: UserProgressDto?) {
0198     val level = progress?.level ?: 1
0199     val totalXp = progress?.totalXp ?: 0
0200     val levelStartXp = levelStartXp(level)
0201     val nextLevelXp = nextLevelXp(level)
0202     val xpToNextLevel = nextLevelXp?.let { (it - totalXp).coerceAtLeast(0) }
0203     val progressPercent = nextLevelXp?.let { nextXp ->
0204         ((totalXp - levelStartXp).toFloat() / (nextXp - levelStartXp))
0205         .coerceIn(0f, 1f)
0206     } ?: 1f
0207
0208     Card(
0209         modifier = Modifier.fillMaxWidth(),
0210         colors = CardDefaults.cardColors(containerColor = NSyncCardWhite),

```

```

0211     shape = RoundedRectangleBorder(8.dp),
0212     elevation = CardDefaults.cardElevation(defaultElevation = 1.dp),
0213     border = DashboardCardBorder
0214 ) {
0215     Column(
0216         modifier = Modifier.padding(horizontal = 14.dp, vertical = 12.dp),
0217         verticalArrangement = Arrangement.spacedBy(6.dp)
0218     ) {
0219         Text(
0220             text = "LEVEL",
0221             color = NSyncMutedText,
0222             style = DashboardMicroLabelStyle
0223         )
0224
0225         Row(
0226             modifier = Modifier.fillMaxWidth(),
0227             horizontalArrangement = Arrangement.SpaceBetween,
0228             verticalAlignment = Alignment.Bottom
0229         ) {
0230             Text(
0231                 text = "Level $level",
0232                 color = NSyncBlue,
0233                 style = DashboardLevelStyle
0234             )
0235             Text(
0236                 text = xpToNextLevel?.let {
0237                     "${formatNumber(totalXp)} XP • $it to Level ${level + 1}"
0238                 } ?: "${formatNumber(totalXp)} XP • Maximum level",
0239                 color = NSyncMutedText,
0240                 style = DashboardSmallBoldStyle
0241             )
0242         }
0243
0244         LinearProgressIndicator(
0245             progress = { progressPercent },
0246             modifier = Modifier
0247                 .fillMaxWidth()
0248                 .height(6.dp)
0249                 .clip(RoundedCornerShape(999.dp)),
0250             color = Color(0xFF7D889A),
0251             trackColor = Color(0xFFE2E5EA)
0252         )
0253     }
0254 }
0255 }
0256
0257 @Composable
0258 private fun DashboardStatsRow(streakDays: Int, accuracyPercent: Int) {
0259     Row(
0260         modifier = Modifier.fillMaxWidth(),
0261         horizontalArrangement = Arrangement.spacedBy(16.dp)
0262     ) {
0263         StatCard(
0264             modifier = Modifier.weight(1f),
0265             icon = R.drawable.ic_wind,
0266             iconTint = NSyncRed,
0267             value = streakDays.toString(),
0268             label = "Day streak"
0269         )
0270
0271         StatCard(
0272             modifier = Modifier.weight(1f),
0273             icon = R.drawable.ic_target,
0274             iconTint = NSyncBlue,
0275             value = "$accuracyPercent%",
0276             label = "Accuracy"
0277         )
0278     }
0279 }
0280
0281 @Composable
0282 private fun StatCard(
0283     modifier: Modifier,
0284     icon: Int,
0285     iconTint: Color,
0286     value: String,
0287     label: String
0288 ) {
0289     Card(
0290         modifier = modifier.height(84.dp),
0291         colors = CardDefaults.cardColors(containerColor = NSyncCardWhite),
0292         shape = RoundedRectangleBorder(8.dp),
0293         elevation = CardDefaults.cardElevation(defaultElevation = 1.dp),
0294         border = DashboardCardBorder
0295     ) {
0296         Column(
0297             modifier = Modifier
0298                 .fillMaxSize()
0299                 .padding(vertical = 12.dp),
0300             horizontalAlignment = Alignment.CenterHorizontally,
0301             verticalArrangement = Arrangement.Center
0302         ) {
0303             Icon(
0304                 painter = painterResource(id = icon),
0305                 contentDescription = null,
0306                 modifier = Modifier.size(28.dp),
0307                 tint = iconTint
0308             )
0309             Text(
0310                 text = value,
0311                 color = DashboardTitle,
0312                 style = DashboardStatValueStyle
0313             )
0314             Text(
0315                 text = label,
0316                 color = NSyncMutedText,
0317                 style = DashboardMicroBoldStyle
0318             )
0319         }
0320     }
0321 }
0322

```

```

0323 @Composable
0324 private fun DashboardActions(
0325     onStartReviewClick: () -> Unit,
0326     onAddClick: () -> Unit
0327 ) {
0328     Row(
0329         modifier = Modifier.fillMaxWidth(),
0330         horizontalArrangement = Arrangement.spacedBy(14.dp),
0331         verticalAlignment = Alignment.CenterVertically
0332     ) {
0333         Surface(
0334             modifier = Modifier
0335                 .weight(1f)
0336                 .height(48.dp)
0337                 .shadow(10.dp, RoundedCornerShape(999.dp))
0338                 .clickable(onClick = onStartReviewClick),
0339             color = NSyncBlue,
0340             shape = RoundedCornerShape(999.dp)
0341         ) {
0342             Box(contentAlignment = Alignment.Center) {
0343                 Text(
0344                     text = "► Start Review",
0345                     color = Color.White,
0346                     style = DashboardActionStyle
0347                 )
0348             }
0349         }
0350
0351         Surface(
0352             modifier = Modifier
0353                 .size(48.dp)
0354                 .clickable(onClick = onAddClick),
0355             color = Color.Transparent,
0356             shape = CircleShape,
0357             border = BorderStroke(2.dp, NSyncBlue)
0358         ) {
0359             Box(contentAlignment = Alignment.Center) {
0360                 Text(
0361                     text = "+",
0362                     color = NSyncBlue,
0363                     style = TextStyle(
0364                         fontFamily = InterFontFamily,
0365                         fontSize = 28.sp,
0366                         lineHeight = 28.sp,
0367                         fontWeight = FontWeight.Normal
0368                     )
0369                 )
0370             }
0371         }
0372     }
0373 }
0374
0375 @Composable
0376 private fun RecentKnowledgeSection(
0377     recentKnowledge: KnowledgeItem?,
0378     onKnowledgeClick: (KnowledgeItem) -> Unit
0379 ) {
0380     Column(verticalArrangement = Arrangement.spacedBy(12.dp)) {
0381         Text(
0382             text = "Recent Knowledge",
0383             color = DashboardTitle,
0384             style = DashboardSectionTitleStyle
0385         )
0386
0387         if (recentKnowledge == null) {
0388             Text(
0389                 text = "No knowledge saved yet.",
0390                 color = NSyncMutedText,
0391                 style = DashboardBodyStyle
0392             )
0393             return@Column
0394         }
0395
0396         Card(
0397             modifier = Modifier
0398                 .fillMaxWidth()
0399                 .clickable { onKnowledgeClick(recentKnowledge) },
0400             colors = CardDefaults.cardColors(containerColor = NSyncCardWhite),
0401             shape = RoundedCornerShape(8.dp),
0402             elevation = CardDefaults.cardElevation(defaultElevation = 1.dp),
0403             border = DashboardCardBorder
0404         ) {
0405             Row(
0406                 modifier = Modifier.padding(14.dp),
0407                 verticalAlignment = Alignment.CenterVertically
0408             ) {
0409                 Box(
0410                     modifier = Modifier
0411                         .size(40.dp)
0412                         .clip(RoundedCornerShape(8.dp))
0413                         .background(Color(0xFFF0F3F8)),
0414                     contentAlignment = Alignment.Center
0415                 ) {
0416                     Icon(
0417                         painter = painterResource(id = R.drawable.ic_briefcase),
0418                         contentDescription = null,
0419                         modifier = Modifier.size(22.dp),
0420                         tint = NSyncMutedText
0421                     )
0422                 }
0423
0424                 Spacer(modifier = Modifier.width(14.dp))
0425
0426                 Column(
0427                     modifier = Modifier.weight(1f),
0428                     verticalArrangement = Arrangement.spacedBy(3.dp)
0429                 ) {
0430                     Text(
0431                         text = recentKnowledge.title,
0432                         color = DashboardTitle,
0433                         style = DashboardKnowledgeTitleStyle,
0434                         maxLines = 2,

```



```

0435         overflow = TextOverflow.Ellipsis
0436     )
0437     Text(
0438         text = recentKnowledge.collection,
0439         color = NSyncMutedText,
0440         style = DashboardBodyStyle
0441     )
0442     Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
0443         InfoPill("${recentKnowledge.reviewCardCount} Cards")
0444         InfoPill("${recentKnowledge.xpEarned} XP")
0445     }
0446 }
0447
0448 Text(
0449     text = ">",
0450     color = Color(0xFFC8CEDA),
0451     style = TextStyle(
0452         fontFamily = InterFontFamily,
0453         fontSize = 28.sp,
0454         lineHeight = 28.sp,
0455         fontWeight = FontWeight.Normal
0456     )
0457 )
0458 }
0459 }
0460 }
0461 }
0462
0463 @Composable
0464 private fun InfoPill(text: String) {
0465     Box(
0466         modifier = Modifier
0467             .clip(RoundedCornerShape(999.dp))
0468             .background(Color(0xFFE8ECF3))
0469             .padding(horizontal = 8.dp, vertical = 3.dp)
0470     ) {
0471         Text(
0472             text = text,
0473             color = NSyncMutedText,
0474             style = DashboardPillStyle
0475         )
0476     }
0477 }
0478
0479 private fun levelStartXp(level: Int): Int = when (level) {
0480     5 -> 1000
0481     4 -> 500
0482     3 -> 250
0483     2 -> 100
0484     else -> 0
0485 }
0486
0487 private fun nextLevelXp(level: Int): Int? = when (level) {
0488     1 -> 100
0489     2 -> 250
0490     3 -> 500
0491     4 -> 1000
0492     else -> null
0493 }
0494
0495 private fun formatNumber(value: Int): String = "%,d".format(value)
0496
0497 private val DashboardTitle = Color(0xFF151927)
0498 private val DashboardCardBorder = BorderStroke(1.dp, Color(0xFFE9EDF5))
0499
0500 private val DashboardLogoStyle = TextStyle(
0501     fontFamily = InterFontFamily,
0502     fontSize = 21.sp,
0503     lineHeight = 26.sp,
0504     fontWeight = FontWeight.ExtraBold
0505 )
0506
0507 private val DashboardHeadlineStyle = TextStyle(
0508     fontFamily = InterFontFamily,
0509     fontSize = 28.sp,
0510     lineHeight = 34.sp,
0511     fontWeight = FontWeight.ExtraBold
0512 )
0513
0514 private val DashboardBodyStyle = TextStyle(
0515     fontFamily = InterFontFamily,
0516     fontSize = 14.sp,
0517     lineHeight = 18.sp,
0518     fontWeight = FontWeight.Normal
0519 )
0520
0521 private val DashboardMicroLabelStyle = TextStyle(
0522     fontFamily = InterFontFamily,
0523     fontSize = 9.sp,
0524     lineHeight = 12.sp,
0525     fontWeight = FontWeight.ExtraBold,
0526     letterSpacing = 0.7.sp
0527 )
0528
0529 private val DashboardSmallBoldStyle = TextStyle(
0530     fontFamily = InterFontFamily,
0531     fontSize = 11.sp,
0532     lineHeight = 14.sp,
0533     fontWeight = FontWeight.Bold
0534 )
0535
0536 private val DashboardLevelStyle = TextStyle(
0537     fontFamily = InterFontFamily,
0538     fontSize = 20.sp,
0539     lineHeight = 24.sp,
0540     fontWeight = FontWeight.ExtraBold
0541 )
0542
0543 private val DashboardStatValueStyle = TextStyle(
0544     fontFamily = InterFontFamily,
0545     fontSize = 15.sp,
0546     lineHeight = 19.sp,

```

```
0547     fontWeight = FontWeight.ExtraBold
0548 )
0549
0550 private val DashboardMicroBoldStyle = TextStyle(
0551     fontFamily = InterFontFamily,
0552     fontSize = 9.sp,
0553     lineHeight = 12.sp,
0554     fontWeight = FontWeight.Bold
0555 )
0556
0557 private val DashboardActionStyle = TextStyle(
0558     fontFamily = InterFontFamily,
0559     fontSize = 15.sp,
0560     lineHeight = 19.sp,
0561     fontWeight = FontWeight.Bold
0562 )
0563
0564 private val DashboardSectionTitleStyle = TextStyle(
0565     fontFamily = InterFontFamily,
0566     fontSize = 16.sp,
0567     lineHeight = 21.sp,
0568     fontWeight = FontWeight.ExtraBold
0569 )
0570
0571 private val DashboardKnowledgeTitleStyle = TextStyle(
0572     fontFamily = InterFontFamily,
0573     fontSize = 14.sp,
0574     lineHeight = 17.sp,
0575     fontWeight = FontWeight.ExtraBold
0576 )
0577
0578 private val DashboardPillStyle = TextStyle(
0579     fontFamily = InterFontFamily,
0580     fontSize = 9.sp,
0581     lineHeight = 11.sp,
0582     fontWeight = FontWeight.Bold
0583 )
```

KnowledgeBaseScreen.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/screens/knowledge/KnowledgeBaseScreen.kt

Purpose: Knowledge-base UI that lists notes and opens note creation/detail flows.

```
0001 package com.example.mobile.ui.screens.knowledge
0002
0003 import androidx.compose.foundation.lazy.items
0004 import androidx.compose.runtime.Composable
0005 import androidx.compose.material3.Text
0006 import androidx.lifecycle.viewmodel.compose.viewModel
0007 import com.example.mobile.data.KnowledgeItem
0008 import com.example.mobile.navigation.Routes
0009 import com.example.mobile.ui.components.MainScreenScaffold
0010 import com.example.mobile.ui.components.PrimaryScreenButton
0011 import com.example.mobile.ui.components.KnowledgeListCard
0012 import com.example.mobile.ui.viewmodel.KnowledgeBaseViewModel
0013
0014 @Composable
0015 fun KnowledgeBaseScreen(
0016     onKnowledgeClick: (KnowledgeItem) -> Unit,
0017     onNewNoteClick: () -> Unit,
0018     onRouteClick: (String) -> Unit,
0019     viewModel: KnowledgeBaseViewModel = viewModel()
0020 ) {
0021     val uiState = viewModel.uiState
0022
0023     MainScreenScaffold(
0024         currentRoute = Routes.KNOWLEDGE_BASE,
0025         title = "Knowledge Base",
0026         subtitle = "Capture, search, and organize important ideas.",
0027         onRouteClick = onRouteClick
0028     ) {
0029         if (uiState.isLoading) {
0030             item {
0031                 Text("Loading knowledge...")
0032             }
0033         }
0034
0035         uiState.error?.let { message ->
0036             item {
0037                 Text("Unable to load knowledge: $message")
0038             }
0039         }
0040
0041         if (!uiState.isLoading && uiState.error == null && uiState.items.isEmpty()) {
0042             item {
0043                 Text("No knowledge saved yet.")
0044             }
0045         }
0046
0047         items(uiState.items) { item ->
0048             KnowledgeListCard(
0049                 item = item,
0050                 onClick = { onKnowledgeClick(item) }
0051             )
0052         }
0053
0054         item {
0055             PrimaryScreenButton(
0056                 text = "+ New Note",
0057                 onClick = onNewNoteClick
0058             )
0059         }
0060     }
0061 }
```

KnowledgeDetailScreen.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/screens/knowledge/KnowledgeDetailScreen.kt

Purpose: Knowledge detail UI for note summary, full note, review start, edit, and delete actions.

```
0001 package com.example.mobile.ui.screens.knowledge
0002
0003 import androidx.compose.foundation.layout.Arrangement
0004 import androidx.compose.foundation.layout.Row
0005 import androidx.compose.foundation.layout.fillMaxWidth
0006 import androidx.compose.foundation.layout.padding
0007 import androidx.compose.material3.OutlinedButton
0008 import androidx.compose.material3.Text
0009 import androidx.compose.runtime.LaunchedEffect
0010 import androidx.compose.runtime.Composable
0011 import androidx.compose.ui.Modifier
0012 import androidx.compose.ui.graphics.Color
0013 import androidx.compose.ui.unit.dp
0014 import androidx.lifecycle.viewmodel.compose.viewModel
0015 import com.example.mobile.navigation.Routes
0016 import com.example.mobile.ui.components.MainScreenScaffold
0017 import com.example.mobile.ui.components.PrimaryScreenButton
0018 import com.example.mobile.ui.components.ProgressSummaryCard
0019 import com.example.mobile.ui.theme.NSyncBlue
0020 import com.example.mobile.ui.theme.ScreenBodyStyle
0021 import com.example.mobile.ui.theme.ScreenButtonStyle
0022 import com.example.mobile.ui.theme.ScreenHeroStyle
0023 import com.example.mobile.ui.theme.ScreenSectionStyle
0024 import com.example.mobile.ui.theme.ScreenTitle
0025 import com.example.mobile.ui.theme.NSyncMutedText
0026 import com.example.mobile.ui.viewmodel.KnowledgeDetailViewModel
0027
0028 @Composable
0029 fun KnowledgeDetailScreen(
0030     noteId: Int,
0031     onBackClick: () -> Unit,
0032     onEditClick: () -> Unit,
0033     onAddReviewCardClick: () -> Unit,
0034     onDelete: () -> Unit,
0035     onStartReviewClick: () -> Unit,
0036     onRouteClick: (String) -> Unit,
0037     viewModel: KnowledgeDetailViewModel = viewModel()
0038 ) {
0039     val uiState = viewModel.uiState
0040     val item = uiState.item
0041
0042     LaunchedEffect(noteId) {
0043         viewModel.loadNote(noteId)
0044     }
0045
0046     LaunchedEffect(uiState.deleted) {
0047         if (uiState.deleted) {
0048             onDelete()
0049         }
0050     }
0051
0052     MainScreenScaffold(
0053         currentRoute = Routes.KNOWLEDGE_BASE,
0054         title = "Knowledge Detail",
0055         subtitle = item?.collection ?: "Loading note...",
0056         onRouteClick = onRouteClick
0057     ) {
0058         if (uiState.isLoading) {
0059             item {
0060                 Text("Loading note...", color = NSyncMutedText, style = ScreenBodyStyle)
0061             }
0062         }
0063
0064         uiState.error?.let { message ->
0065             item {
0066                 Text(message, color = Color(0xFFD21F2B), style = ScreenBodyStyle)
0067             }
0068         }
0069
0070         if (item == null && !uiState.isLoading) {
0071             item {
0072                 PrimaryScreenButton(
0073                     text = "Back to Knowledge Base",
0074                     onClick = onBackClick
0075                 )
0076             }
0077             return@MainScreenScaffold
0078         }
0079
0080         item ?: return@MainScreenScaffold
0081
0082         item {
0083             Text(
0084                 text = item.title,
0085                 color = ScreenTitle,
0086                 style = ScreenHeroStyle
0087             )
0088         }
0089
0090         item {
0091             ProgressSummaryCard(
0092                 title = "${item.masteryPercent}% Mastered",
0093                 subtitle = "${item.reviewCardCount} review cards • ${item.xpEarned} XP earned",
0094                 progress = item.masteryPercent / 100f
0095             )
0096         }
0097
0098         item {
```

```

0099         PrimaryScreenButton(
0100             text = "Start Review",
0101             onClick = onStartReviewClick
0102         )
0103     }
0104
0105     item {
0106         OutlinedButton(
0107             onClick = onAddReviewCardClick,
0108             modifier = Modifier.fillMaxWidth()
0109         ) {
0110             Text("Add Review Card", color = NSyncBlue, style = ScreenButtonStyle)
0111         }
0112     }
0113
0114     item {
0115         Row(horizontalArrangement = Arrangement.spacedBy(12.dp)) {
0116             OutlinedButton(
0117                 onClick = onEditClick,
0118                 modifier = Modifier.weight(1f)
0119             ) {
0120                 Text("Edit", color = NSyncBlue, style = ScreenButtonStyle)
0121             }
0122             OutlinedButton(
0123                 onClick = { viewModel.deleteNote(noteId) },
0124                 modifier = Modifier.weight(1f)
0125             ) {
0126                 Text(
0127                     if (uiState.isDeleting) "Deleting..." else "Delete",
0128                     color = Color(0xFFD21F2B),
0129                     style = ScreenButtonStyle
0130                 )
0131             }
0132         }
0133     }
0134
0135     item {
0136         Text("Full Note", color = ScreenTitle, style = ScreenSectionStyle)
0137         Text(
0138             text = item.fullNote,
0139             modifier = Modifier.padding(top = 10.dp),
0140             color = NSyncMutedText,
0141             style = ScreenBodyStyle
0142         )
0143     }
0144 }
0145 }

```

NewNoteScreen.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/screens/knowledge/NewNoteScreen.kt

Purpose: Create/edit note UI with form inputs and save/cancel actions.

```
0001 package com.example.mobile.ui.screens.knowledge
0002
0003 import androidx.compose.foundation.background
0004 import androidx.compose.foundation.clickable
0005 import androidx.compose.foundation.layout.Arrangement
0006 import androidx.compose.foundation.layout.Box
0007 import androidx.compose.foundation.layout.Column
0008 import androidx.compose.foundation.layout.Row
0009 import androidx.compose.foundation.layout.Spacer
0010 import androidx.compose.foundation.layout.fillMaxSize
0011 import androidx.compose.foundation.layout.fillMaxWidth
0012 import androidx.compose.foundation.layout.height
0013 import androidx.compose.foundation.layout.padding
0014 import androidx.compose.foundation.lazy.LazyColumn
0015 import androidx.compose.foundation.shape.RoundedCornerShape
0016 import androidx.compose.material3.Button
0017 import androidx.compose.material3.ButtonDefaults
0018 import androidx.compose.material3.Card
0019 import androidx.compose.material3.CardDefaults
0020 import androidx.compose.material3.OutlinedButton
0021 import androidx.compose.material3.OutlinedTextField
0022 import androidx.compose.material3.Scaffold
0023 import androidx.compose.material3.Text
0024 import androidx.compose.runtime.Composable
0025 import androidx.compose.runtime.LaunchedEffect
0026 import androidx.compose.runtime.getValue
0027 import androidx.compose.runtime.mutableStateOf
0028 import androidx.compose.runtime.remember
0029 import androidx.compose.runtime.setValue
0030 import androidx.compose.ui.Alignment
0031 import androidx.compose.ui.Modifier
0032 import androidx.compose.ui.graphics.Color
0033 import androidx.compose.ui.text.TextStyle
0034 import androidx.compose.ui.text.font.FontWeight
0035 import androidx.compose.ui.unit.dp
0036 import androidx.compose.ui.unit.sp
0037 import com.example.mobile.navigation.Routes
0038 import com.example.mobile.ui.components.BottomNavBar
0039 import com.example.mobile.ui.theme.InterFontFamily
0040 import com.example.mobile.ui.theme.NSyncBlue
0041 import com.example.mobile.ui.theme.NSyncCardWhite
0042 import com.example.mobile.ui.theme.NSyncLightBackground
0043 import com.example.mobile.ui.theme.NSyncMutedText
0044 import com.example.mobile.ui.theme.ScreenButtonStyle
0045 import com.example.mobile.ui.theme.ScreenCardBorder
0046 import com.example.mobile.ui.theme.ScreenSectionStyle
0047 import com.example.mobile.ui.theme.ScreenSmallBoldStyle
0048 import com.example.mobile.ui.theme.nsyncTextFieldColors
0049
0050 import com.example.mobile.ui.viewmodel.NewNoteViewModel
0051 import androidx.lifecycle.viewmodel.compose.viewModel
0052
0053 @Composable
0054 fun NewNoteScreen(
0055     noteId: Int? = null,
0056     onBackClick: () -> Unit,
0057     onSaveClick: () -> Unit,
0058     onRouteClick: (String) -> Unit,
0059     viewModel: NewNoteViewModel = viewModel()
0060 ) {
0061     val isEditing = noteId != null
0062     var title by remember { mutableStateOf("") }
0063     var collection by remember { mutableStateOf("") }
0064     var source by remember { mutableStateOf("Money management") }
0065     var context by remember { mutableStateOf("Training") }
0066     var content by remember {
0067         mutableStateOf("")
0068     }
0069
0070     LaunchedEffect(noteId) {
0071         noteId?.let { viewModel.loadNote(it) }
0072     }
0073
0074     LaunchedEffect(viewModel.loaded) {
0075         if (viewModel.loaded) {
0076             title = viewModel.title
0077             collection = viewModel.tag
0078             content = viewModel.content
0079         }
0080     }
0081
0082     LaunchedEffect(viewModel.saved) {
0083         if (viewModel.saved) {
0084             onSaveClick()
0085         }
0086     }
0087
0088     Scaffold(
0089         containerColor = NSyncLightBackground,
0090         bottomBar = {
0091             BottomNavBar(
0092                 currentRoute = Routes.KNOWLEDGE_BASE,
0093                 onRouteClick = onRouteClick
0094             )
0095         }
0096     ) { innerPadding ->
0097         LazyColumn(
0098             modifier = Modifier
```

```

01000      .fillMaxSize()
01001      .padding(innerPadding)
01002      .padding(horizontal = 20.dp, vertical = 18.dp),
01003      verticalArrangement = Arrangement.spacedBy(16.dp)
01004  ) {
01005      item {
01006          Row(
01007              modifier = Modifier
01008                  .fillMaxWidth()
01009                  .height(42.dp),
01010              horizontalArrangement = Arrangement.SpaceBetween,
01011              verticalAlignment = Alignment.CenterVertically
01012          ) {
01013              Text(
01014                  text = "...",
01015                  modifier = Modifier.clickable(onClick = onBackPressed),
01016                  color = NSyncBlue,
01017                  style = NewNoteTopActionStyle
01018              )
01019              Text(
01020                  if (isEditing) "Edit Note" else "New Note",
01021                  color = NSyncBlue,
01022                  style = ScreenSectionStyle
01023              )
01024              Text(
01025                  text = if (viewModel.isSaving) "Saving" else "Save",
01026                  modifier = Modifier.clickable {
01027                      if (isEditing) {
01028                          viewModel.updateNote(
01029                              id = noteId,
01030                              title = title,
01031                              content = content,
01032                              tag = collection
01033                          )
01034                      } else {
01035                          viewModel.createNote(
01036                              title = title,
01037                              content = content,
01038                              tag = collection
01039                          )
01040                      }
01041                  },
01042                  color = NSyncBlue,
01043                  style = ScreenSmallBoldStyle
01044              )
01045          }
01046      }
01047      item {
01048          Card(
01049              modifier = Modifier.fillMaxWidth(),
01050              colors = CardDefaults.cardColors(containerColor = NSyncCardWhite),
01051              shape = RoundedCornerShape(10.dp),
01052              border = ScreenCardBorder
01053          ) {
01054              Column(
01055                  modifier = Modifier.padding(12.dp),
01056                  verticalArrangement = Arrangement.spacedBy(10.dp)
01057              ) {
01058                  {
01059                      NewNoteField(
01060                          value = title,
01061                          onValueChange = { title = it },
01062                          placeholder = "Note title"
01063                      )
01064                      NewNoteLabel("Collection")
01065                      NewNoteField(
01066                          value = collection,
01067                          onValueChange = { collection = it },
01068                          placeholder = "Collection"
01069                      )
01070                      Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
01071                          Column(modifier = Modifier.weight(1f)) {
01072                              NewNoteLabel("Source")
01073                              NewNoteField(
01074                                  value = source,
01075                                  onValueChange = { source = it },
01076                                  placeholder = "Source"
01077                              )
01078                          }
01079                          Column(modifier = Modifier.weight(1f)) {
01080                              NewNoteLabel("Context")
01081                              NewNoteField(
01082                                  value = context,
01083                                  onValueChange = { context = it },
01084                                  placeholder = "Context"
01085                              )
01086                          }
01087                      }
01088                  }
01089                  Card(
01090                      modifier = Modifier
01091                          .fillMaxWidth()
01092                          .height(240.dp),
01093                      colors = CardDefaults.cardColors(containerColor = NSyncCardWhite),
01094                      shape = RoundedCornerShape(8.dp),
01095                      border = ScreenCardBorder
01096                  ) {
01097                      Column(
01098                          modifier = Modifier.padding(12.dp),
01099                          verticalArrangement = Arrangement.spacedBy(12.dp)
01100                      ) {
01101                          {
01102                              Row(
01103                                  modifier = Modifier.fillMaxWidth(),
01104                                  horizontalArrangement = Arrangement.SpaceBetween,
01105                                  verticalAlignment = Alignment.CenterVertically
01106                              ) {
01107                                  Text("B I = * □", color = Color.Black, style = ScreenSmallBoldStyle)
01108                                  Box(
01109                                      modifier = Modifier
01110                                          .background(Color(0xFFF0F3FA), RoundedCornerShape(6.dp))
01111                                          .padding(horizontal = 10.dp, vertical = 6.dp)
01112                                  )
01113                              }
01114                          }
01115                      }
01116                  }
01117              }
01118          }
01119      }
01120  }

```

```

0211         Text("Last saved 2m ago", color = NSyncMutedText, style = NewNoteTinyStyle)
0212     }
0213 }
0214
0215     OutlinedTextField(
0216         value = content,
0217         onValueChange = { content = it },
0218         modifier = Modifier.fillMaxSize(),
0219         textStyle = NewNoteFieldStyle,
0220         colors = nsyncTextFieldColors(
0221             focusedBorderColor = Color.Transparent,
0222             unfocusedBorderColor = Color.Transparent,
0223             containerColor = NSyncCardWhite
0224         )
0225     )
0226 }
0227 }
0228 }
0229 }
0230 }
0231
0232     item {
0233         Row(
0234             modifier = Modifier.fillMaxWidth(),
0235             horizontalArrangement = Arrangement.spacedBy(16.dp)
0236         ) {
0237             OutlinedButton(
0238                 onClick = onBackClick,
0239                 modifier = Modifier
0240                     .weight(1f)
0241                     .height(52.dp),
0242                 shape = RoundedCornerShape(999.dp)
0243             ) {
0244                 Text("Cancel", color = NSyncBlue, style = ScreenButtonStyle)
0245             }
0246             Button(
0247                 onClick = {
0248                     if (isEditing) {
0249                         viewModel.updateNote(
0250                             id = noteId,
0251                             title = title,
0252                             content = content,
0253                             tag = collection
0254                         )
0255                     } else {
0256                         viewModel.createNote(
0257                             title = title,
0258                             content = content,
0259                             tag = collection
0260                         )
0261                     }
0262                 },
0263                 modifier = Modifier
0264                     .weight(1f)
0265                     .height(52.dp),
0266                 shape = RoundedCornerShape(999.dp),
0267                 colors = ButtonDefaults.buttonColors(containerColor = NSyncBlue)
0268             ) {
0269                 Text(
0270                     if (viewModel.isSaving) {
0271                         "Saving..."
0272                     } else if (isEditing) {
0273                         "Update Note"
0274                     } else {
0275                         "Save Note"
0276                     },
0277                     color = Color.White,
0278                     style = ScreenButtonStyle
0279                 )
0280             }
0281         }
0282     }
0283
0284     viewModel.error?.let { message ->
0285         item {
0286             Text(message, color = Color(0xFFD21F2B), style = ScreenSmallBoldStyle)
0287         }
0288     }
0289
0290     item {
0291         Spacer(modifier = Modifier.height(72.dp))
0292     }
0293 }
0294 }
0295 }
0296
0297 @Composable
0298 private fun NewNoteLabel(text: String) {
0299     Text(text, color = Color.Black, style = ScreenSmallBoldStyle)
0300 }
0301
0302 @Composable
0303 private fun NewNoteField(
0304     value: String,
0305     onValueChange: (String) -> Unit,
0306     placeholder: String
0307 ) {
0308     OutlinedTextField(
0309         value = value,
0310         onValueChange = onValueChange,
0311         modifier = Modifier.fillMaxWidth(),
0312         placeholder = {
0313             Text(placeholder, color = Color.Black, style = NewNoteFieldStyle)
0314         },
0315         singleLine = true,
0316         textStyle = NewNoteFieldStyle,
0317         shape = RoundedCornerShape(8.dp),
0318         colors = nsyncTextFieldColors(
0319             focusedBorderColor = Color.Transparent,
0320             unfocusedBorderColor = Color.Transparent,
0321             containerColor = Color(0xFFF8F9FF)
0322         )
0323     )
0324 }

```



```
0323     )
0324 }
0325
0326 private val NewNoteTopActionStyle = TextStyle(
0327     fontFamily = InterFontFamily,
0328     fontSize = 20.sp,
0329     lineHeight = 24.sp,
0330     fontWeight = FontWeight.Bold
0331 )
0332
0333 private val NewNoteTinyStyle = TextStyle(
0334     fontFamily = InterFontFamily,
0335     fontSize = 9.sp,
0336     lineHeight = 12.sp,
0337     fontWeight = FontWeight.Bold
0338 )
0339
0340 private val NewNoteFieldStyle = TextStyle(
0341     fontFamily = InterFontFamily,
0342     fontSize = 14.sp,
0343     lineHeight = 20.sp,
0344     fontWeight = FontWeight.Normal,
0345     color = Color.Black
0346 )
```

ProfileScreen.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/screens/profile/ProfileScreen.kt

Purpose: Profile UI showing current user information, stats, settings, and logout.

```
0001 package com.example.mobile.ui.screens.profile
0002
0003 import androidx.compose.foundation.background
0004 import androidx.compose.foundation.layout.Arrangement
0005 import androidx.compose.foundation.layout.Box
0006 import androidx.compose.foundation.layout.Column
0007 import androidx.compose.foundation.layout.Row
0008 import androidx.compose.foundation.layout.fillMaxSize
0009 import androidx.compose.foundation.layout.fillMaxWidth
0010 import androidx.compose.foundation.layout.height
0011 import androidx.compose.foundation.layout.padding
0012 import androidx.compose.foundation.layout.size
0013 import androidx.compose.foundation.layout.widthIn
0014 import androidx.compose.foundation.shape.RoundedCornerShape
0015 import androidx.compose.material3.Button
0016 import androidx.compose.material3.ButtonDefaults
0017 import androidx.compose.material3.Card
0018 import androidx.compose.material3.CardDefaults
0019 import androidx.compose.material3.Icon
0020 import androidx.compose.material3.OutlinedButton
0021 import androidx.compose.material3.Text
0022 import androidx.compose.runtime.Composable
0023 import androidx.compose.runtime.DisposableEffect
0024 import androidx.compose.ui.Alignment
0025 import androidx.compose.ui.Modifier
0026 import androidx.compose.ui.draw.clip
0027 import androidx.compose.ui.graphics.Color
0028 import androidx.compose.ui.res.painterResource
0029 import androidx.compose.ui.text.style.TextAlign
0030 import androidx.compose.ui.unit.dp
0031 import androidx.lifecycle.Lifecycle
0032 import androidx.lifecycle.LifecycleEventObserver
0033 import androidx.lifecycle.compose.LocalLifecycleOwner
0034 import androidx.lifecycle.viewmodel.compose.viewModel
0035 import com.example.mobile.R
0036 import com.example.mobile.navigation.Routes
0037 import com.example.mobile.ui.components.MainScreenScaffold
0038 import com.example.mobile.ui.theme.ScreenBodyStyle
0039 import com.example.mobile.ui.theme.ScreenCardBorder
0040 import com.example.mobile.ui.theme.ScreenHeroStyle
0041 import com.example.mobile.ui.theme.ScreenSectionStyle
0042 import com.example.mobile.ui.theme.NSyncBlue
0043 import com.example.mobile.ui.theme.NSyncCardWhite
0044 import com.example.mobile.ui.theme.NSyncMutedText
0045 import com.example.mobile.ui.theme.NSyncRed
0046 import com.example.mobile.ui.viewmodel.ProfileViewModel
0047 import kotlin.math.roundToInt
0048
0049 @Composable
0050 fun ProfileScreen(
0051     onRouteClick: (String) -> Unit,
0052     onSettingsClick: () -> Unit,
0053     onLogoutClick: () -> Unit,
0054     profileViewModel: ProfileViewModel = viewModel()
0055 ) {
0056     val uiState = profileViewModel.uiState
0057     val progress = uiState.progress
0058     val totalXp = progress?.totalXp ?: 0
0059     val totalReviews = progress?.totalReviews ?: 0
0060     val lifecycleOwner = LocalLifecycleOwner.current
0061
0062     DisposableEffect(lifecycleOwner, profileViewModel) {
0063         val observer = LifecycleEventObserver { _, event ->
0064             if (event == Lifecycle.Event.ON_RESUME) {
0065                 profileViewModel.loadProfile()
0066             }
0067         }
0068         lifecycleOwner.lifecycle.addObserver(observer)
0069         onDispose { lifecycleOwner.lifecycle.removeObserver(observer) }
0070     }
0071
0072     MainScreenScaffold(
0073         currentRoute = Routes.PROFILE,
0074         title = "User Profile",
0075         subtitle = uiState.learningGoal,
0076         onRouteClick = onRouteClick
0077     ) {
0078         if (uiState.isLoading) {
0079             item { Text("Loading profile...", color = NSyncMutedText, style = ScreenBodyStyle) }
0080         }
0081
0082         uiState.error?.let { message ->
0083             item {
0084                 Text(
0085                     "Unable to load profile: $message",
0086                     color = NSyncRed,
0087                     style = ScreenBodyStyle
0088                 )
0089             }
0090         }
0091
0092         item {
0093             Box(
0094                 modifier = Modifier.fillMaxWidth(),
0095                 contentAlignment = Alignment.Center
0096             ) {
0097                 Card(
0098                     modifier = Modifier
```

```

0099         .widthIn(max = 380.dp)
0100         .fillMaxWidth(),
0101         colors = CardDefaults.cardColors(containerColor = NSyncCardWhite),
0102         shape = RoundedCornerShape(18.dp),
0103         border = ScreenCardBorder
0104     ) {
0105         Column(
0106             modifier = Modifier
0107                 .fillMaxWidth()
0108                 .padding(20.dp),
0109             horizontalAlignment = Alignment.CenterHorizontally,
0110             verticalArrangement = Arrangement.spacedBy(8.dp)
0111         ) {
0112             Box(
0113                 modifier = Modifier
0114                     .size(64.dp)
0115                     .clip(RoundedCornerShape(20.dp))
0116                     .background(Color(0xFFE8F1FF)),
0117                 contentAlignment = Alignment.Center
0118             ) {
0119                 Text(uiState.displayName.take(1), color = NSyncBlue, style = ScreenHeroStyle)
0120             }
0121             Text(
0122                 uiState.displayName,
0123                 color = Color(0xFF151927),
0124                 style = ScreenSectionStyle,
0125                 textAlign = TextAlign.Center
0126             )
0127             Text(
0128                 uiState.email,
0129                 color = NSyncMutedText,
0130                 style = ScreenBodyStyle,
0131                 textAlign = TextAlign.Center
0132             )
0133             Text(
0134                 uiState.learningGoal,
0135                 color = NSyncMutedText,
0136                 style = ScreenBodyStyle,
0137                 textAlign = TextAlign.Center
0138             )
0139             Text(
0140                 "LVL ${progress?.level ?: 1}",
0141                 color = NSyncBlue,
0142                 style = ScreenSectionStyle,
0143                 textAlign = TextAlign.Center
0144             )
0145             Text(
0146                 "$totalXp XP - $totalReviews cards reviewed",
0147                 color = NSyncMutedText,
0148                 style = ScreenBodyStyle,
0149                 textAlign = TextAlign.Center
0150             )
0151         }
0152     }
0153 }
0154
0155 item {
0156     Box(
0157         modifier = Modifier.fillMaxWidth(),
0158         contentAlignment = Alignment.Center
0159     ) {
0160         Row(
0161             modifier = Modifier
0162                 .widthIn(max = 380.dp)
0163                 .fillMaxWidth(),
0164             horizontalArrangement = Arrangement.spacedBy(10.dp)
0165         ) {
0166             ProfileStatCard(
0167                 modifier = Modifier.weight(1f),
0168                 icon = R.drawable.ic_wind,
0169                 iconTint = NSyncRed,
0170                 value = (progress?.streak ?: 0).toString(),
0171                 label = "Day streak"
0172             )
0173             ProfileStatCard(
0174                 modifier = Modifier.weight(1f),
0175                 icon = R.drawable.ic_target,
0176                 iconTint = NSyncBlue,
0177                 value = "${progress?.accuracy?.roundToInt() ?: 0}%",
0178                 label = "Accuracy"
0179             )
0180         }
0181     }
0182 }
0183
0184 item {
0185     Box(
0186         modifier = Modifier.fillMaxWidth(),
0187         contentAlignment = Alignment.Center
0188     ) {
0189         OutlinedButton(
0190             onClick = onSettingsClick,
0191             modifier = Modifier
0192                 .widthIn(max = 380.dp)
0193                 .fillMaxWidth()
0194         ) {
0195             Text("Settings", color = NSyncBlue, style = ScreenSectionStyle)
0196         }
0197     }
0198 }
0199
0200 item {
0201     Box(
0202         modifier = Modifier.fillMaxWidth(),
0203         contentAlignment = Alignment.Center
0204     ) {
0205         Button(
0206             onClick = onLogoutClick,
0207             modifier = Modifier
0208                 .widthIn(max = 380.dp)
0209                 .fillMaxWidth()
0210         )
0211     }
0212 }

```

```

0211         .height(52.dp),
0212         shape = RoundedCornerShape(999.dp),
0213         colors = ButtonDefaults.buttonColors(containerColor = NSyncRed)
0214     ) {
0215         Text("Logout", color = Color.White, style = ScreenSectionStyle)
0216     }
0217 }
0218 }
0219 }
0220 }
0221
0222 @Composable
0223 private fun ProfileStatCard(
0224     modifier: Modifier,
0225     icon: Int,
0226     iconTint: Color,
0227     value: String,
0228     label: String
0229 ) {
0230     Card(
0231         modifier = modifier.height(88.dp),
0232         colors = CardDefaults.cardColors(containerColor = NSyncCardWhite),
0233         shape = RoundedCornerShape(10.dp),
0234         border = ScreenCardBorder
0235     ) {
0236         Column(
0237             modifier = Modifier
0238                 .fillMaxSize()
0239                 .padding(12.dp),
0240             horizontalAlignment = Alignment.CenterHorizontally,
0241             verticalArrangement = Arrangement.Center
0242         ) {
0243             Icon(
0244                 painter = painterResource(id = icon),
0245                 contentDescription = null,
0246                 modifier = Modifier.size(26.dp),
0247                 tint = iconTint
0248             )
0249             Text(value, color = Color(0xFF151927), style = ScreenSectionStyle)
0250             Text(label, color = NSyncMutedText, style = ScreenBodyStyle)
0251         }
0252     }
0253 }

```

MasteryScreen.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/screens/progress/MasteryScreen.kt

Purpose: Mastery/progress UI grouped by collection.

```
0001 package com.example.mobile.ui.screens.progress
0002
0003 import androidx.compose.foundation.layout.Arrangement
0004 import androidx.compose.foundation.layout.Row
0005 import androidx.compose.foundation.lazy.items
0006 import androidx.compose.material3.Text
0007 import androidx.compose.runtime.Composable
0008 import androidx.compose.runtime.DisposableEffect
0009 import androidx.compose.ui.Modifier
0010 import androidx.compose.ui.unit.dp
0011 import androidx.lifecycle.Lifecycle
0012 import androidx.lifecycle.LifecycleEventObserver
0013 import androidx.lifecycle.compose.LocalLifecycleOwner
0014 import com.example.mobile.navigation.Routes
0015 import com.example.mobile.ui.components.MainScreenScaffold
0016 import com.example.mobile.ui.components.ProgressSummaryCard
0017 import com.example.mobile.ui.components.SummaryMetric
0018 import com.example.mobile.ui.viewmodel.MasteryViewModel
0019 import androidx.lifecycle.viewmodel.compose.viewModel
0020 import com.example.mobile.ui.theme.NSyncMutedText
0021 import com.example.mobile.ui.theme.ScreenBodyStyle
0022
0023 @Composable
0024 fun MasteryScreen(
0025     onRouteClick: (String) -> Unit,
0026     masteryViewModel: MasteryViewModel = viewModel()
0027 ) {
0028     val uiState = masteryViewModel.uiState
0029     val progress = uiState.progress
0030     val totalXp = progress?.totalXp ?: 0
0031     val totalReviews = progress?.totalReviews ?: 0
0032     val lifecycleOwner = LocalLifecycleOwner.current
0033
0034     DisposableEffect(lifecycleOwner, masteryViewModel) {
0035         val observer = LifecycleEventObserver { _, event ->
0036             if (event == Lifecycle.Event.ON_RESUME) {
0037                 masteryViewModel.loadMastery()
0038             }
0039         }
0040         lifecycleOwner.lifecycle.addObserver(observer)
0041         onDispose { lifecycleOwner.lifecycle.removeObserver(observer) }
0042     }
0043
0044     MainScreenScaffold(
0045         currentRoute = Routes.MASTERY,
0046         title = "Mastery",
0047         subtitle = "Track progress by collection",
0048         onRouteClick = onRouteClick
0049     ) {
0050         item {
0051             Row(horizontalArrangement = Arrangement.spacedBy(10.dp)) {
0052                 SummaryMetric("TOTAL XP", totalXp.toString(), Modifier.weight(1f))
0053                 SummaryMetric("TOTAL REVIEWS", totalReviews.toString(), Modifier.weight(1f))
0054             }
0055         }
0056
0057         if (uiState.isLoading) {
0058             item {
0059                 Text("Loading mastery...", color = NSyncMutedText, style = ScreenBodyStyle)
0060             }
0061         }
0062
0063         uiState.error?.let { message ->
0064             item {
0065                 Text("Unable to load mastery: $message", color = NSyncMutedText, style = ScreenBodyStyle)
0066             }
0067         }
0068
0069         if (!uiState.isLoading && uiState.error == null && uiState.masteryGroups.isEmpty()) {
0070             item {
0071                 Text("No review cards saved yet.", color = NSyncMutedText, style = ScreenBodyStyle)
0072             }
0073         }
0074
0075         items(uiState.masteryGroups) { mastery ->
0076             ProgressSummaryCard(
0077                 title = mastery.collection,
0078                 subtitle = mastery.context,
0079                 progress = mastery.masteryPercent / 100f,
0080                 trailing = "${mastery.masteryPercent}%"
0081             )
0082         }
0083     }
0084 }
```

FlashcardDetailScreen.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/screens/review/FlashcardDetailScreen.kt

Purpose: Flashcard detail UI for viewing a specific review card.

```
0001 package com.example.mobile.ui.screens.review
0002
0003 import androidx.compose.foundation.layout.Arrangement
0004 import androidx.compose.foundation.layout.Row
0005 import androidx.compose.ui.unit.dp
0006 import androidx.compose.material3.OutlinedButton
0007 import androidx.compose.material3.Text
0008 import androidx.compose.runtime.Composable
0009 import androidx.compose.runtime.LaunchedEffect
0010 import androidx.compose.ui.Modifier
0011 import androidx.compose.ui.graphics.Color
0012 import androidx.lifecycle.viewmodel.compose.viewModel
0013 import com.example.mobile.navigation.Routes
0014 import com.example.mobile.ui.components.MainScreenScaffold
0015 import com.example.mobile.ui.components.PrimaryScreenButton
0016 import com.example.mobile.ui.components.ProgressSummaryCard
0017 import com.example.mobile.ui.theme.NSyncBlue
0018 import com.example.mobile.ui.theme.NSyncMutedText
0019 import com.example.mobile.ui.theme.ScreenBodyStyle
0020 import com.example.mobile.ui.theme.ScreenButtonStyle
0021 import com.example.mobile.ui.theme.ScreenHeroStyle
0022 import com.example.mobile.ui.theme.ScreenSectionStyle
0023 import com.example.mobile.ui.theme.ScreenTitle
0024 import com.example.mobile.ui.viewmodel.FlashcardDetailViewModel
0025
0026 @Composable
0027 fun FlashcardDetailScreen(
0028     cardId: Int,
0029     onBackPressed: () -> Unit,
0030     onEditClick: () -> Unit,
0031     onStartSessionClick: (Int) -> Unit,
0032     onDelete: () -> Unit,
0033     onRouteClick: (String) -> Unit,
0034     viewModel: FlashcardDetailViewModel = viewModel()
0035 ) {
0036     val uiState = viewModel.uiState
0037     val card = uiState.card
0038
0039     LaunchedEffect(cardId) { viewModel.loadCard(cardId) }
0040     LaunchedEffect(uiState.deleted) { if (uiState.deleted) onDelete() }
0041
0042     MainScreenScaffold(
0043         currentRoute = Routes.REVIEW_CARDS,
0044         title = "Review Card",
0045         subtitle = card?.collection ?: "Loading card...",
0046         onRouteClick = onRouteClick
0047     ) {
0048         if (uiState.isLoading) {
0049             item { Text("Loading review card...", color = NSyncMutedText, style = ScreenBodyStyle) }
0050         }
0051         uiState.error?.let { message ->
0052             item { Text(message, color = Color(0xFFD21F2B), style = ScreenBodyStyle) }
0053         }
0054         if (card == null && !uiState.isLoading) {
0055             item { PrimaryScreenButton("Back to Review Cards", onBackPressed) }
0056             return@MainScreenScaffold
0057         }
0058         card ?: return@MainScreenScaffold
0059
0060         item { Text(card.question, color = ScreenTitle, style = ScreenHeroStyle) }
0061         item {
0062             Text("Answer", color = ScreenTitle, style = ScreenSectionStyle)
0063             Text(card.answer, color = NSyncMutedText, style = ScreenBodyStyle)
0064         }
0065         item {
0066             ProgressSummaryCard(
0067                 title = card.masteryLabel,
0068                 subtitle = "${card.difficulty} - ${card.masteryPercent}% mastery",
0069                 progress = card.masteryPercent / 100f
0070             )
0071         }
0072         item {
0073             PrimaryScreenButton(
0074                 text = "Start Note Session",
0075                 onClick = { onStartSessionClick(card.knowledgeItemId) }
0076             )
0077         }
0078         item {
0079             Row(horizontalArrangement = Arrangement.spacedBy(12.dp)) {
0080                 OutlinedButton(onClick = onEditClick, modifier = Modifier.weight(1f)) {
0081                     Text("Edit", color = NSyncBlue, style = ScreenButtonStyle)
0082                 }
0083                 OutlinedButton(onClick = { viewModel.deleteCard(cardId) }, modifier = Modifier.weight(1f)) {
0084                     Text(
0085                         if (uiState.isDeleting) "Deleting..." else "Delete",
0086                         color = Color(0xFFD21F2B),
0087                         style = ScreenButtonStyle
0088                     )
0089                 }
0090             }
0091         }
0092     }
0093 }
```

NewFlashcardScreen.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/screens/review/NewFlashcardScreen.kt

Purpose: Create flashcard UI for a selected note.

```
0001 package com.example.mobile.ui.screens.review
0002
0003 import androidx.compose.foundation.layout.Arrangement
0004 import androidx.compose.foundation.layout.Column
0005 import androidx.compose.foundation.layout.Row
0006 import androidx.compose.foundation.layout.fillMaxSize
0007 import androidx.compose.foundation.layout.fillMaxWidth
0008 import androidx.compose.foundation.layout.height
0009 import androidx.compose.foundation.layout.padding
0010 import androidx.compose.foundation.layout.Spacer
0011 import androidx.compose.foundation.layout.width
0012 import androidx.compose.material3.Button
0013 import androidx.compose.material3.ButtonDefaults
0014 import androidx.compose.material3.OutlinedButton
0015 import androidx.compose.material3.OutlinedTextField
0016 import androidx.compose.material3.Text
0017 import androidx.compose.runtime.Composable
0018 import androidx.compose.runtime.LaunchedEffect
0019 import androidx.compose.runtime.getValue
0020 import androidx.compose.runtime.mutableStateOf
0021 import androidx.compose.runtime.remember
0022 import androidx.compose.runtime.setValue
0023 import androidx.compose.ui.Alignment
0024 import androidx.compose.ui.Modifier
0025 import androidx.compose.ui.graphics.Color
0026 import androidx.compose.ui.unit.dp
0027 import androidx.lifecycle.viewmodel.compose.viewModel
0028 import com.example.mobile.ui.theme.NSyncBlue
0029 import com.example.mobile.ui.theme.ScreenBodyStyle
0030 import com.example.mobile.ui.theme.ScreenButtonStyle
0031 import com.example.mobile.ui.theme.ScreenHeroStyle
0032 import com.example.mobile.ui.theme.nsyncTextFieldColors
0033 import com.example.mobile.ui.viewmodel.NewFlashcardViewModel
0034
0035 @Composable
0036 fun NewFlashcardScreen(
0037     noteId: Int? = null,
0038     cardId: Int? = null,
0039     onBackClick: () -> Unit,
0040     onSave: () -> Unit,
0041     viewModel: NewFlashcardViewModel = viewModel()
0042 ) {
0043     val isEditing = cardId != null
0044     var question by remember { mutableStateOf("") }
0045     var answer by remember { mutableStateOf("") }
0046     var difficulty by remember { mutableStateOf("Easy") }
0047
0048     LaunchedEffect(cardId) {
0049         cardId?.let { viewModel.loadCard(it) }
0050     }
0051
0052     LaunchedEffect(viewModel.loadedCard) {
0053         viewModel.loadedCard?.let { card ->
0054             question = card.question
0055             answer = card.answer
0056             difficulty = card.difficulty
0057         }
0058     }
0059
0060     LaunchedEffect(viewModel.saved) {
0061         if (viewModel.saved) onSave()
0062     }
0063
0064     Column(
0065         modifier = Modifier
0066             .fillMaxSize()
0067             .padding(20.dp),
0068         verticalArrangement = Arrangement.spacedBy(16.dp)
0069     ) {
0070         Text(
0071             if (isEditing) "Edit Review Card" else "New Review Card",
0072             color = NSyncBlue,
0073             style = ScreenHeroStyle
0074         )
0075         Text(
0076             if (isEditing) "Update this question and answer." else "Create a question and answer for this knowledge item.",
0077             style = ScreenBodyStyle
0078         )
0079
0080         OutlinedTextField(
0081             value = question,
0082             onValueChange = { question = it },
0083             modifier = Modifier.fillMaxWidth(),
0084             label = { Text("Question") },
0085             colors = nsyncTextFieldColors()
0086         )
0087         OutlinedTextField(
0088             value = answer,
0089             onValueChange = { answer = it },
0090             modifier = Modifier
0091                 .fillMaxWidth()
0092                 .height(180.dp),
0093             label = { Text("Answer") },
0094             colors = nsyncTextFieldColors()
0095         )
0096         OutlinedTextField(
0097             value = difficulty,
0098             onValueChange = { difficulty = it },
```

```

0099         modifier = Modifier.fillMaxWidth(),
0100         label = { Text("Difficulty") },
0101         colors = nsyncTextFieldColors()
0102     )
0103
0104     viewModel.error?.let { message ->
0105         Text(message, color = Color(0xFFD21F2B), style = ScreenBodyStyle)
0106     }
0107
0108     Row(horizontalArrangement = Arrangement.spacedBy(12.dp)) {
0109         OutlinedButton(
0110             onClick = onBackClick,
0111             modifier = Modifier.weight(1f)
0112         ) {
0113             Text("Cancel", style = ScreenButtonStyle)
0114         }
0115         Button(
0116             onClick = {
0117                 if (isEditing) {
0118                     viewModel.loadedCard?.let { card ->
0119                         viewModel.updateFlashcard(card, question, answer, difficulty)
0120                     }
0121                 } else {
0122                     viewModel.saveFlashcard(noteId ?: return@Button, question, answer, difficulty)
0123                 }
0124             },
0125             modifier = Modifier.weight(1f),
0126             colors = ButtonDefaults.buttonColors(containerColor = NSyncBlue)
0127         ) {
0128             Text(
0129                 if (viewModel.isSaving) "Saving..." else if (isEditing) "Update Card" else "Save Card",
0130                 style = ScreenButtonStyle
0131             )
0132         }
0133     }
0134 }
0135 }

```


ReviewCardsScreen.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/screens/review/ReviewCardsScreen.kt

Purpose: Review sessions UI grouped by connected note.

```
0001 package com.example.mobile.ui.screens.review
0002
0003 import androidx.compose.runtime.Composable
0004 import androidx.compose.runtime.DisposableEffect
0005 import androidx.compose.material3.Text
0006 import androidx.lifecycle.viewmodel.compose.viewModel
0007 import androidx.lifecycle.Lifecycle
0008 import androidx.lifecycle.LifecycleEventObserver
0009 import androidx.lifecycle.compose.LocalLifecycleOwner
0010 import com.example.mobile.navigation.Routes
0011 import com.example.mobile.ui.components.MainScreenScaffold
0012 import com.example.mobile.ui.components.ReviewSessionListItem
0013 import com.example.mobile.ui.theme.NSyncMutedText
0014 import com.example.mobile.ui.theme.ScreenBodyStyle
0015 import com.example.mobile.ui.viewmodel.ReviewCardsViewModel
0016
0017 @Composable
0018 fun ReviewCardsScreen(
0019     onStartSessionClick: (Int) -> Unit,
0020     onAddCardClick: (Int) -> Unit,
0021     onRouteClick: (String) -> Unit,
0022     viewModel: ReviewCardsViewModel = viewModel()
0023 ) {
0024     val lifecycleOwner = LocalLifecycleOwner.current
0025
0026     DisposableEffect(lifecycleOwner, viewModel) {
0027         val observer = LifecycleEventObserver { _, event ->
0028             if (event == Lifecycle.Event.ON_RESUME) {
0029                 viewModel.loadCards()
0030             }
0031         }
0032         lifecycleOwner.lifecycle.addObserver(observer)
0033         onDispose { lifecycleOwner.lifecycle.removeObserver(observer) }
0034     }
0035
0036     MainScreenScaffold(
0037         currentRoute = Routes.REVIEW_CARDS,
0038         title = "Review Sessions",
0039         subtitle = "Organized by knowledge note",
0040         onRouteClick = onRouteClick
0041     ) {
0042         if (viewModel.isLoading) {
0043             item {
0044                 Text("Loading review cards...", color = NSyncMutedText, style = ScreenBodyStyle)
0045             }
0046         }
0047
0048         viewModel.error?.let { message ->
0049             item {
0050                 Text("Unable to load cards: $message", color = NSyncMutedText, style = ScreenBodyStyle)
0051             }
0052         }
0053
0054         if (!viewModel.isLoading && viewModel.error == null && viewModel.cards.isEmpty()) {
0055             item {
0056                 Text("No review cards saved yet.", color = NSyncMutedText, style = ScreenBodyStyle)
0057             }
0058         }
0059
0060         val cardsBySession = viewModel.cards.groupBy { card ->
0061             Triple(card.knowledgeItemId, card.sourceNoteTitle, card.collection)
0062         }
0063
0064         cardsBySession.forEach { (session, cards) ->
0065             val (noteId, noteTitle, tag) = session
0066             item(key = "session-$noteId") {
0067                 ReviewSessionListItem(
0068                     title = noteTitle,
0069                     tag = tag,
0070                     cardCount = cards.size,
0071                     onStartSessionClick = { onStartSessionClick(noteId) },
0072                     onAddCardClick = { onAddCardClick(noteId) }
0073                 )
0074             }
0075         }
0076     }
0077 }
```

ReviewSessionScreen.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/screens/review/ReviewSessionScreen.kt

Purpose: Quiz-style review UI for question, answer reveal, and recall rating.

```
0001 package com.example.mobile.ui.screens.review
0002
0003 import androidx.compose.foundation.background
0004 import androidx.compose.foundation.layout.Arrangement
0005 import androidx.compose.foundation.layout.Column
0006 import androidx.compose.foundation.layout.Row
0007 import androidx.compose.foundation.layout.fillMaxSize
0008 import androidx.compose.foundation.layout.fillMaxWidth
0009 import androidx.compose.foundation.layout.height
0010 import androidx.compose.foundation.layout.padding
0011 import androidx.compose.foundation.lazy.LazyColumn
0012 import androidx.compose.material3.Button
0013 import androidx.compose.material3.ButtonDefaults
0014 import androidx.compose.material3.OutlinedButton
0015 import androidx.compose.material3.Text
0016 import androidx.compose.runtime.Composable
0017 import androidx.compose.runtime.LaunchedEffect
0018 import androidx.compose.runtime.getValue
0019 import androidx.compose.runtime.mutableStateOf
0020 import androidx.compose.runtime.remember
0021 import androidx.compose.runtime.setValue
0022 import androidx.compose.ui.Alignment
0023 import androidx.compose.ui.Modifier
0024 import androidx.compose.ui.text.style.TextAlign
0025 import androidx.compose.ui.unit.dp
0026 import androidx.lifecycle.viewmodel.compose.viewModel
0027 import com.example.mobile.ui.components.CenteredCard
0028 import com.example.mobile.ui.components.PrimaryScreenButton
0029 import com.example.mobile.ui.theme.ScreenBodyStyle
0030 import com.example.mobile.ui.theme.ScreenHeroStyle
0031 import com.example.mobile.ui.theme.ScreenSectionStyle
0032 import com.example.mobile.ui.theme.ScreenTitle
0033 import com.example.mobile.ui.theme.NSyncBlue
0034 import com.example.mobile.ui.theme.NSyncLightBackground
0035 import com.example.mobile.ui.theme.NSyncMutedText
0036 import com.example.mobile.ui.viewmodel.ReviewSessionViewModel
0037
0038 @Composable
0039 fun ReviewSessionScreen(
0040     onCompleteClick: (score: Int, totalQuestions: Int, xpEarned: Int) -> Unit,
0041     noteId: Int? = null,
0042     cardId: Int? = null,
0043     viewModel: ReviewSessionViewModel = viewModel()
0044 ) {
0045     LaunchedEffect(noteId, cardId) { viewModel.loadSession(noteId, cardId) }
0046     LaunchedEffect(viewModel.completedResult) {
0047         viewModel.completedResult?.let { result ->
0048             viewModel.consumeCompletedResult()
0049             onCompleteClick(result.score, result.totalQuestions, result.xpEarned)
0050         }
0051     }
0052     var showAnswer by remember { mutableStateOf(false) }
0053     LaunchedEffect(viewModel.currentIndex) {
0054         showAnswer = false
0055     }
0056
0057     LazyColumn(
0058         modifier = Modifier
0059             .fillMaxSize()
0060             .background(NSyncLightBackground)
0061             .padding(horizontal = 28.dp, vertical = 52.dp),
0062         horizontalAlignment = Alignment.CenterHorizontally,
0063         verticalArrangement = Arrangement.spacedBy(24.dp)
0064     ) {
0065         item {
0066             Text("${viewModel.cards.size * 25} XP", color = NSyncBlue, style = ScreenSectionStyle)
0067             Text(
0068                 text = when {
0069                     viewModel.isLoading -> "Loading review cards"
0070                     viewModel.currentCard == null -> "No review cards available"
0071                     showAnswer -> "Review Answer"
0072                     else -> "Card ${viewModel.currentIndex + 1} of ${viewModel.cards.size}"
0073                 },
0074                 color = NSyncMutedText,
0075                 style = ScreenBodyStyle
0076             )
0077         }
0078
0079         viewModel.error?.let { message ->
0080             item {
0081                 Text(message, color = androidx.compose.ui.graphics.Color(0xFFD21F2B), style = ScreenBodyStyle)
0082             }
0083         }
0084
0085         viewModel.currentCard?.let { card ->
0086             item {
0087                 CenteredCard(
0088                     modifier = Modifier
0089                         .fillParentMaxWidth()
0090                         .height(280.dp)
0091                 ) {
0092                     Column(
0093                         horizontalAlignment = Alignment.CenterHorizontally,
0094                         verticalArrangement = Arrangement.spacedBy(12.dp)
0095                     ) {
0096                         Text(
0097                             text = if (showAnswer) card.answer else card.question,
0098
```

```

0099         color = ScreenTitle,
0100         style = ScreenHeroStyle,
0101         textAlign = TextAlign.Center
0102     )
0103 }
0104 }
0105 }
0106
0107 if (!showAnswer) {
0108     item {
0109         PrimaryScreenButton(
0110             text = "Show Answer",
0111             onClick = { showAnswer = true }
0112         )
0113     }
0114 } else {
0115     item {
0116         Row(
0117             modifier = Modifier.fillMaxWidth(),
0118             horizontalArrangement = Arrangement.spacedBy(12.dp)
0119         ) {
0120             OutlinedButton(
0121                 onClick = { viewModel.recordAnswer(recalled = false) },
0122                 modifier = Modifier.weight(1f),
0123                 enabled = !viewModel.isCompleting
0124             ) { Text("Review again", style = ScreenBodyStyle)
0125             }
0126             Button(
0127                 onClick = { viewModel.recordAnswer(recalled = true) },
0128                 modifier = Modifier.weight(1f),
0129                 colors = ButtonDefaults.buttonColors(containerColor = NSyncBlue),
0130                 enabled = !viewModel.isCompleting
0131             ) { Text(
0132                 if (viewModel.isCompleting) "Saving..." else "Got it",
0133                 style = ScreenBodyStyle
0134             )
0135             }
0136         }
0137     }
0138 }
0139 }
0140 }
0141 } ?: item {
0142     if (!viewModel.isLoading) {
0143         Text("Create a review card from a knowledge note to begin.", color = NSyncMutedText, style = ScreenBodyStyle)
0144     }
0145 }
0146 }
0147 }

```

SessionCompleteScreen.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/screens/review/SessionCompleteScreen.kt

Purpose: Review result UI showing score, XP, streak, and level progress.

```
0001 package com.example.mobile.ui.screens.review
0002
0003 import androidx.compose.foundation.BorderStroke
0004 import androidx.compose.foundation.Image
0005 import androidx.compose.foundation.background
0006 import androidx.compose.foundation.clickable
0007 import androidx.compose.foundation.layout.Arrangement
0008 import androidx.compose.foundation.layout.Box
0009 import androidx.compose.foundation.layout.Column
0010 import androidx.compose.foundation.layout.Row
0011 import androidx.compose.foundation.layout.fillMaxSize
0012 import androidx.compose.foundation.layout.fillMaxWidth
0013 import androidx.compose.foundation.layout.height
0014 import androidx.compose.foundation.layout.padding
0015 import androidx.compose.foundation.layout.size
0016 import androidx.compose.foundation.rememberScrollState
0017 import androidx.compose.foundation.shape.CircleShape
0018 import androidx.compose.foundation.shape.RoundedCornerShape
0019 import androidx.compose.foundation.verticalScroll
0020 import androidx.compose.material3.Button
0021 import androidx.compose.material3.ButtonDefaults
0022 import androidx.compose.material3.Card
0023 import androidx.compose.material3.CardDefaults
0024 import androidx.compose.material3.LinearProgressIndicator
0025 import androidx.compose.material3.OutlinedButton
0026 import androidx.compose.material3.Scaffold
0027 import androidx.compose.material3.Text
0028 import androidx.compose.runtime.Composable
0029 import androidx.compose.runtime.LaunchedEffect
0030 import androidx.compose.ui.Alignment
0031 import androidx.compose.ui.Modifier
0032 import androidx.compose.ui.draw.clip
0033 import androidx.compose.ui.graphics.Color
0034 import androidx.compose.ui.res.painterResource
0035 import androidx.compose.ui.text.TextStyle
0036 import androidx.compose.ui.text.font.FontWeight
0037 import androidx.compose.ui.text.style.TextAlign
0038 import androidx.compose.ui.unit.dp
0039 import androidx.compose.ui.unit.sp
0040 import androidx.lifecycle.ViewModel.compose.viewModel
0041 import com.example.mobile.R
0042 import com.example.mobile.navigation.Routes
0043 import com.example.mobile.ui.components.BottomNavBar
0044 import com.example.mobile.ui.theme.InterFontFamily
0045 import com.example.mobile.ui.theme.NSyncBlue
0046 import com.example.mobile.ui.theme.NSyncCardWhite
0047 import com.example.mobile.ui.theme.NSyncLightBackground
0048 import com.example.mobile.ui.theme.NSyncMutedText
0049 import com.example.mobile.ui.theme.ScreenTitle
0050 import com.example.mobile.ui.viewmodel.SessionCompleteViewModel
0051 import kotlin.math.roundToInt
0052
0053 @Composable
0054 fun SessionCompleteScreen(
0055     score: Int,
0056     totalQuestions: Int,
0057     xpEarned: Int,
0058     onDashboardClick: () -> Unit,
0059     onReviewAgainClick: () -> Unit,
0060     onRouteClick: (String) -> Unit,
0061     viewModel: SessionCompleteViewModel = viewModel()
0062 ) {
0063     LaunchedEffect(Unit) {
0064         viewModel.loadProgress()
0065     }
0066
0067     val progress = viewModel.uiState.progress
0068     val displayedScore = score
0069     val displayedTotal = totalQuestions
0070     val accuracyPercent = progress?.accuracy?.roundToInt()
0071     ?: if (totalQuestions == 0) 0 else (score * 100 / totalQuestions)
0072
0073     Scaffold(
0074         containerColor = NSyncLightBackground,
0075         bottomBar = {
0076             BottomNavBar(
0077                 currentRoute = Routes.DASHBOARD,
0078                 onRouteClick = onRouteClick
0079             )
0080         }
0081     ) { innerPadding ->
0082         Column(
0083             modifier = Modifier
0084                 .fillMaxSize()
0085                 .padding(innerPadding)
0086                 .verticalScroll(rememberScrollState())
0087                 .padding(horizontal = 24.dp, vertical = 18.dp),
0088             horizontalAlignment = Alignment.CenterHorizontally,
0089             verticalArrangement = Arrangement.spacedBy(14.dp)
0090         ) {
0091             SessionCompleteTopBar(onBackClick = onDashboardClick)
0092
0093             Box(
0094                 modifier = Modifier
0095                     .size(54.dp)
0096                     .clip(CircleShape)
0097                     .background(Color(0xFFE4EEFF)),
0098                 contentAlignment = Alignment.Center
```

```

0099     ) {
0100         Box(
0101             modifier = Modifier
0102                 .size(34.dp)
0103                 .clip(CircleShape)
0104                 .background(NSyncBlue),
0105             contentAlignment = Alignment.Center
0106         ) {
0107             Text("✓", color = Color.White, style = SessionCheckStyle)
0108         }
0109     }
0110
0111     Text(
0112         text = "Session Complete",
0113         color = ScreenTitle,
0114         style = SessionTitleStyle,
0115         textAlign = TextAlign.Center
0116     )
0117     Text(
0118         text = "Nice work - your memory got stronger.",
0119         color = NSyncMutedText,
0120         style = SessionBodyStyle,
0121         textAlign = TextAlign.Center
0122     )
0123
0124     if (viewModel.uiState.isLoading) {
0125         Text("Loading updated progress...", color = NSyncMutedText, style = SessionBodyStyle)
0126     }
0127
0128     viewModel.uiState.error?.let { message ->
0129         Text(message, color = Color(0xFFD21F2B), style = SessionBodyStyle)
0130         OutlinedButton(
0131             onClick = { viewModel.loadProgress() }
0132         ) {
0133             Text("Retry", style = SessionOutlineButtonStyle)
0134         }
0135     }
0136
0137     Row(
0138         modifier = Modifier.fillMaxWidth(),
0139         horizontalArrangement = Arrangement.spacedBy(12.dp)
0140     ) {
0141         SessionMetricCard("Score", "$displayedScore/$displayedTotal", Modifier.weight(1f))
0142         SessionMetricCard("Accuracy", "$accuracyPercent%", Modifier.weight(1f))
0143     }
0144
0145     XpEarnedCard(xpEarned = xpEarned)
0146
0147     StreakCard(streakDays = progress?.streak ?: 0)
0148
0149     LevelProgressSection(
0150         level = progress?.level ?: 1,
0151         totalXp = progress?.totalXp ?: 0
0152     )
0153
0154     Button(
0155         onClick = onReviewAgainClick,
0156         modifier = Modifier
0157             .fillMaxWidth()
0158             .height(54.dp),
0159         shape = RoundedCornerShape(18.dp),
0160         colors = ButtonDefaults.buttonColors(containerColor = NSyncBlue)
0161     ) {
0162         Text("🔄 Review Again", style = SessionButtonStyle)
0163     }
0164
0165     OutlinedButton(
0166         onClick = onDashboardClick,
0167         modifier = Modifier
0168             .fillMaxWidth()
0169             .height(54.dp),
0170         shape = RoundedCornerShape(18.dp),
0171         border = BorderStroke(1.dp, Color(0xFF9AA3B4)),
0172         colors = ButtonDefaults.outlinedButtonColors(contentColor = NSyncBlue)
0173     ) {
0174         Text("🏠 Back to Dashboard", style = SessionOutlineButtonStyle)
0175     }
0176 }
0177
0178 }
0179
0180 @Composable
0181 private fun SessionCompleteTopBar(onBackClick: () -> Unit) {
0182     Row(
0183         modifier = Modifier.fillMaxWidth(),
0184         horizontalArrangement = Arrangement.SpaceBetween,
0185         verticalAlignment = Alignment.CenterVertically
0186     ) {
0187         Row(
0188             horizontalArrangement = Arrangement.spacedBy(6.dp),
0189             verticalAlignment = Alignment.CenterVertically
0190         ) {
0191             Text(
0192                 text = "<",
0193                 color = NSyncBlue,
0194                 style = SessionBackStyle,
0195                 modifier = Modifier.clickable(onClick = onBackClick)
0196             )
0197             Text("NSync", color = NSyncBlue, style = SessionLogoStyle)
0198         }
0199         Image(
0200             painter = painterResource(id = R.drawable.roosc),
0201             contentDescription = "User profile",
0202             modifier = Modifier
0203                 .size(34.dp)
0204                 .clip(CircleShape)
0205         )
0206     }
0207 }
0208
0209 @Composable
0210 private fun SessionMetricCard(

```

```

0211     label: String,
0212     value: String,
0213     modifier: Modifier = Modifier
0214 ) {
0215     Card(
0216         modifier = modifier.height(84.dp),
0217         colors = CardDefaults.cardColors(containerColor = NSyncCardWhite),
0218         shape = RoundedCornerShape(8.dp),
0219         border = BorderStroke(1.dp, Color(0xFFE8ECF4))
0220     ) {
0221         Column(
0222             modifier = Modifier.fillMaxSize(),
0223             horizontalAlignment = Alignment.CenterHorizontally,
0224             verticalArrangement = Arrangement.Center
0225         ) {
0226             Text(label.uppercase(), color = NSyncMutedText, style = SessionTinyStyle)
0227             Text(value, color = NSyncBlue, style = SessionMetricValueStyle)
0228         }
0229     }
0230 }
0231
0232 @Composable
0233 private fun XpEarnedCard(xpEarned: Int) {
0234     Card(
0235         modifier = Modifier
0236             .fillMaxWidth()
0237             .height(74.dp),
0238         colors = CardDefaults.cardColors(containerColor = NSyncCardWhite),
0239         shape = RoundedCornerShape(8.dp),
0240         border = BorderStroke(1.dp, Color(0xFFE8ECF4))
0241     ) {
0242         Row(
0243             modifier = Modifier
0244                 .fillMaxSize()
0245                 .padding(horizontal = 16.dp),
0246             horizontalArrangement = Arrangement.SpaceBetween,
0247             verticalAlignment = Alignment.CenterVertically
0248         ) {
0249             Row(
0250                 horizontalArrangement = Arrangement.spacedBy(10.dp),
0251                 verticalAlignment = Alignment.CenterVertically
0252             ) {
0253                 Box(
0254                     modifier = Modifier
0255                         .size(28.dp)
0256                         .clip(CircleShape)
0257                         .background(Color(0xFF7F8DA3)),
0258                     contentAlignment = Alignment.Center
0259                 ) {
0260                     Text("•", color = Color.White, style = SessionTinyStyle)
0261                 }
0262                 Column {
0263                     Text("XP earned", color = NSyncMutedText, style = SessionTinyStyle)
0264                     Text("+${xpEarned} XP", color = ScreenTitle, style = SessionSmallBoldStyle)
0265                 }
0266             }
0267             Box(
0268                 modifier = Modifier
0269                     .clip(RoundedCornerShape(999.dp))
0270                     .background(NSyncBlue)
0271                     .padding(horizontal = 9.dp, vertical = 4.dp)
0272             ) {
0273                 Text("BOOSTED", color = Color.White, style = SessionBadgeStyle)
0274             }
0275         }
0276     }
0277 }
0278
0279 @Composable
0280 private fun StreakCard(streakDays: Int) {
0281     Card(
0282         modifier = Modifier
0283             .fillMaxWidth()
0284             .height(96.dp),
0285         colors = CardDefaults.cardColors(containerColor = Color(0xFFEDEF66)),
0286         shape = RoundedCornerShape(8.dp)
0287     ) {
0288         Column(
0289             modifier = Modifier
0290                 .fillMaxSize()
0291                 .padding(16.dp),
0292             verticalArrangement = Arrangement.spacedBy(8.dp)
0293         ) {
0294             Row(
0295                 horizontalArrangement = Arrangement.spacedBy(10.dp),
0296                 verticalAlignment = Alignment.CenterVertically
0297             ) {
0298                 Text("•", color = Color(0xFFFF6B00), style = SessionMetricValueStyle)
0299                 Column {
0300                     Text("${streakDays} day streak", color = ScreenTitle, style = SessionSmallBoldStyle)
0301                     Text("Keep reviewing to maintain momentum.", color = NSyncMutedText, style = SessionTinyStyle)
0302                 }
0303             }
0304             Row(horizontalArrangement = Arrangement.spacedBy(7.dp)) {
0305                 repeat(8) { index ->
0306                     Box(
0307                         modifier = Modifier
0308                             .weight(1f)
0309                             .height(6.dp)
0310                             .clip(RoundedCornerShape(999.dp))
0311                             .background(if (index < 5) NSyncBlue else Color(0xFFBFD7F3))
0312                     )
0313                 }
0314             }
0315         }
0316     }
0317 }
0318
0319 @Composable
0320 private fun LevelProgressSection(level: Int, totalXp: Int) {
0321     val levelStartXp = when (level) {
0322

```

```

0323         5 -> 1000
0324         4 -> 500
0325         3 -> 250
0326         2 -> 100
0327         else -> 0
0328     }
0329     val nextLevelXp = when (level) {
0330     1 -> 100
0331     2 -> 250
0332     3 -> 500
0333     4 -> 1000
0334     else -> null
0335     }
0336     val progressFraction = nextLevelXp?.let { nextXp ->
0337     ((totalXp - levelStartXp).toFloat() / (nextXp - levelStartXp))
0338     .coerceIn(0f, 1f)
0339     } ?: 1f
0340
0341     Column(
0342     modifier = Modifier.fillMaxWidth(),
0343     verticalArrangement = Arrangement.spacedBy(6.dp)
0344     ) {
0345         Row(
0346         modifier = Modifier.fillMaxWidth(),
0347         horizontalArrangement = Arrangement.SpaceBetween
0348         ) {
0349             Text("Level $level", color = NSyncMutedText, style = SessionTinyStyle)
0350             Text(
0351             text = nextLevelXp?.let { "${(it - totalXp).coerceAtLeast(0)} XP to Level ${level + 1}" }
0352             ?: "Maximum level reached",
0353             color = NSyncBlue,
0354             style = SessionTinyBoldBlueStyle
0355             )
0356         }
0357         LinearProgressIndicator(
0358         progress = { progressFraction },
0359         modifier = Modifier
0360         .fillMaxWidth()
0361         .height(9.dp)
0362         .clip(RoundedCornerShape(999.dp)),
0363         color = NSyncBlue,
0364         trackColor = Color(0xFFDCE2EC)
0365         )
0366         Text(
0367         text = "Level progress updated",
0368         modifier = Modifier.fillMaxWidth(),
0369         color = NSyncMutedText,
0370         style = SessionTinyItalicStyle,
0371         textAlign = TextAlign.Center
0372         )
0373     }
0374 }
0375
0376 private val SessionLogoStyle = TextStyle(
0377     fontFamily = InterFontFamily,
0378     fontSize = 20.sp,
0379     fontWeight = FontWeight.Bold
0380 )
0381
0382 private val SessionBackStyle = TextStyle(
0383     fontFamily = InterFontFamily,
0384     fontSize = 28.sp,
0385     fontWeight = FontWeight.Bold
0386 )
0387
0388 private val SessionCheckStyle = TextStyle(
0389     fontFamily = InterFontFamily,
0390     fontSize = 20.sp,
0391     fontWeight = FontWeight.Black
0392 )
0393
0394 private val SessionTitleStyle = TextStyle(
0395     fontFamily = InterFontFamily,
0396     fontSize = 28.sp,
0397     fontWeight = FontWeight.Bold
0398 )
0399
0400 private val SessionBodyStyle = TextStyle(
0401     fontFamily = InterFontFamily,
0402     fontSize = 13.sp,
0403     fontWeight = FontWeight.Medium
0404 )
0405
0406 private val SessionTinyStyle = TextStyle(
0407     fontFamily = InterFontFamily,
0408     fontSize = 10.sp,
0409     fontWeight = FontWeight.Bold
0410 )
0411
0412 private val SessionTinyBoldBlueStyle = TextStyle(
0413     fontFamily = InterFontFamily,
0414     fontSize = 9.sp,
0415     fontWeight = FontWeight.Bold
0416 )
0417
0418 private val SessionTinyItalicStyle = TextStyle(
0419     fontFamily = InterFontFamily,
0420     fontSize = 9.sp,
0421     fontWeight = FontWeight.Bold
0422 )
0423
0424 private val SessionMetricValueStyle = TextStyle(
0425     fontFamily = InterFontFamily,
0426     fontSize = 23.sp,
0427     fontWeight = FontWeight.Bold
0428 )
0429
0430 private val SessionSmallBoldStyle = TextStyle(
0431     fontFamily = InterFontFamily,
0432     fontSize = 13.sp,
0433     fontWeight = FontWeight.Bold
0434 )

```

```
0435
0436 private val SessionBadgeStyle = TextStyle(
0437     fontFamily = InterFontFamily,
0438     fontSize = 8.sp,
0439     fontWeight = FontWeight.Black
0440 )
0441
0442 private val SessionButtonStyle = TextStyle(
0443     fontFamily = InterFontFamily,
0444     fontSize = 14.sp,
0445     fontWeight = FontWeight.Bold
0446 )
0447
0448 private val SessionOutlineButtonStyle = TextStyle(
0449     fontFamily = InterFontFamily,
0450     fontSize = 13.sp,
0451     fontWeight = FontWeight.Bold
0452 )
```


SettingsPreferences.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/screens/settings/SettingsPreferences.kt

Purpose: Local settings persistence for review preferences and notification options.

```
0001 package com.example.mobile.ui.screens.settings
0002
0003 import android.content.Context
0004 import android.content.SharedPreferences
0005
0006 object SettingsPreferences {
0007     private const val FILE_NAME = "nsync_settings"
0008
0009     const val DAILY_GOAL = "daily_goal"
0010     const val REMINDER_TIME = "reminder_time"
0011     const val REVIEW_DIFFICULTY = "review_difficulty"
0012     const val STREAK_REMINDERS = "streak_reminders"
0013     const val NOTIFICATIONS_ENABLED = "notifications_enabled"
0014
0015     fun from(context: Context): SharedPreferences =
0016         context.getSharedPreferences(FILE_NAME, Context.MODE_PRIVATE)
0017 }
```

SettingsScreen.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/screens/settings/SettingsScreen.kt

Purpose: Settings UI for account information and app/review preferences.

```
0001 package com.example.mobile.ui.screens.settings
0002
0003 import android.Manifest
0004 import android.content.pm.PackageManager
0005 import android.os.Build
0006 import androidx.compose.foundation.background
0007 import androidx.compose.foundation.clickable
0008 import androidx.compose.foundation.layout.Arrangement
0009 import androidx.compose.foundation.layout.Column
0010 import androidx.compose.foundation.layout.Row
0011 import androidx.compose.foundation.layout.fillMaxSize
0012 import androidx.compose.foundation.layout.fillMaxWidth
0013 import androidx.compose.foundation.layout.padding
0014 import androidx.compose.foundation.rememberScrollState
0015 import androidx.compose.foundation.verticalScroll
0016 import androidx.compose.material3.Icon
0017 import androidx.compose.material3.Text
0018 import androidx.compose.runtime.Composable
0019 import androidx.compose.runtime.LaunchedEffect
0020 import androidx.compose.runtime.getValue
0021 import androidx.compose.runtime.mutableIntStateOf
0022 import androidx.compose.runtime.mutableStateOf
0023 import androidx.compose.runtime.remember
0024 import androidx.compose.runtime.saveable.rememberSaveable
0025 import androidx.compose.runtime.setValue
0026 import androidx.compose.ui.Alignment
0027 import androidx.compose.ui.Modifier
0028 import androidx.compose.ui.platform.LocalContext
0029 import androidx.compose.ui.res.painterResource
0030 import androidx.compose.ui.unit.dp
0031 import androidx.activity.compose.rememberLauncherForActivityResult
0032 import androidx.activity.result.contract.ActivityResultContracts
0033 import androidx.core.content.ContextCompat
0034 import com.example.mobile.R
0035 import com.example.mobile.data.remote.RetrofitClient
0036 import com.example.mobile.notifications.ReviewReminderScheduler
0037 import com.example.mobile.ui.components.SettingsDivider
0038 import com.example.mobile.ui.components.SettingsDropDownRow
0039 import com.example.mobile.ui.components.SettingsRow
0040 import com.example.mobile.ui.components.SettingsSection
0041 import com.example.mobile.ui.components.SettingsToggleRow
0042 import com.example.mobile.ui.theme.NSyncBlue
0043 import com.example.mobile.ui.theme.NSyncLightBackground
0044 import com.example.mobile.ui.theme.ScreenSectionStyle
0045 import kotlinx.coroutines.flow.first
0046
0047 @Composable
0048 fun SettingsScreen(
0049     onClick: () -> Unit,
0050     onProfileClick: () -> Unit
0051 ) {
0052     val context = LocalContext.current
0053     val preferences = remember(context) { SettingsPreferences.from(context) }
0054     val goals = listOf(10, 20, 30, 50, 100)
0055     val reminderTimes = listOf("8:00 AM", "12:00 PM", "6:00 PM")
0056     val difficulties = listOf("Adaptive", "Easy", "Medium", "Hard")
0057     var accountEmail by rememberSaveable { mutableStateOf("") }
0058
0059     var dailyGoal by rememberSaveable {
0060         mutableIntStateOf(preferences.getInt(SettingsPreferences.DAILY_GOAL, 50))
0061     }
0062     var reminderTime by rememberSaveable {
0063         mutableStateOf(preferences.getString(SettingsPreferences.REMINDER_TIME, "8:00 AM") ?: "8:00 AM")
0064     }
0065     var difficulty by rememberSaveable {
0066         mutableStateOf(preferences.getString(SettingsPreferences.REVIEW_DIFFICULTY, "Adaptive") ?: "Adaptive")
0067     }
0068     var streakReminders by rememberSaveable {
0069         mutableStateOf(preferences.getBoolean(SettingsPreferences.STREAK_REMINDERS, true))
0070     }
0071     var notificationsEnabled by rememberSaveable {
0072         mutableStateOf(preferences.getBoolean(SettingsPreferences.NOTIFICATIONS_ENABLED, true))
0073     }
0074     val notificationPermissionLauncher = rememberLauncherForActivityResult(
0075         ActivityResultContracts.RequestPermission()
0076     ) { granted ->
0077         notificationsEnabled = granted
0078         preferences.edit().putBoolean(SettingsPreferences.NOTIFICATIONS_ENABLED, granted).apply()
0079         if (granted) ReviewReminderScheduler.schedule(context) else ReviewReminderScheduler.cancel(context)
0080     }
0081
0082     LaunchedEffect(Unit) {
0083         accountEmail = RetrofitClient.sessionStore.sessionFlow.first()?.email.orEmpty()
0084     }
0085
0086     LaunchedEffect(notificationsEnabled) {
0087         val hasNotificationPermission =
0088             Build.VERSION.SDK_INT < Build.VERSION_CODES.TIRAMISU ||
0089             ContextCompat.checkSelfPermission(
0090                 context,
0091                 Manifest.permission.POST_NOTIFICATIONS
0092             ) == PackageManager.PERMISSION_GRANTED
0093         if (notificationsEnabled && hasNotificationPermission) {
0094             ReviewReminderScheduler.schedule(context)
0095         }
0096     }
0097
0098     Column(
```

```

0099         modifier = Modifier
0100             .fillMaxSize()
0101             .background(NSyncLightBackground)
0102             .verticalScroll(rememberScrollState())
0103             .padding(horizontal = 28.dp, vertical = 18.dp),
0104         verticalArrangement = Arrangement.spacedBy(26.dp)
0105     ) {
0106         SettingsTopBar(onBackClick)
0107
0108         SettingsSection(title = "ACCOUNT") {
0109             SettingsRow(
0110                 label = "Profile",
0111                 iconRes = R.drawable.ic_profile,
0112                 onClick = onProfileClick,
0113                 showChevron = true
0114             )
0115             SettingsDivider()
0116             SettingsRow(
0117                 label = "Email Address",
0118                 value = accountEmail.ifBlank { "No email found" },
0119                 iconRes = R.drawable.ic_mail_outline
0120             )
0121             SettingsDivider()
0122             SettingsRow(label = "Change Password", value = "Unavailable", iconRes = R.drawable.ic_lock_outline)
0123         }
0124
0125         SettingsSection(title = "REVIEW PREFERENCES") {
0126             SettingsDropDownRow(
0127                 label = "Daily Review Goal",
0128                 value = "$dailyGoal Cards",
0129                 iconRes = R.drawable.ic_target,
0130                 valueColor = NSyncBlue,
0131                 options = goals.map { "$it Cards" },
0132                 onOptionSelected = { selected ->
0133                     dailyGoal = selected.substringBefore(" ").toInt()
0134                     preferences.edit().putInt(SettingsPreferences.DAILY_GOAL, dailyGoal).apply()
0135                     if (notificationsEnabled) ReviewReminderScheduler.schedule(context)
0136                 }
0137             )
0138             SettingsDivider()
0139             SettingsDropDownRow(
0140                 label = "Reminder Time",
0141                 value = reminderTime,
0142                 iconRes = R.drawable.ic_note,
0143                 options = reminderTimes,
0144                 onOptionSelected = { selected ->
0145                     reminderTime = selected
0146                     preferences.edit().putString(SettingsPreferences.REMINDER_TIME, selected).apply()
0147                     if (notificationsEnabled) ReviewReminderScheduler.schedule(context)
0148                 }
0149             )
0150             SettingsDivider()
0151             SettingsDropDownRow(
0152                 label = "Review Difficulty",
0153                 value = difficulty,
0154                 iconRes = R.drawable.ic_flashcard,
0155                 options = difficulties,
0156                 onOptionSelected = { selected ->
0157                     difficulty = selected
0158                     preferences.edit().putString(SettingsPreferences.REVIEW_DIFFICULTY, selected).apply()
0159                 }
0160             )
0161             SettingsDivider()
0162             SettingsToggleRow(
0163                 label = "Streak Reminders",
0164                 iconRes = R.drawable.ic_wind,
0165                 checked = streakReminders,
0166                 onCheckedChange = {
0167                     streakReminders = it
0168                     preferences.edit().putBoolean(SettingsPreferences.STREAK_REMINDERS, it).apply()
0169                     if (notificationsEnabled) ReviewReminderScheduler.schedule(context)
0170                 }
0171             )
0172         }
0173
0174         SettingsSection(title = "APP PREFERENCES") {
0175             SettingsToggleRow(
0176                 label = "Notifications",
0177                 iconRes = R.drawable.ic_note,
0178                 checked = notificationsEnabled,
0179                 onCheckedChange = {
0180                     if (!it) {
0181                         notificationsEnabled = false
0182                         preferences.edit().putBoolean(SettingsPreferences.NOTIFICATIONS_ENABLED, false).apply()
0183                         ReviewReminderScheduler.cancel(context)
0184                     } else if (
0185                         Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU &&
0186                         ContextCompat.checkSelfPermission(
0187                             context,
0188                             Manifest.permission.POST_NOTIFICATIONS
0189                         ) != PackageManager.PERMISSION_GRANTED
0190                     ) {
0191                         notificationPermissionLauncher.launch(Manifest.permission.POST_NOTIFICATIONS)
0192                     } else {
0193                         notificationsEnabled = true
0194                         preferences.edit().putBoolean(SettingsPreferences.NOTIFICATIONS_ENABLED, true).apply()
0195                         ReviewReminderScheduler.schedule(context)
0196                     }
0197                 }
0198             )
0199             SettingsDivider()
0200             SettingsRow(label = "Appearance", value = "Light", iconRes = R.drawable.ic_profile)
0201         }
0202     }
0203 }
0204
0205 @Composable
0206 private fun SettingsTopBar(onBackClick: () -> Unit) {
0207     Row(
0208         modifier = Modifier.fillMaxWidth()
0209         .padding(top = 20.dp),
0210         verticalAlignment = Alignment.CenterVertically,

```

```
0211         horizontalArrangement = Arrangement.SpaceBetween
0212     ) {
0213         Icon(
0214             painter = painterResource(R.drawable.ic_arrow_back),
0215             contentDescription = "Back",
0216             modifier = Modifier.clickable(onClick = onBackClick)
0217         )
0218         Text("NSync", color = NSyncBlue, style = ScreenSectionStyle)
0219         Text(" ", style = ScreenSectionStyle)
0220     }
0221 }
```

AuthUiState.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/state/AuthUiState.kt

Purpose: Defines authentication UI state for loading, error, and session status.

```
0001 package com.example.mobile.ui.state
0002
0003 import com.example.mobile.data.local.AuthSession
0004
0005 data class AuthUiState(
0006     val isLoading: Boolean = false,
0007     val session: AuthSession? = null,
0008     val errorMessage: String? = null,
0009     val hasRestoredSession: Boolean = false,
0010 )
```

DashboardUiState.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/state/DashboardUiState.kt

Purpose: Defines dashboard loading, error, user, progress, and recent note state.

```
0001 package com.example.mobile.ui.state
0002
0003 import com.example.mobile.data.KnowledgeItem
0004 import com.example.mobile.data.remote.dto.UserProgressDto
0005
0006 data class DashboardUiState(
0007     val isLoading: Boolean = false,
0008     val error: String? = null,
0009     val displayName: String = "there",
0010     val email: String = "",
0011     val progress: UserProgressDto? = null,
0012     val recentKnowledge: List<KnowledgeItem> = emptyList()
0013 )
```

FlashcardDetailUiState.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/state/FlashcardDetailUiState.kt

Purpose: Defines flashcard detail loading, error, and selected card state.

```
0001 package com.example.mobile.ui.state
0002
0003 import com.example.mobile.data.ReviewCard
0004
0005 data class FlashcardDetailUiState(
0006     val card: ReviewCard? = null,
0007     val isLoading: Boolean = false,
0008     val isDeleting: Boolean = false,
0009     val error: String? = null,
0010     val deleted: Boolean = false
0011 )
```

KnowledgeBaseUiState.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/state/KnowledgeBaseUiState.kt

Purpose: Defines knowledge-base loading, error, and note list state.

```
0001 package com.example.mobile.ui.state
0002
0003 import com.example.mobile.data.KnowledgeItem
0004
0005 data class KnowledgeBaseUiState(
0006     val items: List<KnowledgeItem> = emptyList(),
0007     val isLoading: Boolean = false,
0008     val error: String? = null
0009 )
```


KnowledgeDetailUiState.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/state/KnowledgeDetailUiState.kt

Purpose: Defines knowledge-detail loading, error, note, and deletion state.

```
0001 package com.example.mobile.ui.state
0002
0003 import com.example.mobile.data.KnowledgeItem
0004
0005 data class KnowledgeDetailUiState(
0006     val item: KnowledgeItem? = null,
0007     val isLoading: Boolean = false,
0008     val isDeleting: Boolean = false,
0009     val error: String? = null,
0010     val deleted: Boolean = false
0011 )
```

MasteryUiState.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/state/MasteryUiState.kt

Purpose: Defines mastery loading, error, total XP, total reviews, and collection progress state.

```
0001 package com.example.mobile.ui.state
0002
0003 import com.example.mobile.data.CollectionMastery
0004 import com.example.mobile.data.remote.dto.UserProgressDto
0005
0006 data class MasteryUiState(
0007     val isLoading: Boolean = false,
0008     val error: String? = null,
0009     val progress: UserProgressDto? = null,
0010     val masteryGroups: List<CollectionMastery> = emptyList()
0011 )
```

ProfileUiState.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/state/ProfileUiState.kt

Purpose: Defines profile loading, error, identity, and progress state.

```
0001 package com.example.mobile.ui.state
0002
0003 import com.example.mobile.data.remote.dto.UserProgressDto
0004
0005 data class ProfileUiState(
0006     val isLoading: Boolean = false,
0007     val error: String? = null,
0008     val displayName: String = "User",
0009     val email: String = "",
0010     val learningGoal: String = "Ready to strengthen what matters today?",
0011     val progress: UserProgressDto? = null
0012 )
```

SessionCompleteUiState.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/state/SessionCompleteUiState.kt

Purpose: Defines session-complete result, XP, accuracy, streak, and progress state.

```
0001 package com.example.mobile.ui.state
0002
0003 import com.example.mobile.data.remote.dto.UserProgressDto
0004
0005 data class SessionCompleteUiState(
0006     val isLoading: Boolean = false,
0007     val progress: UserProgressDto? = null,
0008     val error: String? = null
0009 )
```

Color.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/theme/Color.kt

Purpose: Defines app colors and shared text-field color helpers.

```
0001 package com.example.mobile.ui.theme
0002
0003 import androidx.compose.material3.OutlinedTextFieldDefaults
0004 import androidx.compose.material3.TextFieldColors
0005 import androidx.compose.runtime.Composable
0006 import androidx.compose.ui.graphics.Color
0007
0008 val Purple80 = Color(0xFFD08CFF)
0009 val PurpleGrey80 = Color(0xFFCCC2DC)
0010 val Pink80 = Color(0xFFE8B8C8)
0011
0012 val Purple40 = Color(0xFF6650a4)
0013 val PurpleGrey40 = Color(0xFF625b71)
0014 val Pink40 = Color(0xFF7D5260)
0015
0016
0017 // Color Palette for NSync
0018 val NSyncBlue = Color(0xFF0058C8)
0019 val NSyncLightBackground = Color(0xFFFFF9FF)
0020 val NSyncCardWhite = Color(0xFFFFFFFF)
0021 val NSyncMutedText = Color(0xFF5F6675)
0022 val NSyncRed = Color(0xFFD21F2B)
0023 val NSyncBorder = Color(0xFFE1E5EF)
0024 val NSyncInputText = Color(0xFF151927)
0025 val NSyncPlaceholderText = Color(0xFF8D94A3)
0026
0027 @Composable
0028 fun nsyncTextFieldColors(
0029     focusedBorderColor: Color = NSyncBlue,
0030     unfocusedBorderColor: Color = NSyncBorder,
0031     containerColor: Color = NSyncCardWhite,
0032 ): TextFieldColors = OutlinedTextFieldDefaults.colors(
0033     focusedTextColor = NSyncInputText,
0034     unfocusedTextColor = NSyncInputText,
0035     disabledTextColor = NSyncInputText.copy(alpha = 0.6f),
0036     errorTextColor = NSyncInputText,
0037     cursorColor = NSyncBlue,
0038     errorCursorColor = NSyncRed,
0039     focusedBorderColor = focusedBorderColor,
0040     unfocusedBorderColor = unfocusedBorderColor,
0041     disabledBorderColor = unfocusedBorderColor,
0042     errorBorderColor = NSyncRed,
0043     focusedContainerColor = containerColor,
0044     unfocusedContainerColor = containerColor,
0045     disabledContainerColor = containerColor,
0046     errorContainerColor = containerColor,
0047     focusedPlaceholderColor = NSyncPlaceholderText,
0048     unfocusedPlaceholderColor = NSyncPlaceholderText,
0049     disabledPlaceholderColor = NSyncPlaceholderText.copy(alpha = 0.6f),
0050     focusedLabelColor = NSyncMutedText,
0051     unfocusedLabelColor = NSyncMutedText,
0052     disabledLabelColor = NSyncMutedText.copy(alpha = 0.6f),
0053     focusedLeadingIconColor = NSyncMutedText,
0054     unfocusedLeadingIconColor = NSyncMutedText,
0055 )
```

ScreenStyles.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/theme/ScreenStyles.kt

Purpose: Defines shared screen spacing, sizing, and visual constants.

```
0001 package com.example.mobile.ui.theme
0002
0003 import androidx.compose.foundation.BorderStroke
0004 import androidx.compose.ui.graphics.Color
0005 import androidx.compose.ui.text.TextStyle
0006 import androidx.compose.ui.text.font.FontWeight
0007 import androidx.compose.ui.unit.dp
0008 import androidx.compose.ui.unit.sp
0009
0010 val ScreenTitle = Color(0xFF151927)
0011 val ScreenCardBorder = BorderStroke(1.dp, Color(0xFFE9EDF5))
0012
0013 val ScreenLogoStyle = TextStyle(
0014     fontFamily = InterFontFamily,
0015     fontSize = 21.sp,
0016     lineHeight = 26.sp,
0017     fontWeight = FontWeight.ExtraBold
0018 )
0019
0020 val ScreenHeroStyle = TextStyle(
0021     fontFamily = InterFontFamily,
0022     fontSize = 28.sp,
0023     lineHeight = 34.sp,
0024     fontWeight = FontWeight.ExtraBold
0025 )
0026
0027 val ScreenSectionStyle = TextStyle(
0028     fontFamily = InterFontFamily,
0029     fontSize = 16.sp,
0030     lineHeight = 21.sp,
0031     fontWeight = FontWeight.ExtraBold
0032 )
0033
0034 val ScreenBodyStyle = TextStyle(
0035     fontFamily = InterFontFamily,
0036     fontSize = 14.sp,
0037     lineHeight = 20.sp,
0038     fontWeight = FontWeight.Bold
0039 )
0040
0041 val ScreenSmallStyle = TextStyle(
0042     fontFamily = InterFontFamily,
0043     fontSize = 12.sp,
0044     lineHeight = 16.sp,
0045     fontWeight = FontWeight.Normal
0046 )
0047
0048 val ScreenSmallBoldStyle = TextStyle(
0049     fontFamily = InterFontFamily,
0050     fontSize = 11.sp,
0051     lineHeight = 15.sp,
0052     fontWeight = FontWeight.Bold
0053 )
0054
0055 val ScreenButtonStyle = TextStyle(
0056     fontFamily = InterFontFamily,
0057     fontSize = 15.sp,
0058     lineHeight = 19.sp,
0059     fontWeight = FontWeight.Bold
0060 )
```

Theme.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/theme/Theme.kt

Purpose: Applies the Material 3 app theme.

```
0001 package com.example.mobile.ui.theme
0002
0003 import android.os.Build
0004 import androidx.compose.foundation.isSystemInDarkTheme
0005 import androidx.compose.material3.MaterialTheme
0006 import androidx.compose.material3.darkColorScheme
0007 import androidx.compose.material3.dynamicDarkColorScheme
0008 import androidx.compose.material3.dynamicLightColorScheme
0009 import androidx.compose.material3.lightColorScheme
0010 import androidx.compose.runtime.Composable
0011 import androidx.compose.ui.graphics.Color
0012 import androidx.compose.ui.platform.LocalContext
0013
0014 private val DarkColorScheme = darkColorScheme(
0015     primary = NSyncBlue,
0016     secondary = NSyncMutedText,
0017     tertiary = NSyncRed,
0018     background = NSyncLightBackground,
0019     surface = NSyncCardWhite,
0020     onPrimary = Color.White,
0021     onSecondary = Color.White,
0022     onTertiary = Color.White
0023 )
0024
0025 private val LightColorScheme = lightColorScheme(
0026     primary = NSyncBlue,
0027     secondary = NSyncMutedText,
0028     tertiary = NSyncRed,
0029     background = NSyncLightBackground,
0030     surface = NSyncCardWhite,
0031     onPrimary = Color.White,
0032     onSecondary = Color.White,
0033     onTertiary = Color.White
0034 )
0035
0036 @Composable
0037 fun MobileTheme(
0038     darkTheme: Boolean = isSystemInDarkTheme(),
0039     dynamicColor: Boolean = false,
0040     content: @Composable () -> Unit
0041 ) {
0042     val colorScheme = when {
0043         dynamicColor && Build.VERSION.SDK_INT >= Build.VERSION_CODES.S -> {
0044             val context = LocalContext.current
0045             if (darkTheme) dynamicDarkColorScheme(context) else dynamicLightColorScheme(context)
0046         }
0047         else -> {
0048             darkTheme -> DarkColorScheme
0049             else -> LightColorScheme
0050         }
0051     }
0052     MaterialTheme(
0053         colorScheme = colorScheme,
0054         typography = Typography,
0055         content = content
0056     )
0057 }
```

Type.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/theme/Type.kt

Purpose: Defines Inter typography and app text styles.

```
0001 package com.example.mobile.ui.theme
0002
0003 import androidx.compose.material3.Typography
0004 import androidx.compose.ui.text.TextStyle
0005 import androidx.compose.ui.text.font.Font
0006 import androidx.compose.ui.text.font.FontFamily
0007 import androidx.compose.ui.text.font.FontWeight
0008 import androidx.compose.ui.unit.sp
0009 import com.example.mobile.R
0010
0011 val InterFontFamily = FontFamily(
0012     Font(R.font.inter, FontWeight.Normal),
0013     Font(R.font.inter, FontWeight.Medium),
0014     Font(R.font.inter, FontWeight.SemiBold),
0015     Font(R.font.inter, FontWeight.Bold),
0016     Font(R.font.inter, FontWeight.ExtraBold)
0017 )
0018
0019 val Typography = Typography(
0020     bodyLarge = TextStyle(
0021         fontFamily = InterFontFamily,
0022         fontWeight = FontWeight.Normal,
0023         fontSize = 16.sp,
0024         lineHeight = 24.sp,
0025         letterSpacing = 0.5.sp
0026     )
0027 )
```


AuthViewModel.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/viewmodel/AuthViewModel.kt

Purpose: Coordinates login, register, logout, and auth session state for auth screens.

```
0001 package com.example.mobile.ui.viewmodel
0002
0003 import androidx.compose.runtime.getValue
0004 import androidx.compose.runtime.mutableStateOf
0005 import androidx.compose.runtime.setValue
0006 import androidx.lifecycle.ViewModel
0007 import androidx.lifecycle.ViewModelScope
0008 import com.example.mobile.data.repository.AuthRepository
0009 import com.example.mobile.ui.state.AuthUiState
0010 import kotlinx.coroutines.launch
0011
0012 class AuthViewModel(
0013     private val repository: AuthRepository,
0014 ) : ViewModel() {
0015     var uiState by mutableStateOf(AuthUiState())
0016     private set
0017
0018     fun login(email: String, password: String) {
0019         viewModelScope.launch {
0020             uiState = uiState.copy(isLoading = true, errorMessage = null)
0021             repository.login(email, password)
0022                 .onSuccess { session ->
0023                     uiState = uiState.copy(isLoading = false, session = session)
0024                 }
0025                 .onFailure { error ->
0026                     uiState = uiState.copy(isLoading = false, errorMessage = error.message)
0027                 }
0028         }
0029     }
0030
0031     fun register(displayName: String, email: String, password: String) {
0032         viewModelScope.launch {
0033             uiState = uiState.copy(isLoading = true, errorMessage = null)
0034             repository.register(displayName, email, password)
0035                 .onSuccess { session ->
0036                     uiState = uiState.copy(isLoading = false, session = session)
0037                 }
0038                 .onFailure { error ->
0039                     uiState = uiState.copy(isLoading = false, errorMessage = error.message)
0040                 }
0041         }
0042     }
0043
0044     fun logout() {
0045         viewModelScope.launch {
0046             uiState = uiState.copy(isLoading = true, errorMessage = null)
0047             repository.logout()
0048                 .onFailure { error ->
0049                     uiState = uiState.copy(errorMessage = error.message)
0050                 }
0051             uiState = uiState.copy(isLoading = false, session = null)
0052         }
0053     }
0054
0055     fun restoreSession() {
0056         viewModelScope.launch {
0057             uiState = uiState.copy(isLoading = true, errorMessage = null)
0058             repository.verifySession()
0059                 .onSuccess { session ->
0060                     uiState = uiState.copy(
0061                         isLoading = false,
0062                         session = session,
0063                         hasRestoredSession = true,
0064                     )
0065                 }
0066                 .onFailure { error ->
0067                     uiState = uiState.copy(
0068                         isLoading = false,
0069                         errorMessage = error.message,
0070                         hasRestoredSession = true,
0071                     )
0072                 }
0073         }
0074     }
0075
0076     fun clearError() {
0077         uiState = uiState.copy(errorMessage = null)
0078     }
0079 }
```

AuthViewModelFactory.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/viewmodel/AuthViewModelFactory.kt

Purpose: Factory for constructing AuthViewModel with repository dependencies.

```
0001 package com.example.mobile.ui.viewmodel
0002
0003 import androidx.lifecycle.ViewModel
0004 import androidx.lifecycle.ViewModelProvider
0005 import com.example.mobile.data.repository.AuthRepository
0006
0007 class AuthViewModelFactory(
0008     private val repository: AuthRepository,
0009 ) : ViewModelProvider.Factory {
0010     @Suppress("UNCHECKED_CAST")
0011     override fun <T : ViewModel> create(modelClass: Class<T>): T {
0012         if (modelClass.isAssignableFrom(AuthViewModel::class.java)) {
0013             return AuthViewModel(repository) as T
0014         }
0015         throw IllegalArgumentException("Unknown ViewModel class: ${modelClass.name}")
0016     }
0017 }
0018 }
```

DashboardViewModel.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/viewmodel/DashboardViewModel.kt

Purpose: Loads dashboard progress and recent notes from repositories.

```
0001 package com.example.mobile.ui.viewmodel
0002
0003 import androidx.compose.runtime.getValue
0004 import androidx.compose.runtime.mutableStateOf
0005 import androidx.compose.runtime.setValue
0006 import androidx.lifecycle.ViewModel
0007 import androidx.lifecycle.ViewModelScope
0008 import com.example.mobile.data.local.AuthSession
0009 import com.example.mobile.data.remote.RetrofitClient
0010 import com.example.mobile.data.repository.NSyncRepository
0011 import com.example.mobile.ui.state.DashboardUiState
0012 import kotlinx.coroutines.flow.first
0013 import kotlinx.coroutines.launch
0014
0015
0016 class DashboardViewModel : ViewModel() {
0017     private val repository = NSyncRepository()
0018
0019     var uiState by mutableStateOf(DashboardUiState())
0020     private set
0021
0022     init {
0023         loadDashboard()
0024     }
0025
0026     fun loadDashboard() {
0027         viewModelScope.launch {
0028             val session = RetrofitClient.sessionStore.sessionFlow.first()
0029             uiState = DashboardUiState(
0030                 isLoading = true,
0031                 displayName = session.dashboardName(),
0032                 email = session?.email.orEmpty()
0033             )
0034             try {
0035                 val progress = repository.getProgress()
0036                 val items = repository.getKnowledgeItems()
0037                 uiState = DashboardUiState(
0038                     displayName = session.dashboardName(),
0039                     email = session?.email.orEmpty(),
0040                     progress = progress,
0041                     recentKnowledge = items
0042                 )
0043             } catch (e: Exception) {
0044                 uiState = DashboardUiState(
0045                     displayName = session.dashboardName(),
0046                     email = session?.email.orEmpty(),
0047                     error = e.message ?: "Unable to load dashboard."
0048                 )
0049             }
0050         }
0051     }
0052
0053     private fun AuthSession?.dashboardName(): String {
0054         return this?.displayName
0055             ?.trim()
0056             ?.takeIf { it.isNotEmpty() }
0057             ?: this?.email
0058                 ?.substringBefore("@")
0059                 ?.takeIf { it.isNotEmpty() }
0060             ?: "there"
0061     }
0062 }
```

FlashcardDetailViewModel.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/viewmodel/FlashcardDetailViewModel.kt

Purpose: Loads data for a selected flashcard.

```
0001 package com.example.mobile.ui.viewmodel
0002
0003 import androidx.compose.runtime.getValue
0004 import androidx.compose.runtime.mutableStateOf
0005 import androidx.compose.runtime.setValue
0006 import androidx.lifecycle.ViewModel
0007 import androidx.lifecycle.ViewModelScope
0008 import com.example.mobile.data.repository.NSyncRepository
0009 import com.example.mobile.ui.state.FlashcardDetailUiState
0010 import kotlinx.coroutines.launch
0011
0012 class FlashcardDetailViewModel : ViewModel() {
0013     private val repository = NSyncRepository()
0014
0015     var uiState by mutableStateOf(FlashcardDetailUiState())
0016     private set
0017
0018     fun loadCard(id: Int) {
0019         viewModelScope.launch {
0020             uiState = uiState.copy(isLoading = true, error = null, deleted = false)
0021             try {
0022                 val card = repository.getReviewCardById(id)
0023                 uiState = if (card != null) {
0024                     uiState.copy(card = card, isLoading = false)
0025                 } else {
0026                     uiState.copy(error = "Review card not found.", isLoading = false)
0027                 }
0028             } catch (e: Exception) {
0029                 uiState = uiState.copy(error = e.message, isLoading = false)
0030             }
0031         }
0032     }
0033
0034     fun deleteCard(id: Int) {
0035         viewModelScope.launch {
0036             uiState = uiState.copy(isDeleting = true, error = null)
0037             try {
0038                 val deleted = repository.deleteFlashcard(id)
0039                 uiState = if (deleted) {
0040                     uiState.copy(isDeleting = false, deleted = true)
0041                 } else {
0042                     uiState.copy(isDeleting = false, error = "Unable to delete review card.")
0043                 }
0044             } catch (e: Exception) {
0045                 uiState = uiState.copy(isDeleting = false, error = e.message)
0046             }
0047         }
0048     }
0049 }
```

KnowledgeBaseViewModel.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/viewmodel/KnowledgeBaseViewModel.kt

Purpose: Loads and refreshes the note list.

```
0001 package com.example.mobile.ui.viewmodel
0002
0003 import androidx.compose.runtime.getValue
0004 import androidx.compose.runtime.mutableStateOf
0005 import androidx.compose.runtime.setValue
0006 import androidx.lifecycle.ViewModel
0007 import androidx.lifecycle.ViewModelScope
0008 import com.example.mobile.data.repository.NSyncRepository
0009 import com.example.mobile.ui.state.KnowledgeBaseUiState
0010 import kotlinx.coroutines.launch
0011
0012 class KnowledgeBaseViewModel : ViewModel() {
0013     private val repository = NSyncRepository()
0014
0015     var uiState by mutableStateOf(KnowledgeBaseUiState())
0016     private set
0017
0018     init {
0019         loadNotes()
0020     }
0021
0022     fun loadNotes() {
0023         viewModelScope.launch {
0024             uiState = uiState.copy(isLoading = true, error = null)
0025             try {
0026                 val items = repository.getKnowledgeItems()
0027                 uiState = uiState.copy(items = items, isLoading = false)
0028             } catch (e: Exception) {
0029                 uiState = uiState.copy(error = e.message, isLoading = false)
0030             }
0031         }
0032     }
0033 }
0034 }
```

KnowledgeDetailViewModel.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/viewmodel/KnowledgeDetailViewModel.kt

Purpose: Loads, updates, and deletes selected note data.

```
0001 package com.example.mobile.ui.viewmodel
0002
0003 import androidx.compose.runtime.getValue
0004 import androidx.compose.runtime.mutableStateOf
0005 import androidx.compose.runtime.setValue
0006 import androidx.lifecycle.ViewModel
0007 import androidx.lifecycle.ViewModelScope
0008 import com.example.mobile.data.repository.NSyncRepository
0009 import com.example.mobile.ui.state.KnowledgeDetailUiState
0010 import kotlinx.coroutines.launch
0011
0012 class KnowledgeDetailViewModel : ViewModel() {
0013     private val repository = NSyncRepository()
0014
0015     var uiState by mutableStateOf(KnowledgeDetailUiState())
0016     private set
0017
0018     fun loadNote(id: Int) {
0019         viewModelScope.launch {
0020             uiState = uiState.copy(isLoading = true, error = null, deleted = false)
0021             try {
0022                 val item = repository.getKnowledgeItemById(id)
0023                 uiState = if (item != null) {
0024                     uiState.copy(item = item, isLoading = false)
0025                 } else {
0026                     uiState.copy(error = "Note not found.", isLoading = false)
0027                 }
0028             } catch (e: Exception) {
0029                 uiState = uiState.copy(error = e.message, isLoading = false)
0030             }
0031         }
0032     }
0033
0034     fun deleteNote(id: Int) {
0035         viewModelScope.launch {
0036             uiState = uiState.copy(isDeleting = true, error = null)
0037             try {
0038                 val deleted = repository.deleteNote(id)
0039                 uiState = if (deleted) {
0040                     uiState.copy(isDeleting = false, deleted = true)
0041                 } else {
0042                     uiState.copy(isDeleting = false, error = "Unable to delete note.")
0043                 }
0044             } catch (e: Exception) {
0045                 uiState = uiState.copy(isDeleting = false, error = e.message)
0046             }
0047         }
0048     }
0049 }
```

MasteryViewModel.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/viewmodel/MasteryViewModel.kt

Purpose: Loads progress data and maps it into mastery sections.

```
0001 package com.example.mobile.ui.viewmodel
0002
0003 import androidx.compose.runtime.getValue
0004 import androidx.compose.runtime.mutableStateOf
0005 import androidx.compose.runtime.setValue
0006 import androidx.lifecycle.ViewModel
0007 import androidx.lifecycle.ViewModelScope
0008 import com.example.mobile.data.CollectionMastery
0009 import com.example.mobile.data.repository.NSyncRepository
0010 import com.example.mobile.ui.state.MasteryUiState
0011 import kotlinx.coroutines.launch
0012 import kotlin.math.roundToInt
0013
0014 class MasteryViewModel : ViewModel() {
0015     private val repository = NSyncRepository()
0016
0017     var uiState by mutableStateOf(MasteryUiState())
0018     private set
0019
0020     init {
0021         loadMastery()
0022     }
0023
0024     fun loadMastery() {
0025         viewModelScope.launch {
0026             uiState = MasteryUiState(isLoading = true)
0027             try {
0028                 val progress = repository.getProgress()
0029                 val reviewCards = repository.getReviewCards()
0030                 val masteryGroups = reviewCards
0031                     .groupBy { it.collection }
0032                     .map { (collection, cards) ->
0033                         val noteCount = cards
0034                             .map { it.sourceNoteTitle }
0035                             .distinct()
0036                             .size
0037
0038                         CollectionMastery(
0039                             collection = collection,
0040                             context = "${cards.size} cards across $noteCount notes",
0041                             masteryPercent = cards
0042                                 .map { it.masteryPercent }
0043                                 .average()
0044                                 .roundToInt()
0045                         )
0046                     }
0047                 uiState = MasteryUiState(
0048                     progress = progress,
0049                     masteryGroups = masteryGroups
0050                 )
0051             } catch (e: Exception) {
0052                 uiState = MasteryUiState(isLoading = false, error = e.message)
0053             }
0054         }
0055     }
0056 }
```

NewFlashcardViewModel.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/viewmodel/NewFlashcardViewModel.kt

Purpose: Handles create-flashcard form state and save logic.

```
0001 package com.example.mobile.ui.viewmodel
0002
0003 import androidx.compose.runtime.getValue
0004 import androidx.compose.runtime.mutableStateOf
0005 import androidx.compose.runtime.setValue
0006 import androidx.lifecycle.ViewModel
0007 import androidx.lifecycle.ViewModelScope
0008 import com.example.mobile.data.ReviewCard
0009 import com.example.mobile.data.repository.NSyncRepository
0010 import kotlinx.coroutines.launch
0011
0012 class NewFlashcardViewModel : ViewModel() {
0013     private val repository = NSyncRepository()
0014
0015     var isSaving by mutableStateOf(false)
0016     private set
0017
0018     var error by mutableStateOf<String?>(null)
0019     private set
0020
0021     var saved by mutableStateOf(false)
0022     private set
0023
0024     var loadedCard by mutableStateOf<ReviewCard?>(null)
0025     private set
0026
0027     fun saveFlashcard(noteId: Int, question: String, answer: String, difficulty: String) {
0028         if (question.isBlank() || answer.isBlank()) {
0029             error = "Question and answer are required."
0030             return
0031         }
0032
0033         viewModelScope.launch {
0034             isSaving = true
0035             error = null
0036             saved = false
0037
0038             val card = repository.createFlashcard(noteId, question, answer, difficulty)
0039             if (card != null) {
0040                 saved = true
0041             } else {
0042                 error = "Unable to save review card."
0043             }
0044
0045             isSaving = false
0046         }
0047     }
0048
0049     fun loadCard(cardId: Int) {
0050         if (loadedCard?.id == cardId) return
0051
0052         viewModelScope.launch {
0053             loadedCard = repository.getReviewCardById(cardId)
0054             if (loadedCard == null) error = "Unable to load review card."
0055         }
0056     }
0057
0058     fun updateFlashcard(card: ReviewCard, question: String, answer: String, difficulty: String) {
0059         if (question.isBlank() || answer.isBlank()) {
0060             error = "Question and answer are required."
0061             return
0062         }
0063
0064         viewModelScope.launch {
0065             isSaving = true
0066             error = null
0067             saved = false
0068
0069             val updated = repository.updateFlashcard(
0070                 id = card.id,
0071                 noteId = card.knowledgeItemId,
0072                 question = question,
0073                 answer = answer,
0074                 difficulty = difficulty,
0075                 masteryLevel = when (card.masteryLabel) {
0076                     "Mastered" -> 3
0077                     "Familiar" -> 2
0078                     "Learning" -> 1
0079                     else -> 0
0080                 }
0081             )
0082             if (updated != null) saved = true else error = "Unable to update review card."
0083             isSaving = false
0084         }
0085     }
0086 }
```


NewNoteViewModel.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/viewmodel/NewNoteViewModel.kt

Purpose: Handles create-note form state and save logic.

```
0001 package com.example.mobile.ui.viewmodel
0002
0003 import androidx.compose.runtime.getValue
0004 import androidx.compose.runtime.mutableStateOf
0005 import androidx.compose.runtime.setValue
0006 import androidx.lifecycle.ViewModel
0007 import androidx.lifecycle.ViewModelScope
0008 import com.example.mobile.data.repository.NSyncRepository
0009 import kotlinx.coroutines.launch
0010
0011
0012 class NewNoteViewModel : ViewModel() {
0013     private val repository = NSyncRepository()
0014
0015     var isSaving by mutableStateOf(false)
0016     private set
0017
0018     var error by mutableStateOf<String?>(null)
0019     private set
0020
0021     var saved by mutableStateOf(false)
0022     private set
0023
0024     var title by mutableStateOf("")
0025     private set
0026
0027     var tag by mutableStateOf("")
0028     private set
0029
0030     var content by mutableStateOf("")
0031     private set
0032
0033     var loaded by mutableStateOf(false)
0034     private set
0035
0036     fun createNote(title: String, content: String, tag: String) {
0037         viewModelScope.launch {
0038             isSaving = true
0039             error = null
0040             saved = false
0041
0042             val createdNote = repository.createNote(title, content, tag)
0043             if (createdNote != null) {
0044                 saved = true
0045             } else {
0046                 error = "Unable to save note."
0047             }
0048
0049             isSaving = false
0050         }
0051     }
0052
0053     fun loadNote(id: Int) {
0054         if (loaded) return
0055
0056         viewModelScope.launch {
0057             error = null
0058             val item = repository.getKnowledgeItemById(id)
0059             if (item != null) {
0060                 title = item.title
0061                 tag = item.collection
0062                 content = item.fullNote
0063                 loaded = true
0064             } else {
0065                 error = "Unable to load note."
0066             }
0067         }
0068     }
0069
0070     fun updateNote(id: Int, title: String, content: String, tag: String) {
0071         viewModelScope.launch {
0072             isSaving = true
0073             error = null
0074             saved = false
0075
0076             val updatedNote = repository.updateNote(id, title, content, tag)
0077             if (updatedNote != null) {
0078                 saved = true
0079             } else {
0080                 error = "Unable to update note."
0081             }
0082
0083             isSaving = false
0084         }
0085     }
0086 }
```

ProfileViewModel.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/viewmodel/ProfileViewModel.kt

Purpose: Loads profile identity and learning progress data.

```
0001 package com.example.mobile.ui.viewmodel
0002
0003 import androidx.compose.runtime.getValue
0004 import androidx.compose.runtime.mutableStateOf
0005 import androidx.compose.runtime.setValue
0006 import androidx.lifecycle.ViewModel
0007 import androidx.lifecycle.ViewModelScope
0008 import com.example.mobile.data.local.AuthSession
0009 import com.example.mobile.data.remote.RetrofitClient
0010 import com.example.mobile.data.repository.NSyncRepository
0011 import com.example.mobile.ui.state.ProfileUiState
0012 import kotlinx.coroutines.flow.first
0013 import kotlinx.coroutines.launch
0014
0015
0016 class ProfileViewModel : ViewModel() {
0017     private val repository = NSyncRepository()
0018
0019     var uiState by mutableStateOf(ProfileUiState())
0020     private set
0021
0022     init {
0023         loadProfile()
0024     }
0025
0026     fun loadProfile() {
0027         viewModelScope.launch {
0028             val session = RetrofitClient.sessionStore.sessionFlow.first()
0029             uiState = ProfileUiState(
0030                 isLoading = true,
0031                 displayName = session.profileName(),
0032                 email = session?.email.orEmpty()
0033             )
0034             try {
0035                 val progress = repository.getProgress()
0036                 uiState = ProfileUiState(
0037                     displayName = session.profileName(),
0038                     email = session?.email.orEmpty(),
0039                     progress = progress
0040                 )
0041             } catch (e: Exception) {
0042                 uiState = ProfileUiState(
0043                     displayName = session.profileName(),
0044                     email = session?.email.orEmpty(),
0045                     error = e.message ?: "Unable to load profile."
0046                 )
0047             }
0048         }
0049     }
0050
0051     private fun AuthSession?.profileName(): String {
0052         return this?.displayName
0053             ?.trim()
0054             ?.takeIf { it.isNotEmpty() }
0055             ?: this?.email
0056                 ?.substringBefore("@")
0057                 ?.takeIf { it.isNotEmpty() }
0058             ?: "User"
0059     }
0060 }
```

ReviewCardsViewModel.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/viewmodel/ReviewCardsViewModel.kt

Purpose: Loads flashcards and groups them into note-based review sessions.

```
0001 package com.example.mobile.ui.viewmodel
0002
0003 import androidx.compose.runtime.getValue
0004 import androidx.compose.runtime.mutableStateOf
0005 import androidx.compose.runtime.setValue
0006 import androidx.lifecycle.ViewModel
0007 import androidx.lifecycle.ViewModelScope
0008 import com.example.mobile.data.ReviewCard
0009 import com.example.mobile.data.repository.NSyncRepository
0010 import kotlinx.coroutines.launch
0011
0012 class ReviewCardsViewModel : ViewModel() {
0013     private val repository = NSyncRepository()
0014
0015     var isLoading by mutableStateOf(false)
0016     private set
0017
0018     var error by mutableStateOf<String?>(null)
0019     private set
0020
0021     var cards by mutableStateOf<List<ReviewCard>>(emptyList())
0022     private set
0023
0024     init {
0025         loadCards()
0026     }
0027
0028     fun loadCards() {
0029         viewModelScope.launch {
0030             isLoading = true
0031             error = null
0032
0033             try {
0034                 cards = repository.getReviewCards()
0035             } catch (e: Exception) {
0036                 cards = emptyList()
0037                 error = e.message ?: "Could not load review cards."
0038             } finally {
0039                 isLoading = false
0040             }
0041         }
0042     }
0043 }
```

ReviewSessionViewModel.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/viewmodel/ReviewSessionViewModel.kt

Purpose: Controls review-card order, answer reveal, recall rating, and completion payload.

```
0001 package com.example.mobile.ui.viewmodel
0002
0003 import androidx.compose.runtime.getValue
0004 import androidx.compose.runtime.mutableStateOf
0005 import androidx.compose.runtime.setValue
0006 import androidx.lifecycle.ViewModel
0007 import androidx.lifecycle.ViewModelScope
0008 import com.example.mobile.data.ReviewCard
0009 import com.example.mobile.data.repository.NSyncRepository
0010 import com.example.mobile.data.remote.dto.ReviewAnswerDto
0011 import kotlinx.coroutines.launch
0012
0013 data class ReviewSessionResult(
0014     val score: Int,
0015     val totalQuestions: Int,
0016     val xpEarned: Int
0017 )
0018
0019 class ReviewSessionViewModel : ViewModel() {
0020     private val repository = NSyncRepository()
0021
0022     var cards by mutableStateOf<List<ReviewCard>>(emptyList())
0023     private set
0024
0025     var currentIndex by mutableStateOf(0)
0026     private set
0027
0028     var isLoading by mutableStateOf(false)
0029     private set
0030
0031     var error by mutableStateOf<String?>(null)
0032     private set
0033
0034     var correctAnswers by mutableStateOf(0)
0035     private set
0036
0037     var isCompleting by mutableStateOf(false)
0038     private set
0039
0040     var completedResult by mutableStateOf<ReviewSessionResult?>(null)
0041     private set
0042
0043     private val answers = mutableListOf<ReviewAnswerDto>()
0044
0045     val currentCard: ReviewCard?
0046         get() = cards.getOrNull(currentIndex)
0047
0048     val hasNextCard: Boolean
0049         get() = currentIndex < cards.lastIndex
0050
0051     fun loadSession(noteId: Int? = null, cardId: Int? = null) {
0052         viewModelScope.launch {
0053             isLoading = true
0054             error = null
0055             currentIndex = 0
0056             correctAnswers = 0
0057             completedResult = null
0058             isCompleting = false
0059             answers.clear()
0060             try {
0061                 val reviewCards = repository.getReviewCards()
0062                 cards = when {
0063                     noteId != null -> reviewCards.filter { it.knowledgeItemId == noteId }
0064                     cardId != null -> reviewCards.filter { it.id == cardId }
0065                     else -> reviewCards
0066                 }
0067             } catch (e: Exception) {
0068                 cards = emptyList()
0069                 error = e.message
0070             } finally {
0071                 isLoading = false
0072             }
0073         }
0074     }
0075
0076     fun nextCard() {
0077         if (hasNextCard) currentIndex += 1
0078     }
0079
0080     fun recordAnswer(recalled: Boolean) {
0081         val card = currentCard ?: return
0082         if (isCompleting) return
0083
0084         answers += ReviewAnswerDto(card.id, recalled = recalled)
0085         if (recalled) correctAnswers += 1
0086
0087         if (hasNextCard) {
0088             nextCard()
0089             return
0090         }
0091
0092         completeSession()
0093     }
0094
0095     fun consumeCompletedResult() {
0096         completedResult = null
0097     }
0098 }
```

```
0099     private fun completeSession() {
0100         viewModelScope.launch {
0101             isCompleting = true
0102             error = null
0103             try {
0104                 val response = repository.completeReview(answers.toList())
0105                 completedResult = ReviewSessionResult(
0106                     score = response.attempt.score,
0107                     totalQuestions = response.attempt.totalQuestions,
0108                     xpEarned = response.attempt.xpEarned
0109                 )
0110             } catch (e: Exception) {
0111                 error = e.message ?: "Unable to save review results."
0112             } finally {
0113                 isCompleting = false
0114             }
0115         }
0116     }
0117 }
```

SessionCompleteViewModel.kt

Path: mobile/app/src/main/java/com/example/mobile/ui/viewmodel/SessionCompleteViewModel.kt

Purpose: Loads session completion/progress data after a review.

```
0001 package com.example.mobile.ui.viewmodel
0002
0003 import androidx.compose.runtime.getValue
0004 import androidx.compose.runtime.mutableStateOf
0005 import androidx.compose.runtime.setValue
0006 import androidx.lifecycle.ViewModel
0007 import androidx.lifecycle.ViewModelScope
0008 import com.example.mobile.data.repository.NSyncRepository
0009 import com.example.mobile.ui.state.SessionCompleteUiState
0010 import kotlinx.coroutines.launch
0011
0012 class SessionCompleteViewModel : ViewModel() {
0013     private val repository = NSyncRepository()
0014
0015     var uiState by mutableStateOf(SessionCompleteUiState())
0016     private set
0017
0018     fun loadProgress() {
0019         if (uiState.isLoading) return
0020
0021         viewModelScope.launch {
0022             uiState = SessionCompleteUiState(isLoading = true)
0023             try {
0024                 uiState = SessionCompleteUiState(
0025                     progress = repository.getProgress()
0026                 )
0027             } catch (e: Exception) {
0028                 uiState = SessionCompleteUiState(
0029                     error = e.message ?: "Unable to save review results."
0030                 )
0031             }
0032         }
0033     }
0034 }
```

Backend API Source Code

manage.py

Path: backend/manage.py

Purpose: Django command-line entry point.

```
0001 #!/usr/bin/env python
0002 """Django's command-line utility for administrative tasks."""
0003 import os
0004 import sys
0005
0006
0007 def main():
0008     """Run administrative tasks."""
0009     os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'Nsync.settings')
0010     try:
0011         from django.core.management import execute_from_command_line
0012     except ImportError as exc:
0013         raise ImportError(
0014             "Couldn't import Django. Are you sure it's installed and "
0015             "available on your PYTHONPATH environment variable? Did you "
0016             "forget to activate a virtual environment?"
0017         ) from exc
0018     execute_from_command_line(sys.argv)
0019
0020
0021 if __name__ == '__main__':
0022     main()
```


settings.py

Path: backend/NSync/settings.py

Purpose: Django settings for installed apps, PostgreSQL, JWT, CORS, and REST Framework.

```
0001 """
0002 Django settings for NSync project.
0003
0004 Generated by 'django-admin startproject' using Django 6.0.6.
0005
0006 For more information on this file, see
0007 https://docs.djangoproject.com/en/6.0/topics/settings/
0008
0009 For the full list of settings and their values, see
0010 https://docs.djangoproject.com/en/6.0/ref/settings/
0011 """
0012
0013 import os
0014 from pathlib import Path
0015 from dotenv import load_dotenv
0016 from datetime import timedelta
0017
0018 # Build paths inside the project like this: BASE_DIR / 'subdir'.
0019 BASE_DIR = Path(__file__).resolve().parent.parent
0020
0021 load_dotenv(BASE_DIR / '.env')
0022
0023 # Quick-start development settings - unsuitable for production
0024 # See https://docs.djangoproject.com/en/6.0/howto/deployment/checklist/
0025
0026 # SECURITY WARNING: keep the secret key used in production secret!
0027 SECRET_KEY = os.getenv("DJANGO_SECRET_KEY")
0028
0029
0030 # SECURITY WARNING: don't run with debug turned on in production!
0031 DEBUG = os.getenv("DJANGO_DEBUG", "False").lower() == "true"
0032
0033 ALLOWED_HOSTS = os.getenv("DJANGO_ALLOWED_HOSTS", "127.0.0.1,localhost").split(",")
0034
0035
0036 # Application definition
0037
0038 INSTALLED_APPS = [
0039     'django.contrib.admin',
0040     'django.contrib.auth',
0041     'django.contrib.contenttypes',
0042     'django.contrib.sessions',
0043     'django.contrib.messages',
0044     'django.contrib.staticfiles',
0045     'rest_framework',
0046     'rest_framework_simplejwt.token_blacklist',
0047     'core',
0048 ]
0049
0050 REST_FRAMEWORK = {
0051     "DEFAULT_AUTHENTICATION_CLASSES": (
0052         "rest_framework_simplejwt.authentication.JWTAuthentication",
0053     ),
0054     "DEFAULT_PERMISSION_CLASSES": [
0055         "rest_framework.permissions.IsAuthenticated",
0056     ]
0057 }
0058
0059 SIMPLE_JWT = {
0060     "ACCESS_TOKEN_LIFETIME": timedelta(minutes=15),
0061     "REFRESH_TOKEN_LIFETIME": timedelta(days=7),
0062     "ROTATE_REFRESH_TOKENS": True,
0063     "BLACKLIST_AFTER_ROTATION": True,
0064     "AUTH_HEADER_TYPES": ("Bearer",),
0065 }
0066
0067 MIDDLEWARE = [
0068     'django.middleware.security.SecurityMiddleware',
0069     'django.contrib.sessions.middleware.SessionMiddleware',
0070     'django.middleware.common.CommonMiddleware',
0071     'django.middleware.csrf.CsrfViewMiddleware',
0072     'django.contrib.auth.middleware.AuthenticationMiddleware',
0073     'django.contrib.messages.middleware.MessageMiddleware',
0074     'django.middleware.clickjacking.XFrameOptionsMiddleware',
0075 ]
0076
0077 ROOT_URLCONF = 'NSync.urls'
0078
0079 TEMPLATES = [
0080     {
0081         'BACKEND': 'django.template.backends.django.DjangoTemplates',
0082         'DIRS': [],
0083         'APP_DIRS': True,
0084         'OPTIONS': {
0085             'context_processors': [
0086                 'django.template.context_processors.request',
0087                 'django.contrib.auth.context_processors.auth',
0088                 'django.contrib.messages.context_processors.messages',
0089             ],
0090         },
0091     },
0092 ]
0093
0094 WSGI_APPLICATION = 'NSync.wsgi.application'
0095
0096
0097 # Database
0098 # https://docs.djangoproject.com/en/6.0/ref/settings/#databases
```

```
0099
0100 DATABASES = {
0101     "default": {
0102         "ENGINE": "django.db.backends.postgresql",
0103         "NAME": os.getenv("DB_NAME"),
0104         "USER": os.getenv("DB_USER"),
0105         "PASSWORD": os.getenv("DB_PASSWORD"),
0106         "HOST": os.getenv("DB_HOST", "127.0.0.1"),
0107         "PORT": os.getenv("DB_PORT", "5432"),
0108     }
0109 }
0110
0111
0112 # Password validation
0113 # https://docs.djangoproject.com/en/6.0/ref/settings/#auth-password-validators
0114
0115 AUTH_PASSWORD_VALIDATORS = [
0116     {
0117         'NAME': 'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',
0118     },
0119     {
0120         'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',
0121     },
0122     {
0123         'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator',
0124     },
0125     {
0126         'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator',
0127     },
0128 ]
0129
0130
0131 # Internationalization
0132 # https://docs.djangoproject.com/en/6.0/topics/i18n/
0133
0134 LANGUAGE_CODE = 'en-us'
0135
0136 TIME_ZONE = 'UTC'
0137
0138 USE_I18N = True
0139
0140 USE_TZ = True
0141
0142
0143 # Static files (CSS, JavaScript, Images)
0144 # https://docs.djangoproject.com/en/6.0/howto/static-files/
0145
0146 STATIC_URL = 'static/'
```

urls.py

Path: backend/NSync/urls.py

Purpose: Project-level URL routing.

```
0001 """
0002 URL configuration for NSync project.
0003
0004 The `urlpatterns` list routes URLs to views. For more information please see:
0005     https://docs.djangoproject.com/en/6.0/topics/http/urls/
0006 Examples:
0007 Function views
0008     1. Add an import:  from my_app import views
0009     2. Add a URL to urlpatterns:  path('', views.home, name='home')
0010 Class-based views
0011     1. Add an import:  from other_app.views import Home
0012     2. Add a URL to urlpatterns:  path('', Home.as_view(), name='home')
0013 Including another URLconf
0014     1. Import the include() function: from django.urls import include, path
0015     2. Add a URL to urlpatterns:  path('blog/', include('blog.urls'))
0016 """
0017 from django.contrib import admin
0018 from django.urls import path, include
0019
0020 urlpatterns = [
0021     path('admin/', admin.site.urls),
0022     path('core/', include('core.urls')),
0023     path('api/', include('core.api_urls')),
0024 ]
```

models.py

Path: backend/core/models.py

Purpose: Database models for users, profiles, notes, flashcards, quiz attempts, and progress.

```
0001 from django.core.validators import MaxValueValidator, MinValueValidator
0002 from django.db import models
0003 from django.conf import settings
0004
0005 # Create your models here.
0006
0007 class User(models.Model):
0008     name = models.CharField(max_length=200)
0009     email = models.EmailField(blank=True)
0010
0011 class UserProfile(models.Model):
0012     user = models.OneToOneField(
0013         settings.AUTH_USER_MODEL,
0014         on_delete=models.CASCADE,
0015         related_name="profile"
0016     )
0017     display_name = models.CharField(max_length=200)
0018
0019 class Note(models.Model):
0020     owner = models.ForeignKey(
0021         settings.AUTH_USER_MODEL,
0022         on_delete=models.CASCADE,
0023         related_name="notes",
0024     )
0025     title = models.CharField(max_length=200)
0026     content = models.TextField()
0027     tag = models.CharField(max_length=100, blank=True)
0028     created_at = models.DateTimeField(auto_now_add=True)
0029     updated_at = models.DateTimeField(auto_now=True)
0030
0031 class Flashcard(models.Model):
0032     connected_note = models.ForeignKey(
0033         Note,
0034         on_delete=models.CASCADE,
0035         related_name="flashcards"
0036     )
0037     question = models.TextField()
0038     answer = models.TextField()
0039     difficulty = models.CharField(max_length=50, blank=True)
0040     mastery_level = models.IntegerField(
0041         default=0,
0042         validators=[MinValueValidator(0), MaxValueValidator(3)],
0043     )
0044
0045
0046
0047 class QuizAttempt(models.Model):
0048     user = models.ForeignKey(
0049         settings.AUTH_USER_MODEL,
0050         on_delete=models.CASCADE,
0051         related_name="quiz_attempts",
0052     )
0053     score = models.IntegerField(default=0)
0054     total_questions = models.PositiveIntegerField(default=0)
0055     xp_earned = models.PositiveIntegerField(default=0)
0056     date_taken = models.DateTimeField(auto_now_add=True)
0057
0058 class UserProgress(models.Model):
0059     user = models.OneToOneField(
0060         settings.AUTH_USER_MODEL,
0061         on_delete=models.CASCADE,
0062         related_name="progress",
0063     )
0064     total_xp = models.PositiveIntegerField(default=0)
0065     level = models.PositiveIntegerField(default=1)
0066     streak = models.PositiveIntegerField(default=0)
0067     last_reviewed_on = models.DateField(null=True, blank=True)
0068     total_reviews = models.PositiveIntegerField(default=0)
0069     correct_reviews = models.PositiveIntegerField(default=0)
0070     accuracy = models.FloatField(default=0.0)
```

serializers.py

Path: backend/core/serializers.py

Purpose: DRF serializers for model JSON conversion and auth payloads.

```
0001 from rest_framework import serializers
0002 from .models import User, Note, Flashcard, QuizAttempt, UserProgress, UserProfile
0003 from django.contrib.auth import authenticate, get_user_model
0004
0005
0006 class UserSerializer(serializers.ModelSerializer):
0007     class Meta:
0008         model = User
0009         fields = [
0010             'id',
0011             'name',
0012             'email',
0013         ]
0014
0015
0016 class NoteSerializer(serializers.ModelSerializer):
0017     class Meta:
0018         model = Note
0019         fields = [
0020             'id',
0021             'title',
0022             'content',
0023             'tag',
0024             'created_at',
0025             'updated_at',
0026         ]
0027
0028
0029 class FlashcardSerializer(serializers.ModelSerializer):
0030     note_title = serializers.CharField(source='connected_note.title', read_only=True)
0031     note_tag = serializers.CharField(source='connected_note.tag', read_only=True)
0032
0033     class Meta:
0034         model = Flashcard
0035         fields = [
0036             'id',
0037             'connected_note',
0038             'question',
0039             'answer',
0040             'note_title',
0041             'note_tag',
0042             'difficulty',
0043             'mastery_level',
0044         ]
0045
0046
0047 class QuizAttemptSerializer(serializers.ModelSerializer):
0048     class Meta:
0049         model = QuizAttempt
0050         fields = [
0051             'id',
0052             'score',
0053             'total_questions',
0054             'xp_earned',
0055             'date_taken',
0056         ]
0057
0058 class UserProgressSerializer(serializers.ModelSerializer):
0059     class Meta:
0060         model = UserProgress
0061         fields = [
0062             'id',
0063             'total_xp',
0064             'level',
0065             'streak',
0066             'total_reviews',
0067             'correct_reviews',
0068             'accuracy',
0069         ]
0070
0071 class AuthenticatedUserSerializer(serializers.ModelSerializer):
0072     display_name = serializers.CharField(
0073         source="profile.display_name",
0074         read_only=True
0075     )
0076
0077     class Meta:
0078         model = get_user_model()
0079         fields = [
0080             'id',
0081             'email',
0082             'display_name',
0083         ]
0084
0085 class RegisterSerializer(serializers.Serializer):
0086     display_name = serializers.CharField(max_length=200)
0087     email = serializers.EmailField()
0088     password = serializers.CharField(
0089         write_only=True,
0090         trim_whitespace=False,
0091     )
0092
0093     def validate_email(self, value):
0094         email = value.strip().lower()
0095         user_model = get_user_model()
0096
0097         if user_model.objects.filter(username=email).exists():
0098             raise serializers.ValidationError(
```

```

0099         "An account with this email already exists."
0100     )
0101     return email
0102
0103     def create(self, validated_data):
0104         user_model = get_user_model()
0105         email = validated_data["email"]
0106
0107         user = user_model.objects.create_user(
0108             username=email,
0109             email=email,
0110             password=validated_data["password"]
0111         )
0112
0113         UserProfile.objects.create(
0114             user=user,
0115             display_name=validated_data["display_name"]
0116         )
0117
0118         return user
0119
0120     class LoginSerializer(serializers.Serializer):
0121         email = serializers.EmailField()
0122         password = serializers.CharField(
0123             write_only=True,
0124             trim_whitespace=False,
0125         )
0126
0127     def validate(self, attrs):
0128         email = attrs["email"].strip().lower()
0129         user = authenticate(
0130             username=email,
0131             password=attrs["password"]
0132         )
0133
0134         if user is None or not user.is_active:
0135             raise serializers.ValidationError(
0136                 "Invalid email or password."
0137             )
0138
0139         attrs["user"] = user
0140         return attrs

```

api_views.py

Path: backend/core/api_views.py

Purpose: DRF API views and viewsets for auth, notes, flashcards, progress, and review completion.

```
0001 from datetime import timedelta
0002
0003 from django.utils import timezone
0004 from rest_framework import status, viewsets
0005 from rest_framework.decorators import api_view, permission_classes
0006 from rest_framework.permissions import AllowAny, IsAuthenticated
0007 from rest_framework.response import Response
0008 from rest_framework_simplejwt.exceptions import TokenError
0009 from rest_framework_simplejwt.tokens import RefreshToken
0010 from rest_framework.exceptions import PermissionDenied
0011
0012 from .models import Note, Flashcard, QuizAttempt, UserProgress
0013 from .serializers import (
0014     NoteSerializer,
0015     FlashcardSerializer,
0016     QuizAttemptSerializer,
0017     UserProgressSerializer,
0018     RegisterSerializer,
0019     LoginSerializer,
0020     AuthenticatedUserSerializer,
0021 )
0022
0023
0024 def build_auth_response(user):
0025     refresh = RefreshToken.for_user(user)
0026
0027     return {
0028         "refresh": str(refresh),
0029         "access": str(refresh.access_token),
0030         "user": AuthenticatedUserSerializer(user).data,
0031     }
0032
0033
0034 @api_view(["GET"])
0035 @permission_classes([AllowAny])
0036 def health_check(request):
0037     return Response({"status": "ok"})
0038
0039 class NoteViewSet(viewsets.ModelViewSet):
0040     queryset = Note.objects.order_by("-updated_at")
0041     serializer_class = NoteSerializer
0042     permission_classes = [IsAuthenticated]
0043
0044     def get_queryset(self):
0045         return Note.objects.filter(owner=self.request.user).order_by("-updated_at")
0046
0047     def perform_create(self, serializer):
0048         serializer.save(owner=self.request.user)
0049
0050 class FlashcardViewSet(viewsets.ModelViewSet):
0051     queryset = Flashcard.objects.all()
0052     serializer_class = FlashcardSerializer
0053     permission_classes = [IsAuthenticated]
0054
0055     def get_queryset(self):
0056         queryset = Flashcard.objects.filter(
0057             connected_note__owner=self.request.user
0058         ).order_by("id")
0059         note_id = self.request.query_params.get("note")
0060
0061         if note_id:
0062             queryset = queryset.filter(connected_note_id=note_id)
0063
0064         return queryset
0065
0066     def perform_create(self, serializer):
0067         connected_note = serializer.validated_data["connected_note"]
0068
0069         if connected_note.owner != self.request.user:
0070             raise PermissionDenied(
0071                 "You do not have permission to add flashcards to this note."
0072             )
0073
0074         serializer.save()
0075
0076 @api_view(["GET"])
0077 @permission_classes([IsAuthenticated])
0078 def progress_detail(request):
0079     progress, created = UserProgress.objects.get_or_create(user=request.user)
0080     serializer = UserProgressSerializer(progress)
0081     return Response(serializer.data)
0082
0083
0084 @api_view(["POST"])
0085 @permission_classes([IsAuthenticated])
0086 def complete_review(request):
0087     answers = request.data.get("answers")
0088
0089     if isinstance(answers, list):
0090         flashcard_ids = [answer.get("flashcard_id") for answer in answers]
0091
0092         if len(flashcard_ids) != len(set(flashcard_ids)):
0093             return Response(
0094                 {"detail": "Duplicate flashcard answers are not allowed."},
0095                 status=status.HTTP_400_BAD_REQUEST,
0096             )
0097
0098     owned_flashcards = Flashcard.objects.filter(
```

```

0099         id_in=flashcard_ids,
0100         connected_note__owner=request.user,
0101     )
0102     flashcards_by_id = {flashcard.id: flashcard for flashcard in owned_flashcards}
0103
0104     if len(flashcards_by_id) != len(set(flashcard_ids)):
0105         return Response(
0106             {"detail": "One or more flashcards do not belong to this user."},
0107             status=status.HTTP_403_FORBIDDEN,
0108         )
0109
0110     score = 0
0111     for answer in answers:
0112         flashcard = flashcards_by_id.get(answer.get("flashcard_id"))
0113         recalled = bool(answer.get("recalled", False))
0114
0115         if recalled:
0116             score += 1
0117             flashcard.mastery_level = min(3, flashcard.mastery_level + 1)
0118         else:
0119             flashcard.mastery_level = max(0, flashcard.mastery_level - 1)
0120
0121         flashcard.save(update_fields=["mastery_level"])
0122
0123     total_questions = len(answers)
0124     xp_earned = total_questions * 25
0125
0126     else:
0127         score = int(request.data.get("score", 0))
0128         total_questions = int(request.data.get("total_questions", 0))
0129         xp_earned = int(request.data.get("xp_earned", 0))
0130
0131     attempt = QuizAttempt.objects.create(
0132         user=request.user,
0133         score=score,
0134         total_questions=total_questions,
0135         xp_earned=xp_earned,
0136     )
0137
0138     progress, created = UserProgress.objects.get_or_create(user=request.user)
0139
0140     progress.total_xp += xp_earned
0141     progress.total_reviews += total_questions
0142     progress.correct_reviews += score
0143
0144     today = timezone.localdate()
0145     if progress.last_reviewed_on == today:
0146         pass
0147     elif progress.last_reviewed_on == today - timedelta(days=1):
0148         progress.streak += 1
0149     else:
0150         progress.streak = 1
0151     progress.last_reviewed_on = today
0152
0153     if progress.total_reviews > 0:
0154         progress.accuracy = progress.correct_reviews / progress.total_reviews * 100
0155     else:
0156         progress.accuracy = 0
0157
0158     progress.level = calculate_level(progress.total_xp)
0159     progress.save()
0160
0161     return Response(
0162         {
0163             "attempt": QuizAttemptSerializer(attempt).data,
0164             "progress": UserProgressSerializer(progress).data,
0165         },
0166         status=status.HTTP_201_CREATED,
0167     )
0168
0169 def calculate_level(total_xp):
0170     if total_xp >= 1000:
0171         return 5
0172     if total_xp >= 500:
0173         return 4
0174     if total_xp >= 250:
0175         return 3
0176     if total_xp >= 100:
0177         return 2
0178     return 1
0179
0180
0181 @api_view(["POST"])
0182 @permission_classes([AllowAny])
0183 def register(request):
0184     input_serializer = RegisterSerializer(data=request.data)
0185     input_serializer.is_valid(raise_exception=True)
0186     user = input_serializer.save()
0187
0188     return Response(build_auth_response(user), status=status.HTTP_201_CREATED)
0189
0190 @api_view(["POST"])
0191 @permission_classes([AllowAny])
0192 def login(request):
0193     serializer = LoginSerializer(data=request.data)
0194     serializer.is_valid(raise_exception=True)
0195
0196     user = serializer.validated_data["user"]
0197     return Response(build_auth_response(user), status=status.HTTP_200_OK)
0198
0199 @api_view(["POST"])
0200 @permission_classes([IsAuthenticated])
0201 def logout(request):
0202     try:
0203         refresh_token = request.data.get("refresh")
0204
0205         if not refresh_token:
0206             return Response(
0207                 {"detail": "Refresh token is required in the request body."},
0208                 status=status.HTTP_400_BAD_REQUEST
0209             )
0210

```



```
0211         token = RefreshToken(refresh_token)
0212         token.blacklist()
0213
0214         return Response(status=status.HTTP_204_NO_CONTENT)
0215     except TokenError:
0216         return Response(
0217             {"detail": "Invalid or expired refresh token."},
0218             status=status.HTTP_400_BAD_REQUEST
0219         )
0220
0221 @api_view(["GET"])
0222 @permission_classes([IsAuthenticated])
0223 def me(request):
0224     serializer = AuthenticatedUserSerializer(request.user)
0225     return Response(serializer.data)
```

api_urls.py

Path: backend/core/api_urls.py

Purpose: API route definitions and router registration.

```
0001 from django.urls import path, include
0002 from rest_framework.routers import DefaultRouter
0003 from rest_framework_simplejwt.views import TokenRefreshView
0004
0005 from . import api_views
0006
0007 router = DefaultRouter()
0008
0009 router.register("notes", api_views.NoteViewSet, basename="notes")
0010 router.register("flashcards", api_views.FlashcardViewSet, basename="flashcards")
0011
0012 urlpatterns = [
0013     path("health/", api_views.health_check, name="api-health"),
0014     path("auth/register/", api_views.register, name="api-register"),
0015     path("auth/login/", api_views.login, name="api-login"),
0016     path("auth/refresh/", TokenRefreshView.as_view(), name="api-token-refresh"),
0017     path("auth/logout/", api_views.logout, name="api-logout"),
0018     path("auth/me/", api_views.me, name="api-me"),
0019     path("progress/", api_views.progress_detail, name="api-progress"),
0020     path("review/complete/", api_views.complete_review, name="api-review-complete"),
0021     path("", include(router.urls)),
0022 ]
```

admin.py

Path: backend/core/admin.py

Purpose: Registers backend models in Django admin.

```
0001 from django.contrib import admin
0002 from .models import User, Note, Flashcard, QuizAttempt, UserProgress
0003
0004 # Register your models here.
0005
0006 admin.site.register(User)
0007 admin.site.register(Note)
0008 admin.site.register(Flashcard)
0009 admin.site.register(QuizAttempt)
0010 admin.site.register(UserProgress)
```

apps.py

Path: backend/core/apps.py

Purpose: Django app configuration for the core app.

```
0001 from django.apps import AppConfig
0002
0003
0004 class CoreConfig(AppConfig):
0005     name = 'core'
```

tests.py

Path: backend/core/tests.py

Purpose: Backend tests for auth, ownership, review scoring, duplicate answer rejection, and progress separation.

```
0001 from datetime import date
0002 from unittest.mock import patch
0003
0004 from django.contrib.auth import get_user_model
0005 from rest_framework import status
0006 from rest_framework.test import APITestCase
0007
0008 from .models import Flashcard, Note, UserProfile, UserProgress
0009
0010
0011 class AuthenticationTests(APITestCase):
0012     def test_register_creates_user_profile_and_tokens(self):
0013         response = self.client.post(
0014             "/api/auth/register/",
0015             {
0016                 "display_name": "Roosc",
0017                 "email": "roosc@example.com",
0018                 "password": "SecurePassword123",
0019             },
0020             format="json",
0021         )
0022
0023         self.assertEqual(response.status_code, status.HTTP_201_CREATED)
0024         self.assertIn("access", response.data)
0025         self.assertIn("refresh", response.data)
0026         self.assertEqual(response.data["user"]["email"], "roosc@example.com")
0027
0028         user = get_user_model().objects.get(email="roosc@example.com")
0029         self.assertTrue(UserProfile.objects.filter(user=user, display_name="Roosc").exists())
0030
0031     def test_login_rejects_invalid_credentials(self):
0032         user_model = get_user_model()
0033         user_model.objects.create_user(
0034             username="roosc@example.com",
0035             email="roosc@example.com",
0036             password="SecurePassword123",
0037         )
0038
0039         response = self.client.post(
0040             "/api/auth/login/",
0041             {"email": "roosc@example.com", "password": "wrong-password"},
0042             format="json",
0043         )
0044
0045         self.assertEqual(response.status_code, status.HTTP_400_BAD_REQUEST)
0046
0047
0048 class LearningDataOwnershipTests(APITestCase):
0049     def setUp(self):
0050         user_model = get_user_model()
0051         self.user_a = user_model.objects.create_user(
0052             username="user-a@example.com",
0053             email="user-a@example.com",
0054             password="SecurePassword123",
0055         )
0056         self.user_b = user_model.objects.create_user(
0057             username="user-b@example.com",
0058             email="user-b@example.com",
0059             password="SecurePassword123",
0060         )
0061         self.user_a_note = Note.objects.create(
0062             owner=self.user_a,
0063             title="User A note",
0064             content="Private knowledge.",
0065             tag="Testing",
0066         )
0067
0068     def test_notes_require_authentication(self):
0069         response = self.client.get("/api/notes/")
0070
0071         self.assertEqual(response.status_code, status.HTTP_401_UNAUTHORIZED)
0072
0073     def test_user_cannot_list_or_retrieve_another_users_note(self):
0074         self.client.force_authenticate(self.user_b)
0075
0076         list_response = self.client.get("/api/notes/")
0077         detail_response = self.client.get(f"/api/notes/{self.user_a_note.id}/")
0078
0079         self.assertEqual(list_response.status_code, status.HTTP_200_OK)
0080         self.assertEqual(list_response.data, [])
0081         self.assertEqual(detail_response.status_code, status.HTTP_404_NOT_FOUND)
0082
0083     def test_user_cannot_create_flashcard_for_another_users_note(self):
0084         self.client.force_authenticate(self.user_b)
0085
0086         response = self.client.post(
0087             "/api/flashcards/",
0088             {
0089                 "connected_note": self.user_a_note.id,
0090                 "question": "Can User B add this?",
0091                 "answer": "No.",
0092             },
0093             format="json",
0094         )
0095
0096         self.assertEqual(response.status_code, status.HTTP_403_FORBIDDEN)
0097         self.assertFalse(Flashcard.objects.exists())
0098
```

```

0099 @patch("core.api_views.timezone.localtime", return_value=date(2026, 6, 23))
0100 def test_review_progress_is_separate_for_each_user(self, _mock_localdate):
0101     self.client.force_authenticate(self.user_a)
0102     user_a_response = self.client.post(
0103         "/api/review/complete/",
0104         {"score": 2, "total_questions": 3, "xp_earned": 75},
0105         format="json",
0106     )
0107
0108     self.client.force_authenticate(self.user_b)
0109     user_b_response = self.client.post(
0110         "/api/review/complete/",
0111         {"score": 1, "total_questions": 2, "xp_earned": 25},
0112         format="json",
0113     )
0114
0115     self.assertEqual(user_a_response.status_code, status.HTTP_201_CREATED)
0116     self.assertEqual(user_b_response.status_code, status.HTTP_201_CREATED)
0117
0118     user_a_progress = UserProgress.objects.get(user=self.user_a)
0119     user_b_progress = UserProgress.objects.get(user=self.user_b)
0120     self.assertEqual(user_a_progress.total_xp, 75)
0121     self.assertEqual(user_b_progress.total_xp, 25)
0122     self.assertEqual(user_a_progress.streak, 1)
0123     self.assertEqual(user_b_progress.streak, 1)
0124
0125 @patch("core.api_views.timezone.localtime", return_value=date(2026, 6, 23))
0126 def test_review_complete_accepts_card_answers(self, _mock_localdate):
0127     first_card = Flashcard.objects.create(
0128         connected_note=self.user_a_note,
0129         question="Question 1",
0130         answer="Answer 1",
0131         mastery_level=0,
0132     )
0133     second_card = Flashcard.objects.create(
0134         connected_note=self.user_a_note,
0135         question="Question 2",
0136         answer="Answer 2",
0137         mastery_level=1,
0138     )
0139
0140     self.client.force_authenticate(self.user_a)
0141     response = self.client.post(
0142         "/api/review/complete/",
0143         {
0144             "answers": [
0145                 {"flashcard_id": first_card.id, "recalled": True},
0146                 {"flashcard_id": second_card.id, "recalled": False},
0147             ],
0148         },
0149         format="json",
0150     )
0151
0152     self.assertEqual(response.status_code, status.HTTP_201_CREATED)
0153     self.assertEqual(response.data["attempt"]["score"], 1)
0154     self.assertEqual(response.data["attempt"]["total_questions"], 2)
0155     self.assertEqual(response.data["attempt"]["xp_earned"], 50)
0156
0157     first_card.refresh_from_db()
0158     second_card.refresh_from_db()
0159     self.assertEqual(first_card.mastery_level, 1)
0160     self.assertEqual(second_card.mastery_level, 0)
0161
0162 def test_review_complete_rejects_duplicate_card_answers(self):
0163     card = Flashcard.objects.create(
0164         connected_note=self.user_a_note,
0165         question="Question",
0166         answer="Answer",
0167     )
0168
0169     self.client.force_authenticate(self.user_a)
0170     response = self.client.post(
0171         "/api/review/complete/",
0172         {
0173             "answers": [
0174                 {"flashcard_id": card.id, "recalled": True},
0175                 {"flashcard_id": card.id, "recalled": False},
0176             ],
0177         },
0178         format="json",
0179     )
0180
0181     self.assertEqual(response.status_code, status.HTTP_400_BAD_REQUEST)

```

docker-compose.yml

Path: backend/docker-compose.yml

Purpose: Docker Compose setup for local PostgreSQL.

```
0001 services:
0002   db:
0003     image: postgres:18
0004     environment:
0005       POSTGRES_DB: ${DB_NAME}
0006       POSTGRES_USER: ${DB_USER}
0007       POSTGRES_PASSWORD: ${DB_PASSWORD}
0008     ports:
0009       - "${DB_PORT}:5432"
0010     volumes:
0011       - postgres_data:/var/lib/postgresql
0012
0013 volumes:
0014   postgres_data:
```

requirements.txt

Path: backend/requirements.txt

Purpose: Python dependency list for backend development.

```
0001 asgiref==3.11.1
0002 Django==6.0.6
0003 django-cors-headers==4.9.0
0004 django-extensions==4.1
0005 django-rest-framework==3.17.1
0006 django-rest-framework-simplejwt==5.5.1
0007 gunicorn==26.0.0
0008 packaging==26.2
0009 psycopg2-binary==2.9.12
0010 PyJWT==2.13.0
0011 python-dotenv==1.2.2
0012 sqlparse==0.5.5
```


tests.http

Path: backend/tests.http

Purpose: REST Client API walkthrough requests.

```
0001 @baseUrl = http://127.0.0.1:8000/api
0002 @userAccessToken = replace-with-user-a-access-token
0003 @userRefreshToken = replace-with-user-a-refresh-token
0004 @userBAccessToken = replace-with-user-b-access-token
0005 @userNoteId = replace-with-user-a-note-id
0006
0007 ### Health check
0008 GET {{baseUrl}}/health/
0009
0010 ### Register User A
0011 POST {{baseUrl}}/auth/register/
0012 Content-Type: application/json
0013 {
0014   "display_name": "User A",
0015   "email": "user-a@example.com",
0016   "password": "SecurePassword123"
0017 }
0018
0019
0020 ### Register User B
0021 POST {{baseUrl}}/auth/register/
0022 Content-Type: application/json
0023 {
0024   "display_name": "User B",
0025   "email": "user-b@example.com",
0026   "password": "SecurePassword123"
0027 }
0028
0029
0030 ### User A creates a note
0031 POST {{baseUrl}}/notes/
0032 Authorization: Bearer {{userAccessToken}}
0033 Content-Type: application/json
0034 {
0035   "title": "User A private note",
0036   "content": "Only User A should access this.",
0037   "tag": "Testing"
0038 }
0039
0040
0041 ### User B lists notes: User A's note must not appear
0042 GET {{baseUrl}}/notes/
0043 Authorization: Bearer {{userBAccessToken}}
0044
0045 ### User B reads User A's note: expect 404
0046 GET {{baseUrl}}/notes/{{userNoteId}}/
0047 Authorization: Bearer {{userBAccessToken}}
0048
0049 ### User B creates a card in User A's note: expect 403
0050 POST {{baseUrl}}/flashcards/
0051 Authorization: Bearer {{userBAccessToken}}
0052 Content-Type: application/json
0053 {
0054   "connected_note": {{userNoteId}},
0055   "question": "Should this work?",
0056   "answer": "No."
0057 }
0058
0059
0060 ### User A completes a review
0061 POST {{baseUrl}}/review/complete/
0062 Authorization: Bearer {{userAccessToken}}
0063 Content-Type: application/json
0064 {
0065   "score": 2,
0066   "total_questions": 3,
0067   "xp_earned": 75
0068 }
0069
0070
0071 ### User B completes a review
0072 POST {{baseUrl}}/review/complete/
0073 Authorization: Bearer {{userBAccessToken}}
0074 Content-Type: application/json
0075 {
0076   "score": 1,
0077   "total_questions": 2,
0078   "xp_earned": 25
0079 }
0080
0081
0082 ### User A progress
0083 GET {{baseUrl}}/progress/
0084 Authorization: Bearer {{userAccessToken}}
0085
0086 ### User B progress
0087 GET {{baseUrl}}/progress/
0088 Authorization: Bearer {{userBAccessToken}}
0089
0090 ### User A logs out
0091 POST {{baseUrl}}/auth/logout/
0092 Authorization: Bearer {{userAccessToken}}
0093 Content-Type: application/json
0094 {
0095   "refresh": "{{userRefreshToken}}"
0096 }
0097
```

API Walkthrough

The backend exposes REST endpoints under `/api/`. Authentication uses JWT access and refresh tokens. Manual testing is documented in `backend/tests.http`, and DRF browsable API pages can be opened in a browser while the backend server is running.

1. Run PostgreSQL with Docker Compose using the backend `docker-compose` file.
2. Run Django migrations so the database has the latest tables.
3. Start the Django server with `python manage.py runserver 0.0.0.0:8000`.
4. Use POST `/api/auth/register/` to create an account.
5. Use POST `/api/auth/login/` to receive access and refresh tokens.
6. Use Authorization: Bearer `<access_token>` for protected requests.
7. Use `/api/notes/` to create and list user-owned notes.
8. Use `/api/flashcards/` to create and list flashcards connected to notes.
9. Use `/api/review/complete/` to submit review answers and update XP, streaks, accuracy, and mastery.
10. Use `/api/progress/` and `/api/auth/me/` to verify the current user and learning progress.

Final Course Learning Reflection

While developing NSync, I learned how to build a complete Android application using Kotlin and Jetpack Compose. I learned how Compose apps are built from reusable composable functions and how each function can represent a specific part of the interface, such as cards, buttons, text fields, navigation bars, and progress indicators.

I learned how navigation works in Jetpack Compose using Navigation Compose. Routes, route arguments, callbacks, and NavHost helped connect screens together without putting all navigation logic inside every screen. I also learned how Scaffold helps create a consistent app layout, especially when using bottom navigation.

Another important lesson was organizing code by responsibility. The app started as a static prototype, but the code became easier to manage after separating screens, components, state classes, ViewModels, repositories, DTOs, and theme files.

I learned how to connect a mobile app to a backend using Retrofit and Django REST Framework. The Android app sends requests to the backend, receives JSON responses, and updates UI state through ViewModels. I also learned how JWT access and refresh tokens work, and how an interceptor attaches tokens to protected API requests.

On the backend side, I learned how Django models, serializers, API views, routes, and tests work together. I also learned the importance of user ownership so that notes, flashcards, quiz attempts, and progress records belong only to the signed-in user.

Overall, this project helped me understand how a prototype can grow into a full-stack mobile application with real data, authentication, organized code, and documentation.